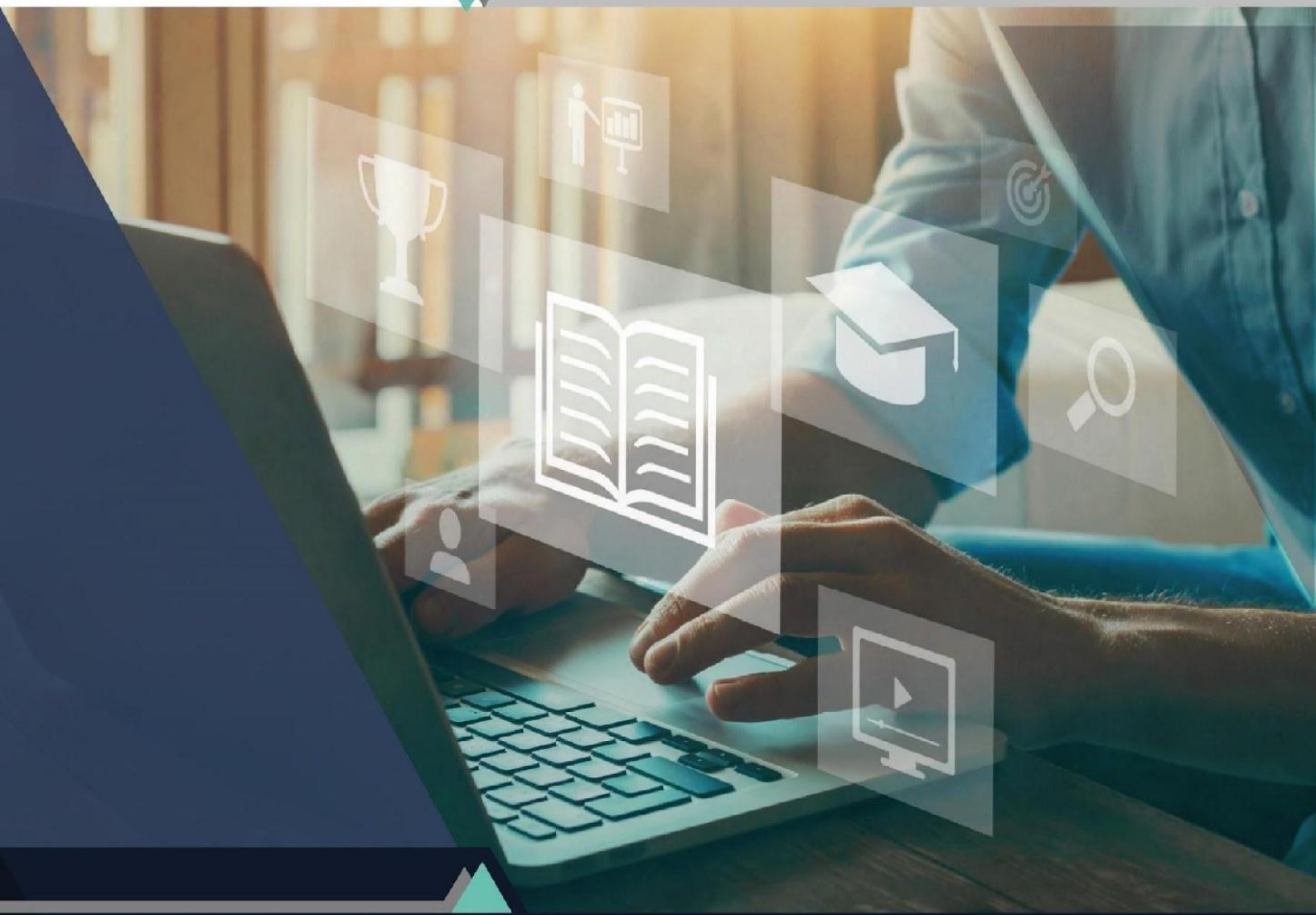




Arquitetura de Sistemas com IA

Autoria: Ahirton Lopes

Introdução

Ao longo desta disciplina, eu vou conduzir você por uma evolução completa de arquitetura, começando pelos fundamentos que diferenciam sistemas tradicionais de sistemas orientados por Inteligência Artificial e chegando a uma plataforma enterprise capaz de coordenar agentes, compartilhar infraestrutura, controlar custos, observar decisões e operar em escala.

O ponto de partida será um case corporativo realista, construído em torno da Vitalis Pharma, uma farmacêutica multinacional que precisa modernizar seu processo de produção de documentos de estudos clínicos.

Hoje, documentos como protocolos de estudo, Termos de Consentimento Livre e Esclarecido e Clinical Study Reports dependem de processos manuais, revisões sequenciais e equipes especializadas. Esse modelo consome tempo, aumenta custos e ainda permite que inconsistências escapem entre versões, idiomas e diferentes etapas do processo regulatório.

Para transformar esse cenário, a Vitalis Pharma inicia o desenvolvimento do Trial Forge, uma plataforma baseada em agentes de Inteligência Artificial Generativa.

A proposta parece simples à primeira vista.

Receber informações sobre um estudo clínico, consultar documentos regulatórios e produzir novos artefatos com maior velocidade.

Entretanto, o verdadeiro problema não está em escolher um modelo de linguagem.

O desafio consiste em construir uma arquitetura capaz de colocar esse modelo em produção com responsabilidade real sobre os resultados.

Ao longo da disciplina, eu vou mostrar que aplicações AI-First não podem ser tratadas como simples integrações com uma API generativa. O modelo é apenas um componente dentro de um sistema muito maior, composto por gateway, orquestração, mecanismos de recuperação de contexto, aprovação humana,

observabilidade, trilha de auditoria, controle de custo e estratégias de degradação.

Essa visão será construída de forma gradual.

Cada módulo ampliará a arquitetura anterior, sem descartar aquilo que já foi desenvolvido. O Trial Forge começará como um agente único e terminará como uma plataforma corporativa compartilhada por múltiplos estudos, equipes e países.

Fundamentos de uma arquitetura AI-First

No primeiro módulo, eu vou estabelecer a diferença fundamental entre software determinístico e sistemas verdadeiramente AI-First.

Em aplicações tradicionais, o comportamento costuma ser previsível. Diante de uma entrada conhecida, o sistema percorre um fluxo de execução definido e produz uma saída esperada. Essa previsibilidade permite escrever testes baseados em igualdade, revisar regras de negócio e confiar em pipelines automatizados de integração e entrega.

Quando um modelo generativo passa a ocupar uma posição central na arquitetura, esse comportamento muda.

O sistema começa a lidar com um componente não determinístico, capaz de produzir respostas diferentes para entradas semelhantes e, mais importante, capaz de responder incorretamente sem apresentar uma exceção técnica.

Esse é um dos principais riscos de arquiteturas baseadas em Inteligência Artificial.

O modelo pode estar disponível, responder dentro do tempo esperado e ainda assim produzir um resultado inadequado.

Por isso, o primeiro módulo apresenta cinco pilares que acompanharão toda a disciplina: não determinismo por design, Approval Gates, observabilidade e trilha de auditoria, custo e latência como restrições de primeira classe e degradação graciosa.

Esses pilares serão transformados em um diagrama de referência composto por Gateway, Orquestrador, Modelo associado a ferramentas e RAG, Approval Gate e uma camada transversal de observabilidade.

Também será apresentado um framework de três perguntas utilizado para decidir quando uma tarefa deve ser executada por um agente e quando uma regra determinística continua sendo a melhor escolha.

Essa decisão será aplicada componente por componente, evitando que processos híbridos sejam tratados como uma única tarefa.

Por fim, o módulo analisará os principais trade-offs da arquitetura AI-First, considerando latência, custo, precisão e performance em escala. O objetivo será compreender que não existe uma arquitetura capaz de maximizar todos esses eixos simultaneamente. Cada componente precisa possuir seu próprio orçamento e suas próprias prioridades.

Arquiteturas Single-Agent

Depois de compreender onde a Inteligência Artificial deve aparecer na arquitetura, eu vou abrir a caixa do modelo.

No segundo módulo, o foco estará na anatomia de um agente único.

Vamos estudar quatro componentes fundamentais: memória, planejamento, ferramentas e ação.

Esses elementos não serão tratados como um checklist obrigatório. Cada tarefa exige uma quantidade diferente de memória, profundidade de planejamento, conjunto de ferramentas e capacidade de ação.

Um agente bem desenhado não é aquele que possui todos esses recursos no nível máximo.

É aquele que utiliza exatamente o necessário para resolver sua tarefa sem produzir complexidade desnecessária.

Dentro desse módulo, veremos a diferença entre memória de curto e longo prazo, incluindo os impactos de retenção, recuperação de contexto e compliance.

Também estudaremos o padrão ReAct, que organiza o ciclo entre raciocínio e ação. Nesse padrão, o agente analisa o contexto, decide utilizar uma ferramenta, observa o resultado e avalia se já possui informações suficientes para concluir a tarefa.

Esse ciclo introduz flexibilidade, mas também cria riscos relacionados a custo crescente e ausência de convergência. Por isso, a arquitetura precisará estabelecer limites claros de iteração, tempo e orçamento.

Em seguida, será apresentado o padrão Reflection.

Nesse caso, o agente não busca novas informações. Ele revisa criticamente aquilo que já produziu, procurando inconsistências, omissões ou erros antes de entregar a resposta.

Também será detalhado o funcionamento das ferramentas externas, incluindo definição de esquemas, tipagem de parâmetros, execução determinística e retorno estruturado.

Essa parte permitirá compreender por que o modelo não executa diretamente ações no mundo real. Ele apenas propõe a chamada, enquanto a aplicação mantém controle sobre a execução.

O módulo será encerrado com a integração de todos esses componentes em um protótipo single-agent calibrado para uma tarefa específica do Trial Forge.

Arquiteturas Multi-Agent

No terceiro módulo, a arquitetura evolui para múltiplos agentes especializados.

Essa evolução acontece porque o Trial Forge não precisa produzir apenas um tipo de documento.

O Termo de Consentimento, o protocolo clínico e o Clinical Study Report possuem públicos, vocabulários, formatos e riscos diferentes.

Concentrar todas essas responsabilidades em um único agente tornaria o sistema excessivamente generalista, difícil de auditar e difícil de evoluir.

Por isso, o módulo apresentará a divisão do trabalho entre agentes especialistas.

Também será estabelecida uma distinção rigorosa entre agente e ferramenta.

Uma ferramenta executa uma operação determinística a partir de um contrato conhecido. Um agente interpreta contexto, toma decisões, utiliza ferramentas e pode reconhecer que uma tarefa está fora do seu domínio.

A partir dessa base, estudaremos seis padrões de orquestração.

O padrão Sequential será utilizado quando uma tarefa depender diretamente da saída anterior.

O Parallel será aplicado quando diferentes atividades puderem ocorrer simultaneamente.

O Supervisor centralizará a distribuição e a consolidação do trabalho.

O Hierarchical organizará supervisores e especialistas em múltiplos níveis.

O Group Chat permitirá debate entre agentes com diferentes perspectivas.

O Handoff será utilizado quando a responsabilidade precisar ser transferida integralmente para outro especialista durante a execução.

Também serão abordados problemas de sistemas distribuídos, como falha de nó, sincronização, controle de estado e mensagens perdidas.

O Teorema CAP será aplicado para decidir como o sistema deve reagir quando um agente deixa de responder.

O padrão Saga será utilizado para implementar ações compensatórias quando uma falha tardia invalidar apenas parte do trabalho já realizado.

Timeouts, retries limitados, idempotência e controle otimista de versão serão incorporados à arquitetura para que o sistema sobreviva a situações reais de produção.

Padrões de Design AI-Específicos

No quarto módulo, eu vou aprofundar padrões específicos de sistemas baseados em Inteligência Artificial.

O primeiro grupo será dedicado ao RAG.

Começaremos pelo Basic RAG, entendendo chunking, embeddings, indexação, recuperação e enriquecimento de contexto.

Depois avançaremos para Hybrid Search, Multi-Index e Agentic RAG, mostrando que esses padrões podem ser combinados para resolver limitações diferentes da busca.

Em seguida, serão apresentados os padrões de roteamento.

O Intent-Based Routing identificará para qual fluxo a requisição deve seguir.

O Model Router escolherá qual modelo deve executar a tarefa, equilibrando custo, latência e precisão.

O terceiro grupo será dedicado à otimização.

O Semantic Cache permitirá reutilizar respostas para perguntas semanticamente equivalentes.

O Prompt Cache evitará reprocessar contextos longos e repetidos.

O Response Streaming reduzirá a percepção de espera durante gerações demoradas.

Depois, o Approval Gate será formalizado como componente técnico.

Veremos como interromper uma execução, aplicar limiares de confiança, definir gates síncronos e assíncronos e registrar todas as decisões em uma trilha de auditoria imutável.

Ao final do módulo, todos esses padrões serão integrados em um único gateway, responsável por identificar intenção, consultar cache, escolher modelo, recuperar contexto, gerar resposta, avaliar confiança, solicitar aprovação quando necessário e registrar todo o processo.

Arquitetura Enterprise

No quinto e último módulo, a perspectiva deixa de ser uma única requisição e passa a ser a plataforma inteira em produção.

O Trial Forge torna-se parte da Vitalis Platform, compartilhada por dezenas de estudos, equipes e países.

A arquitetura será organizada em quatro grandes componentes: API Gateway, orquestração, serviços compartilhados e observabilidade.

Veremos como evitar a reimplementação dos mesmos serviços por diferentes equipes e como princípios como Loose Coupling, Clear Interfaces e Policy-Driven Control tornam esse compartilhamento seguro.

A observabilidade será ampliada para acompanhar não apenas infraestrutura, mas também prompts, versões, modelos, documentos recuperados, qualidade das respostas e deriva de comportamento.

Prompts serão tratados como código, com versionamento, comparação e replay.

Também estudaremos diferentes modelos de implantação.

Kubernetes será utilizado para cargas permanentes.

Serverless atenderá componentes de uso esporádico, considerando o problema do cold start.

Edge Computing permitirá processar informações mais perto do usuário ou diretamente no dispositivo, reduzindo latência e preservando privacidade.

Por fim, o módulo apresentará cascatas de modelos e controle de orçamento por tenant.

Em vez de escolher antecipadamente entre modelo barato e caro, a arquitetura poderá começar pelo modelo de menor custo e escalar apenas quando sinais concretos indicarem que a resposta não é suficiente.

Cada estudo possuirá seu próprio orçamento, verificado antes de qualquer operação custosa.

Ao final da disciplina, teremos acompanhado a construção completa de uma arquitetura AI-First.

Começaremos decidindo quando utilizar Inteligência Artificial.

Depois compreenderemos a anatomia de um agente único, a coordenação entre múltiplos agentes, os padrões específicos de recuperação, roteamento, otimização e aprovação e, finalmente, a operação de toda essa estrutura em escala enterprise.

O objetivo não é apenas conhecer tecnologias ou frameworks.

Eu quero que você desenvolva um critério de arquitetura capaz de responder perguntas concretas.

Quando usar um agente?

Quando manter uma regra determinística?

Quando dividir responsabilidades entre especialistas?

Quando executar em paralelo?

Quando pausar para aprovação?

Quando reutilizar uma resposta?

Quando escolher um modelo mais caro?

Quando bloquear uma requisição antes que ela ultrapasse o orçamento?

Essas decisões formam a base de uma arquitetura AI-First madura, capaz de colocar Inteligência Artificial em produção com qualidade, governança, rastreabilidade e responsabilidade real sobre os resultados.

Módulo 01: Fundamentos de Arquitetura AI-First

Capítulo 1: Fundamentos de Arquitetura AI-First (PT-1)

Ao longo desta disciplina, vou conduzir o desenvolvimento de um case de arquitetura corporativa que representa um cenário bastante próximo da realidade encontrada em grandes organizações. Trabalharemos com a Vitalis Pharma, uma empresa farmacêutica multinacional que enfrenta o desafio de modernizar um processo crítico de negócio por meio da Inteligência Artificial Generativa. O objetivo não é apenas compreender conceitos teóricos, mas analisar como decisões arquiteturais impactam diretamente a construção de sistemas inteligentes capazes de operar em ambientes corporativos complexos e altamente regulados.

A Vitalis Pharma possui um processo tradicional para elaboração de documentos relacionados aos estudos clínicos. Protocolos de pesquisa, termos de consentimento livre e esclarecido e relatórios de estudos clínicos são produzidos de forma manual e passam por diversas etapas de revisão humana antes de serem submetidos aos órgãos reguladores. Esse processo, embora confiável, apresenta um custo elevado em tempo e recursos. Cada novo protocolo exige revisões sequenciais, consumindo aproximadamente duas semanas apenas na etapa de redação, e mesmo assim ainda ocorrem inconsistências entre documentos produzidos em diferentes idiomas.

Diante desse cenário, a empresa decide desenvolver uma plataforma baseada em agentes de Inteligência Artificial denominada **Trial Forge**. Entretanto, o

verdadeiro desafio não está na escolha do modelo de linguagem que será utilizado, mas sim na arquitetura necessária para que esse modelo possa operar de forma segura, confiável e auditável dentro de um ambiente corporativo.

É exatamente esse tipo de problema que estudaremos ao longo desta disciplina. Cada módulo representa uma evolução da arquitetura da solução. Começaremos com um agente simples e, gradativamente, construiremos uma plataforma corporativa completa, incorporando elementos como gateway, mecanismos de orquestração, arquitetura multiagente, governança, observabilidade e trilhas de auditoria.

Antes de desenhar qualquer arquitetura, porém, existe uma pergunta fundamental que precisa ser respondida: por que os padrões tradicionais de arquitetura de software, que funcionaram muito bem durante décadas, deixam de ser suficientes quando o componente principal do sistema passa a ser um modelo de Inteligência Artificial Generativa?

Essa questão marca a principal diferença entre arquiteturas tradicionais e arquiteturas AI-First, também conhecidas na literatura como arquiteturas AI-Driven. Ambos os termos aparecem frequentemente na literatura técnica e representam a ideia de que a Inteligência Artificial deixa de ser um recurso complementar para assumir o papel central nas decisões do sistema.

Ao longo deste módulo, pretendo mostrar não apenas conceitos acadêmicos, mas principalmente princípios arquiteturais utilizados em projetos reais. O foco não será apresentar um framework com um nome chamativo ou uma metodologia genérica, mas discutir decisões práticas tomadas quando o principal componente do sistema é um modelo de linguagem e não uma regra de negócio implementada por meio de código determinístico.

Minha experiência profissional reforça justamente esse ponto. Em diferentes projetos envolvendo Visão Computacional, OCR e Inteligência Artificial Generativa, ficou evidente que o sucesso da solução nunca dependeu exclusivamente do modelo utilizado. O fator decisivo sempre foi a arquitetura construída ao redor desse modelo.

Em um projeto realizado no setor do agronegócio, por exemplo, sistemas de visão computacional passaram a identificar trabalhadores sem equipamentos de proteção individual utilizando câmeras já existentes nas plantas industriais, sem necessidade de substituição de hardware. Em outro projeto, desenvolvido para uma grande seguradora, a combinação entre OCR, Visão Computacional e Inteligência Artificial Generativa reduziu o tempo de validação documental de quarenta e oito horas para apenas trinta e sete segundos.

Embora os resultados desses projetos sejam expressivos, existe um aspecto muito mais importante do que a tecnologia empregada. Em nenhum desses casos o modelo de IA trabalhava isoladamente. Havia uma arquitetura cuidadosamente planejada para lidar com falhas, validar resultados, controlar custos, permitir intervenção humana sempre que necessário e garantir que decisões críticas fossem tomadas de maneira segura.

Essa mesma lógica também apareceu durante minha pesquisa de doutorado, quando participei do desenvolvimento de um sistema de detecção precoce de evasão escolar aplicado à rede estadual de ensino de São Paulo, atendendo milhões de estudantes. Mesmo sendo capaz de identificar padrões de risco, o sistema jamais substituí a decisão humana. Seu papel era produzir evidências e recomendações. A decisão final permanecia sob responsabilidade dos profissionais da educação.

Esse exemplo ilustra um princípio extremamente importante para qualquer arquitetura AI-First: modelos de Inteligência Artificial não existem para substituir completamente o julgamento humano, mas para ampliar sua capacidade de análise e tomada de decisão.

Ao final deste estudo, o objetivo é compreender por que a maior parte dos projetos de Inteligência Artificial Generativa não consegue ultrapassar a fase de protótipo e chegar efetivamente à produção. Frequentemente acredita-se que o problema está na escolha do modelo, quando, na realidade, a principal causa encontra-se na arquitetura construída ao seu redor.

Diversos estudos publicados em 2025 reforçam essa conclusão. Relatórios da Rand Corporation indicam que mais de oitenta por cento dos projetos

corporativos de Inteligência Artificial falham em entregar o valor esperado para o negócio. Pesquisas da S&P Global Market Intelligence mostram um crescimento significativo na quantidade de iniciativas abandonadas antes mesmo da entrada em produção. Da mesma forma, previsões da Gartner apontam que uma parcela considerável dos projetos envolvendo agentes inteligentes será cancelada antes de alcançar maturidade operacional.

Embora esses estudos utilizem diferentes metodologias, todos convergem para uma mesma conclusão: a tecnologia funciona em ambientes controlados, mas encontra enormes dificuldades quando precisa operar com dados reais, usuários reais, restrições de desempenho e processos corporativos complexos.

O motivo não está na falta de modelos sofisticados. Atualmente, modelos de linguagem de alta qualidade tornaram-se praticamente uma commodity. O verdadeiro diferencial está na arquitetura capaz de integrar esses modelos aos processos organizacionais de maneira segura, previsível e governável.

Para compreender essa mudança, é necessário observar como funciona um sistema tradicional.

Em arquiteturas clássicas, aplicações são construídas sobre comportamentos determinísticos. Diante de uma determinada entrada, espera-se sempre a mesma saída. Essa previsibilidade permite criar testes automatizados, pipelines de integração contínua, revisões de código e mecanismos de validação extremamente confiáveis. Todo o comportamento da aplicação pode ser descrito como uma sequência previsível de estados.

Esse paradigma muda completamente quando o componente central passa a ser um modelo de linguagem.

Um agente baseado em Inteligência Artificial não executa simplesmente um conjunto fixo de regras. Ele interpreta linguagem natural, considera contexto, produz inferências e realiza processos de raciocínio probabilístico. Como consequência, duas execuções realizadas com a mesma entrada podem gerar respostas diferentes, sem que isso represente um erro de funcionamento.

Essa característica altera profundamente a maneira como arquitetamos sistemas.

Se continuarmos tratando o modelo como uma função puramente determinística, inevitavelmente construiremos aplicações frágeis, incapazes de lidar com respostas inesperadas ou variações naturais produzidas pela IA.

Existe ainda uma terceira transformação ainda mais significativa.

Modelos generativos tradicionais recebem uma entrada e produzem uma saída textual, como um resumo, uma classificação ou uma resposta.

Já um agente inteligente vai além disso.

Ele recebe informações, interpreta o contexto, produz uma resposta e executa ações no mundo real. Pode iniciar processos, disparar fluxos de trabalho, atualizar sistemas corporativos, registrar documentos ou tomar decisões que afetam diretamente outras pessoas.

É justamente nesse momento que surge o maior risco arquitetural.

Quando um erro deixa de ser apenas um texto incorreto e passa a representar uma ação executada dentro da organização, as consequências tornam-se muito mais relevantes.

No caso da Vitalis Pharma, por exemplo, o Trial Forge não terá apenas a função de resumir protocolos clínicos. Sua responsabilidade será produzir versões preliminares de documentos regulatórios que posteriormente serão utilizados em processos oficiais de auditoria.

Caso essa arquitetura seja construída como uma simples chamada de API responsável por gerar texto, sem mecanismos de validação humana, rastreabilidade e auditoria, qualquer inconsistência poderá gerar impactos regulatórios extremamente sérios durante inspeções conduzidas por órgãos competentes.

Essa situação conduz a uma das perguntas mais importantes de toda a disciplina: **essa tarefa realmente precisa de um agente inteligente ou um processo determinístico continua sendo a melhor solução?**

Nem todo problema deve ser resolvido utilizando Inteligência Artificial.

Agentes fazem sentido quando existe necessidade de julgamento, interpretação contextual, adaptação dinâmica ou tratamento de situações ambíguas impossíveis de serem representadas por regras fixas.

Eles também são apropriados quando diferentes etapas do processo dependem umas das outras, quando existe grande diversidade nas entradas recebidas ou quando o próprio sistema precisa reconhecer suas limitações e solicitar intervenção humana.

Por outro lado, existem inúmeros cenários nos quais um algoritmo determinístico continua sendo superior.

Fluxos repetitivos, validações estruturais, verificações obrigatórias, regras legais bem definidas e processos cujo custo do erro é extremamente elevado normalmente devem permanecer implementados por meio de regras tradicionais.

No próprio Trial Forge existe um excelente exemplo dessa distinção.

Produzir automaticamente um rascunho de um termo de consentimento baseado em um protocolo clínico envolve interpretação de contexto, adequação regulatória e construção de linguagem técnica. Trata-se claramente de uma atividade adequada para um agente inteligente.

Entretanto, verificar se todos os campos obrigatórios de um formulário foram preenchidos corretamente continua sendo uma validação objetiva, baseada em regras determinísticas. Nesse caso, utilizar um agente representaria apenas aumento de custo, complexidade e risco sem qualquer benefício real.

Grande parte dos projetos malsucedidos de Inteligência Artificial nasce justamente da incapacidade de identificar essa fronteira. Em muitos casos,

agentes são utilizados onde bastariam funções tradicionais. Em outros, tenta-se resolver problemas naturalmente ambíguos utilizando apenas regras rígidas.

Para apoiar esse processo de decisão, utilizo um checklist simples durante reuniões de arquitetura. Esse material será aprofundado ao longo da disciplina e posteriormente evoluirá para um framework mais completo utilizado na atividade prática.

Independentemente da evolução desse framework, cinco pilares acompanharão todas as decisões arquiteturais desenvolvidas durante o curso.

O primeiro é o **não determinismo por design**. Em sistemas baseados em IA não faz sentido validar igualdade absoluta entre respostas. Os testes precisam considerar faixas aceitáveis de comportamento e níveis mínimos de confiança.

O segundo pilar corresponde aos **Approval Gates**, também conhecidos como **Human in the Loop**. Sempre que o custo do erro for elevado, decisões produzidas pelo agente devem ser tratadas como recomendações, exigindo aprovação humana antes de qualquer ação definitiva.

O terceiro pilar é a **observabilidade associada à trilha de auditoria**. Cada raciocínio produzido pelo agente, cada decisão tomada e cada ação executada precisam permanecer registrados de forma que seja possível reconstruir posteriormente todo o histórico do processo. Em ambientes regulados, como o da indústria farmacêutica, essa rastreabilidade deixa de ser uma boa prática e passa a representar um requisito obrigatório.

O quarto pilar estabelece **custo e latência como restrições arquiteturais de primeira classe**. Cada interação com um modelo de linguagem possui impacto financeiro e influencia diretamente o tempo de resposta da aplicação. Esses fatores devem ser considerados desde as primeiras decisões de arquitetura e não apenas durante fases posteriores de otimização.

Por fim, o quinto pilar é a **degradação graciosa**. Sempre que o modelo apresentar comportamento inesperado, baixa confiança ou indisponibilidade, o sistema precisa possuir mecanismos de fallback capazes de reduzir funcionalidades sem interromper completamente sua operação.

Esses cinco princípios servirão como referência durante toda a disciplina. Cada novo padrão arquitetural apresentado será analisado sob essa perspectiva, permitindo compreender como arquiteturas AI-First diferem profundamente das arquiteturas tradicionais.

Ao concluir esta primeira etapa, fica evidente que desenvolver aplicações baseadas em Inteligência Artificial vai muito além da integração com um modelo de linguagem. O verdadeiro desafio consiste em projetar arquiteturas capazes de equilibrar autonomia, controle, governança, segurança e responsabilidade sobre cada decisão tomada pelo sistema. É justamente essa mudança de paradigma que fundamenta toda a construção de arquiteturas AI-First e servirá de base para os próximos módulos da disciplina.

Capítulo 2: Fundamentos de Arquitetura AI-First (PT-2)

No primeiro momento desta disciplina, apresentei os princípios que diferenciam uma arquitetura tradicional de uma arquitetura AI-First. Agora, quero dar um passo além. O objetivo deixa de ser apenas compreender esses princípios e passa a ser transformá-los em componentes concretos de arquitetura. Afinal, princípios orientam decisões, mas não implementam sistemas. São os componentes que aparecem nos diagramas, que possuem responsabilidades bem definidas e que serão efetivamente desenvolvidos, testados e mantidos pela equipe de engenharia.

Ao longo deste capítulo, construirei o primeiro diagrama de referência da arquitetura do Trial Forge, a plataforma de agentes inteligentes da Vitalis Pharma. Esse diagrama servirá como base para todos os módulos seguintes. Cada novo assunto aprofundará uma das partes dessa arquitetura, permitindo compreender como uma solução AI-First evolui de forma estruturada até se tornar uma plataforma corporativa completa.

Da teoria para a arquitetura

No capítulo anterior apresentei cinco pilares fundamentais que devem orientar qualquer arquitetura baseada em Inteligência Artificial: não determinismo por design, approval gates, observabilidade, custo e latência e degradação graciosa.

Entretanto, existe uma diferença importante entre um princípio arquitetural e um componente de software.

Um princípio representa uma diretriz que deve ser respeitada durante o projeto. Ele estabelece o comportamento esperado da solução, mas não define onde esse comportamento será implementado. Já um componente possui responsabilidades específicas, interfaces bem definidas e participa diretamente do funcionamento da aplicação.

Essa distinção parece simples, mas é extremamente importante na prática. Em muitos projetos encontro equipes que possuem excelentes apresentações sobre governança, confiabilidade e Inteligência Artificial responsável, porém nenhuma dessas ideias foi traduzida em serviços concretos, contratos de API ou responsabilidades técnicas claramente definidas.

Quando isso acontece, a arquitetura permanece apenas como uma intenção. Sem componentes responsáveis por implementar cada princípio, dificilmente essas boas práticas chegam ao ambiente de produção.

Por isso, antes de desenhar qualquer sistema AI-First, procuro responder uma pergunta bastante objetiva: em qual componente da arquitetura cada princípio será implementado?

Traduzindo os cinco pilares

O primeiro princípio, o não determinismo por design, materializa-se no orquestrador. É ele quem recebe a resposta produzida pelo modelo e avalia se aquele resultado pode seguir para as próximas etapas do fluxo.

Em vez de confiar cegamente na primeira resposta produzida pela Inteligência Artificial, a arquitetura passa a validar o comportamento do modelo antes que qualquer decisão seja tomada.

O segundo princípio, Human in the Loop, transforma-se em um Approval Gate.

Quando o risco associado à decisão é elevado, o sistema não executa automaticamente a ação proposta. Em vez disso, produz uma recomendação que será analisada por um especialista humano antes de sua execução definitiva.

Esse comportamento lembra bastante estratégias utilizadas em processos de deployment canário. Antes de liberar uma nova versão para todos os usuários, realiza-se uma validação controlada. A lógica é semelhante: primeiro valida-se, depois executa-se.

O terceiro princípio corresponde à observabilidade e à trilha de auditoria.

Na arquitetura isso deixa de ser apenas um conjunto de logs espalhados pela aplicação. Passa a existir uma trilha contínua de eventos, registrando cada decisão tomada pelo sistema, cada chamada realizada e cada resposta produzida, sempre associadas a identificadores únicos e informações temporais.

Esse conceito aproxima-se das práticas já conhecidas de tracing distribuído, como OpenTelemetry. A diferença é que agora não observamos apenas componentes determinísticos, mas também decisões produzidas por modelos de linguagem.

O quarto princípio relaciona custo e latência ao roteamento inteligente entre modelos.

Nem toda solicitação exige o modelo mais sofisticado ou mais caro disponível. A arquitetura deve ser capaz de escolher dinamicamente qual modelo atende melhor determinada tarefa, equilibrando qualidade, tempo de resposta e custo operacional.

Essa decisão deixa de ser uma otimização posterior e passa a fazer parte do próprio desenho arquitetural.

Por fim, a degradação graciosa transforma-se em mecanismos de fallback implementados pelo orquestrador.

Caso o modelo apresente baixa confiança, indisponibilidade ou comportamento inesperado, o sistema não interrompe completamente sua operação. Em vez disso, executa um fluxo alternativo previsível, reduzindo funcionalidades quando necessário, mas mantendo a continuidade do serviço.

O primeiro diagrama de referência

Depois de transformar princípios em componentes, chegamos ao primeiro diagrama de referência utilizado durante toda a disciplina.

Esse diagrama é composto por quatro componentes principais organizados de forma linear e uma camada transversal responsável pela observabilidade.

A escolha desse formato não é estética. Ela representa uma decisão arquitetural.

Um erro bastante comum consiste em posicionar o modelo de Inteligência Artificial no centro da arquitetura, conectado diretamente a diversos sistemas externos, como se toda a inteligência da solução estivesse concentrada nele.

Essa representação transmite uma impressão equivocada de que o modelo controla toda a aplicação.

Na prática, isso não acontece.

Grande parte do comportamento de uma aplicação AI-First continua sendo completamente determinística. Apenas uma parte específica da arquitetura trabalha com comportamento probabilístico.

Ao organizar os componentes de forma linear, torna-se muito mais fácil visualizar exatamente onde termina o comportamento tradicional e onde começa a Inteligência Artificial.

Essa separação facilita tanto o desenvolvimento quanto a manutenção da solução.

Gateway: o ponto de entrada

O primeiro componente da arquitetura é o Gateway.

Sua responsabilidade permanece bastante semelhante àquela encontrada em arquiteturas corporativas convencionais.

É nele que ocorrem autenticação, autorização, validação de formato das requisições, limitação de taxa de chamadas e o primeiro roteamento.

Antes que qualquer solicitação chegue ao modelo de linguagem, o Gateway verifica se aquela requisição realmente deveria existir.

Solicitações malformadas, usuários não autenticados ou chamadas claramente inválidas são rejeitados imediatamente.

Essa estratégia evita desperdício de processamento e reduz custos associados às chamadas para modelos generativos.

Nesse ponto também começam a surgir os primeiros elementos relacionados à otimização da arquitetura, pois o Gateway participa do roteamento inicial das solicitações antes mesmo de qualquer interação com a Inteligência Artificial.

O orquestrador como cérebro determinístico

Após passar pelo Gateway, a requisição chega ao Orquestrador.

Esse componente representa o cérebro determinístico da aplicação.

É importante compreender que o orquestrador não é um modelo de Inteligência Artificial.

Ele é software tradicional.

Seu comportamento continua sendo previsível, testável e implementado utilizando as mesmas boas práticas aplicadas ao desenvolvimento convencional.

Sua responsabilidade consiste em coordenar toda a execução do fluxo.

Ele decide quando consultar o modelo, quando recuperar informações adicionais, quando acessar ferramentas externas, quando solicitar dados complementares e quando validar o resultado produzido.

Essa separação entre orquestração determinística e raciocínio probabilístico evita um erro muito comum em projetos de IA: transformar todo o sistema em uma grande caixa-preta.

Na realidade, apenas uma pequena parte da arquitetura executa inferências baseadas em Inteligência Artificial. Todo o restante continua sendo desenvolvido utilizando engenharia de software tradicional.

Também é dentro do orquestrador que surge uma visão arquitetural de Prompt Engineering.

Prompting deixa de ser apenas a escrita de comandos para um modelo e passa a representar um pipeline responsável pela construção, enriquecimento, versionamento e gerenciamento do contexto enviado à Inteligência Artificial.

O núcleo inteligente

O terceiro componente corresponde ao modelo de linguagem juntamente com suas ferramentas auxiliares e mecanismos de recuperação de contexto.

É nesse ponto que ocorre efetivamente o raciocínio realizado pela Inteligência Artificial.

Além do modelo propriamente dito, essa camada pode incorporar bases de conhecimento utilizando Retrieval Augmented Generation (RAG), consultas a bancos de dados, APIs externas e outras ferramentas necessárias para ampliar o contexto disponível durante a geração da resposta.

Esse é o único componente verdadeiramente não determinístico da arquitetura.

Por esse motivo, ele permanece isolado dos demais componentes.

Essa separação permite substituir modelos, ajustar parâmetros, incorporar novas fontes de conhecimento ou alterar estratégias de recuperação de contexto sem impactar o restante da solução.

Também é nessa camada que se concentram as preocupações relacionadas à Inteligência Artificial Responsável.

Questões envolvendo vieses, alucinações, inconsistências e comportamentos inesperados têm origem nesse componente e precisam ser tratadas adequadamente pela arquitetura.

Approval Gate

Depois que o modelo produz sua resposta, entra em ação o Approval Gate.

Nem todas as decisões exigem aprovação humana.

O objetivo desse componente é atuar apenas quando o risco associado à ação ultrapassa um limite previamente definido.

Esse limite não deve ser decidido exclusivamente pela equipe técnica.

No caso da Vitalis Pharma, por exemplo, cabe à responsável pelo processo regulatório estabelecer quais documentos exigem revisão humana obrigatória antes de serem enviados para órgãos externos.

A engenharia implementa esse comportamento.

O negócio define quando ele deve acontecer.

Essa separação deixa claro que arquiteturas AI-First são construídas por equipes multidisciplinares, envolvendo engenharia, especialistas do domínio e responsáveis pelos processos organizacionais.

Além disso, a interface utilizada durante a aprovação representa uma parte importante da experiência do usuário.

O especialista precisa compreender claramente qual decisão foi tomada pelo modelo, quais evidências sustentam essa decisão e exatamente o que será aprovado.

Observabilidade como elemento transversal

Enquanto todos esses componentes executam suas responsabilidades, uma camada transversal registra continuamente tudo o que acontece.

Essa banda de observabilidade acompanha cada etapa da requisição, registrando chamadas, respostas, decisões, aprovações e eventos importantes.

Sua função não se limita ao monitoramento operacional.

Ela permite reconstruir completamente o histórico de qualquer decisão tomada pelo sistema.

Quando alguém perguntar meses depois por que determinado documento foi aprovado ou qual informação levou o agente a produzir determinada resposta, toda essa sequência poderá ser recuperada.

Em ambientes regulados, como o da indústria farmacêutica, essa capacidade deixa de representar uma boa prática e torna-se um requisito indispensável para auditorias e processos de conformidade.

O AI Architecture Canvas

Para facilitar a construção dessas soluções, utilizo um artefato chamado AI Architecture Canvas.

Esse material organiza as principais decisões arquiteturais necessárias durante a criação de um sistema AI-First.

Em vez de iniciar diretamente pelo diagrama técnico, começo identificando a tarefa que será executada.

A primeira pergunta é bastante simples: o problema exige julgamento contextual realizado por um agente inteligente ou pode ser resolvido utilizando um fluxo determinístico?

A partir dessa resposta, passo a identificar quem utilizará o sistema, quais mecanismos de autenticação serão necessários, quais validações devem ocorrer ainda no Gateway, como o contexto será construído, quais modelos serão utilizados, onde existirão mecanismos de aprovação, como funcionará a observabilidade e quais serão os critérios de sucesso da solução.

Esse processo transforma decisões abstratas em uma arquitetura concreta, permitindo que princípios arquiteturais sejam convertidos em componentes implementáveis.

Ao final deste capítulo, fica evidente que uma arquitetura AI-First não nasce da simples adoção de um modelo de linguagem. Ela surge da capacidade de organizar responsabilidades, controlar riscos e integrar Inteligência Artificial aos processos corporativos de forma estruturada. É justamente essa organização que permitirá evoluir o Trial Forge nos próximos módulos da disciplina, aprofundando cada componente apresentado neste primeiro diagrama de referência.

Capítulo 3: Fundamentos de Arquitetura AI-First (PT-3)

Nos capítulos anteriores, apresentei os fundamentos que diferenciam uma arquitetura tradicional de uma arquitetura AI-First e mostrei como esses princípios se transformam em componentes arquiteturais concretos. Agora, quero dar mais um passo nessa evolução. Não basta apenas compreender que existem tarefas mais adequadas para agentes inteligentes e outras que continuam sendo melhor resolvidas por lógica determinística. O verdadeiro desafio é estabelecer um critério objetivo, repetível e auditável para tomar essa decisão.

Em projetos reais, é muito comum que uma equipe de arquitetura se reúna para discutir se determinado processo deve ou não utilizar Inteligência Artificial. Muitas vezes, essa discussão acaba sendo conduzida pela experiência individual de cada profissional. Um arquiteto considera uma tarefa suficientemente complexa para justificar um agente, enquanto outro acredita que regras determinísticas ainda seriam suficientes. Ambos possuem argumentos válidos, mas a decisão acaba sendo baseada em percepção e não em critérios verificáveis.

É exatamente esse tipo de situação que desejo evitar. Em vez de depender da intuição, proponho um framework composto por três perguntas sequenciais. Esse processo transforma uma decisão subjetiva em uma decisão arquitetural fundamentada, permitindo que qualquer integrante da equipe percorra o mesmo raciocínio e chegue à mesma conclusão.

Por que o critério inicial não é suficiente?

No início da disciplina apresentei um critério bastante simples: utilizar agentes quando a decisão exige julgamento e adaptação ao contexto e utilizar regras determinísticas quando o fluxo é fixo e o custo do erro é elevado.

Esse critério continua sendo válido, mas possui limitações importantes.

Ele funciona muito bem como um primeiro filtro, especialmente em conversas rápidas ou durante reuniões iniciais de levantamento de requisitos. Entretanto, quando a arquitetura será implementada e permanecerá em produção durante anos, essa abordagem torna-se simplista demais.

O principal problema é que ela trata cada tarefa como se fosse indivisível.

Na prática, a maioria dos processos corporativos é composta por diversas etapas diferentes. Algumas exigem interpretação, outras seguem regras extremamente objetivas. Quando tratamos todo o processo como uma única atividade, acabamos tomando decisões inadequadas para parte dele.

Existe ainda um segundo problema.

Expressões como "decisão complexa" ou "contexto variável" são subjetivas. Dois profissionais experientes podem interpretar esses conceitos de maneiras completamente diferentes. Consequentemente, a arquitetura passa a depender da percepção individual de quem participa da reunião e não de critérios técnicos consistentes.

Para eliminar essa subjetividade, proponho substituir a discussão baseada em opinião por um processo estruturado de perguntas objetivas.

Primeira pergunta: existe uma regra finita?

A primeira pergunta do framework é bastante simples:

Existe uma regra finita capaz de resolver mais de 90% dos casos reais já observados?

É importante destacar a palavra **reais**.

Não estou falando de situações hipotéticas ou de exceções extremamente improváveis. Em projetos de software existe uma tendência natural de imaginar cenários que talvez nunca aconteçam. Essa prática frequentemente leva equipes a adotarem soluções muito mais complexas do que o necessário.

Se uma regra clara resolve praticamente todos os casos encontrados na operação da empresa, não existe motivo para introduzir um agente inteligente.

No contexto do Trial Forge, um excelente exemplo é a validação de formulários regulatórios.

Cada formulário possui uma lista conhecida de campos obrigatórios. Esses campos são definidos pelas normas regulatórias e permanecem estáveis dentro de uma determinada categoria de estudo clínico.

Nesse cenário, não existe interpretação contextual nem julgamento subjetivo.

Existe apenas uma verificação objetiva.

Se todos os campos obrigatórios estiverem preenchidos corretamente, o processo continua. Caso contrário, o sistema informa quais informações ainda precisam ser fornecidas.

Esse comportamento é perfeitamente implementado por meio de lógica determinística.

Segunda pergunta: o erro é caro e irreversível?

Caso a primeira pergunta seja respondida negativamente, seguimos para a segunda.

O erro possui alto impacto e suas consequências são irreversíveis?

Essa pergunta não define se utilizaremos ou não um agente inteligente.

Ela determina o grau de autonomia que esse agente poderá possuir.

Quando o custo do erro é elevado, o agente continua sendo extremamente útil. Entretanto, sua função deixa de ser executar ações diretamente e passa a produzir recomendações que serão avaliadas por especialistas humanos.

É exatamente nesse ponto que entra o conceito de **Approval Gate**, apresentado anteriormente.

O agente gera uma proposta, mas alguém precisa aprová-la antes que qualquer ação seja executada.

No Trial Forge, um bom exemplo é a geração da versão final de um Termo de Consentimento Livre e Esclarecido.

A elaboração desse documento exige interpretação de contexto, adaptação de linguagem e conhecimento regulatório.

Tudo isso justifica o uso de um agente inteligente.

Entretanto, um erro nesse documento pode comprometer todo o processo de consentimento de participantes de um estudo clínico.

Como as consequências são extremamente relevantes, a arquitetura exige aprovação humana antes que o documento seja utilizado oficialmente.

Terceira pergunta: o comportamento muda conforme o contexto?

Se o erro não for caro nem irreversível, chegamos à terceira pergunta.

A lógica muda conforme o contexto ou permanece sempre a mesma?

Quando o comportamento depende fortemente do contexto apresentado ao sistema, existe um forte indicativo de que a tarefa pertence ao domínio dos agentes inteligentes.

Entretanto, quando a lógica permanece constante, mesmo que pareça extensa ou sofisticada, regras determinísticas continuam sendo mais adequadas.

Esse ponto merece atenção especial.

É comum associar complexidade à necessidade de Inteligência Artificial.

Na realidade, uma regra extensa continua sendo apenas uma regra.

Uma árvore de decisão com dezenas de condições ainda representa lógica determinística.

Ela pode exigir mais trabalho durante sua implementação, mas continua sendo previsível, auditável e significativamente mais barata do que um agente executando raciocínio probabilístico.

Essa distinção evita que Inteligência Artificial seja utilizada apenas porque determinada implementação parece trabalhosa.

Uma decisão arquitetural auditável

Ao responder essas três perguntas sequencialmente, obtemos quatro possibilidades arquiteturais diferentes.

A tarefa pode permanecer completamente determinística.

Pode utilizar um agente supervisionado.

Pode utilizar um agente autônomo acompanhado de observabilidade.

Ou pode exigir uma combinação desses elementos.

O aspecto mais importante desse framework não é exatamente a resposta obtida.

O verdadeiro benefício está no processo utilizado para chegar até ela.

Quando toda a equipe responde às mesmas perguntas, utilizando as mesmas evidências, desaparecem discussões baseadas apenas em percepção individual.

As conversas deixam de girar em torno de opiniões e passam a discutir fatos observáveis.

Essa característica torna a decisão auditável, repetível e muito mais fácil de justificar perante equipes técnicas e áreas de negócio.

O primeiro caso limite: tarefas híbridas

Mesmo utilizando esse framework, ainda existem situações que costumam gerar dúvidas.

A primeira delas são as chamadas tarefas híbridas.

Grande parte dos processos corporativos parece representar uma única atividade.

Na prática, entretanto, eles são compostos por diversas subtarefas completamente diferentes.

Algumas exigem interpretação.

Outras seguem regras rigorosamente definidas.

O erro mais comum consiste em aplicar o framework à tarefa inteira, em vez de decompô-la em partes menores.

Sempre que alguém afirma que determinada atividade é complexa demais para ser resolvida por regras, costumo fazer uma pergunta simples:

Toda a tarefa é complexa ou apenas a parte responsável por compreender a informação recebida?

Na maioria das vezes, apenas essa segunda etapa realmente exige Inteligência Artificial.

Todo o restante continua sendo implementado por lógica determinística.

Essa simples decomposição altera completamente a arquitetura.

O segundo caso limite: reversibilidade

Outro aspecto frequentemente negligenciado diz respeito à reversibilidade das ações.

Nem todo erro caro possui o mesmo tratamento arquitetural.

Quando um erro pode ser corrigido posteriormente sem gerar impactos significativos, a arquitetura pode utilizar um processo de revisão assíncrona.

O agente executa sua tarefa e posteriormente um especialista valida ou corrige o resultado.

Entretanto, quando a ação não pode ser desfeita depois de executada, a situação muda completamente.

Nesse caso, a aprovação precisa acontecer antes da execução.

Imagine um agente corrigindo automaticamente problemas de formatação em documentos internos.

Caso alguma alteração esteja incorreta, outro profissional poderá ajustá-la posteriormente durante uma revisão periódica.

Agora imagine um agente enviando automaticamente documentos regulatórios para uma autoridade governamental.

Depois que esse documento foi protocolado, não existe como simplesmente desfazer a operação.

Nesse cenário, a aprovação prévia torna-se obrigatória.

A pergunta fundamental deixa de ser "o erro é grave?" e passa a ser:

É possível desfazer a ação depois que ela aconteceu?

O terceiro caso limite: classificações evoluem

Existe ainda uma terceira situação importante.

A classificação de uma tarefa não é permanente.

Arquiteturas AI-First maduras evoluem continuamente.

Uma atividade inicialmente tratada por agentes inteligentes pode, após meses de operação, revelar padrões extremamente consistentes.

Os registros produzidos pela camada de observabilidade tornam possível identificar comportamentos repetitivos que anteriormente não eram conhecidos.

Quando isso acontece, parte daquela tarefa pode migrar para lógica determinística.

O movimento inverso também é verdadeiro.

Uma regra inicialmente simples pode crescer ao longo do tempo, acumulando exceções, tratamentos especiais e particularidades de negócio.

Em determinado momento, manter essa lógica torna-se mais complexo do que utilizar um agente inteligente.

Por isso, decisões arquiteturais relacionadas à Inteligência Artificial nunca devem ser encaradas como definitivas.

Elas precisam ser revisitadas periodicamente com base nas evidências produzidas pelo próprio sistema.

Decompondo tarefas complexas

Para facilitar esse processo, utilizo um checklist acompanhado de um template de decomposição.

Em vez de analisar uma atividade inteira, divido o processo em subtarefas menores.

Cada uma delas é avaliada individualmente pelas três perguntas do framework.

Essa abordagem evita decisões excessivamente simplificadas e produz arquiteturas muito mais equilibradas.

O próprio material da disciplina apresenta esse template preenchido utilizando o Trial Forge como exemplo.

Exemplo: processamento de emendas de protocolo

Considere uma emenda realizada em um protocolo clínico.

À primeira vista, pode parecer uma única atividade.

Entretanto, ao analisá-la cuidadosamente, percebemos quatro subtarefas completamente diferentes.

A primeira consiste em identificar quais informações foram alteradas entre duas versões do protocolo.

Essa atividade envolve comparação de linguagem natural e interpretação contextual.

Trata-se claramente de uma responsabilidade adequada para um agente inteligente.

A segunda etapa é classificar o tipo da emenda.

Nesse caso, a classificação segue critérios definidos pela regulamentação.

Existe uma tabela objetiva determinando quando uma alteração é administrativa e quando ela é considerada substancial.

Não existe interpretação subjetiva.

Existe apenas aplicação de regras.

A terceira etapa consiste em decidir se aquela alteração exige aprovação antes de prosseguir.

Essa decisão combina lógica determinística com um Approval Gate.

Emendas administrativas continuam normalmente.

Emendas substanciais interrompem o fluxo até que um especialista realize sua aprovação.

Por fim, após a aprovação, o agente inteligente volta a atuar para regenerar automaticamente todos os documentos afetados pela alteração aprovada.

Observe como uma tarefa aparentemente única acaba distribuída entre diferentes componentes arquiteturais.

As atividades de interpretação pertencem ao modelo de linguagem.

As regras permanecem no orquestrador.

O controle de risco ocorre por meio do Approval Gate.

Framework de decisão e arquitetura tornam-se, na prática, duas formas diferentes de representar exatamente o mesmo sistema.

Outro exemplo: relatos de eventos adversos

Esse padrão também aparece em outra situação bastante comum na indústria farmacêutica.

Imagine que um participante do estudo relate sentir tontura algumas horas após utilizar determinada medicação.

Interpretar esse relato exige compreender linguagem natural.

Cada paciente descreve seus sintomas de maneira diferente.

Nenhuma regra consegue prever todas essas variações.

Essa etapa pertence naturalmente ao agente inteligente.

Depois que o sintoma foi corretamente identificado e classificado em uma categoria padronizada, todo o restante do processo passa a seguir regras bem definidas.

Dependendo da classificação obtida, o evento poderá exigir notificação obrigatória aos órgãos reguladores ou apenas registro em relatórios periódicos.

Essa decisão já não depende mais de interpretação.

Ela segue critérios estabelecidos pelos processos de farmacovigilância.

Caso a classificação implique uma notificação obrigatória, entra novamente em funcionamento o Approval Gate.

Como a comunicação com autoridades regulatórias produz efeitos irreversíveis, nenhum agente deve executar essa ação autonomamente.

Ao concluir este capítulo, fica evidente que o verdadeiro desafio de uma arquitetura AI-First não consiste apenas em decidir quando utilizar Inteligência Artificial. O aspecto mais importante é aprender a decompor problemas complexos em partes menores, identificando exatamente quais etapas exigem julgamento contextual e quais continuam sendo melhor resolvidas por lógica determinística. Esse raciocínio permite construir arquiteturas mais simples, previsíveis, auditáveis e economicamente sustentáveis, preservando a Inteligência Artificial apenas onde ela realmente agrega valor.

Capítulo 4: Fundamentos de Arquitetura AI-First (PT-4)

Ao longo deste primeiro módulo, construímos uma base sólida para compreender o que realmente caracteriza uma arquitetura AI-First. Inicialmente, discutimos por que sistemas baseados em Inteligência Artificial não podem ser tratados da mesma forma que aplicações determinísticas tradicionais. Em seguida, estruturamos um diagrama de referência composto por componentes bem definidos e estabelecemos um framework que permite decidir, de forma objetiva, quando utilizar agentes inteligentes e quando manter regras determinísticas.

Entretanto, ainda falta responder uma pergunta fundamental. Mesmo depois de decidir qual componente utilizar e onde cada responsabilidade deve estar localizada, permanece um aspecto que influencia diretamente a viabilidade técnica e financeira de qualquer projeto: quanto custa cada decisão arquitetural?

Esse custo não pode ser entendido apenas como investimento financeiro. Em arquiteturas AI-First, cada escolha representa um equilíbrio entre quatro fatores que competem constantemente entre si: latência, custo operacional, precisão das respostas e capacidade de processamento em escala.

Esses quatro elementos formam o orçamento arquitetural da solução. Ignorá-los durante o desenho da arquitetura significa transferir a decisão para o ambiente de produção, onde normalmente ela se torna muito mais cara e difícil de corrigir.

O orçamento começa durante a arquitetura

Existe uma tendência bastante comum em projetos de software de considerar desempenho e custo como problemas de infraestrutura. Muitas equipes acreditam que primeiro é necessário construir a aplicação e, somente depois, preocupar-se com otimizações.

Esse raciocínio funciona relativamente bem em aplicações convencionais, mas deixa de ser adequado quando o sistema depende fortemente de modelos de linguagem.

Cada chamada realizada para um modelo generativo possui impacto financeiro imediato. Cada documento recuperado durante uma busca aumenta o volume de tokens processados. Cada etapa adicional de raciocínio acrescenta tempo de resposta e consumo computacional.

Quando essas decisões não são tomadas durante o projeto da arquitetura, acabam sendo definidas automaticamente pelas configurações padrão utilizadas durante o desenvolvimento.

Na prática, isso significa que muitas organizações colocam em produção sistemas capazes de produzir excelentes demonstrações, mas financeiramente inviáveis para operação contínua.

Não é raro encontrar projetos nos quais a utilização permanente do modelo mais sofisticado disponível produz uma conta mensal superior ao benefício operacional obtido com a automação.

Nesse momento fica evidente que custo não representa um detalhe operacional. Ele faz parte da própria arquitetura.

Os quatro eixos de decisão

Toda arquitetura AI-First pode ser analisada sob quatro perspectivas complementares.

A primeira delas é a **latência**.

Latência representa o tempo necessário para que o usuário receba uma resposta.

Quando um sistema realiza apenas uma chamada simples para um modelo de linguagem, esse tempo corresponde basicamente à comunicação com o provedor e ao processamento da inferência.

Entretanto, aplicações modernas raramente executam apenas uma chamada.

É bastante comum que o agente consulte uma base de conhecimento, acione ferramentas externas, recupere documentos utilizando RAG, execute etapas intermediárias de planejamento e somente depois produza a resposta final.

Cada uma dessas operações adiciona novos intervalos de processamento.

Além disso, o contexto cresce progressivamente durante o raciocínio. Cada nova etapa utiliza informações produzidas anteriormente, fazendo com que a quantidade de dados processados aumente continuamente.

Consequentemente, um agente que executa diversas etapas não apresenta apenas uma latência proporcionalmente maior. Seu crescimento costuma ser superior ao esperado justamente porque cada etapa acumula contexto adicional.

O impacto financeiro

O segundo eixo corresponde ao **custo**.

Em sistemas baseados em modelos generativos, custo está diretamente relacionado ao volume de tokens processados durante cada interação.

Toda informação enviada ao modelo possui um preço.

Da mesma forma, toda resposta gerada também produz cobrança.

Quanto maior o contexto enviado, maior será o custo da inferência.

Esse aspecto torna especialmente importante o uso eficiente de mecanismos de recuperação de informação.

Um sistema que recupera documentos excessivamente grandes ou irrelevantes acaba pagando para que o modelo processe informações que dificilmente contribuirão para a resposta.

Da mesma forma, perguntas repetidas representam desperdício financeiro quando não existe algum mecanismo de cache.

Caches semânticos e caches tradicionais tornam-se componentes importantes justamente porque evitam que o sistema realize diversas vezes exatamente o mesmo processamento.

Mais do que reduzir tempo de resposta, eles reduzem diretamente o custo operacional da arquitetura.

Precisão não significa perfeição

O terceiro eixo corresponde à **precisão**.

Quando falamos em precisão, não estamos buscando respostas absolutamente perfeitas.

Na prática, diferentes tarefas exigem diferentes níveis de qualidade.

Uma atividade simples de classificação pode produzir excelentes resultados utilizando modelos menores e significativamente mais baratos.

Por outro lado, tarefas envolvendo geração de documentos regulatórios, contratos, pareceres técnicos ou qualquer artefato que produza consequências legais normalmente justificam a utilização de modelos mais sofisticados.

Essa diferença precisa ser considerada durante a arquitetura.

Não existe motivo para utilizar permanentemente o modelo mais caro da organização quando boa parte das tarefas poderia ser executada por modelos menores.

Da mesma forma, tentar economizar em tarefas altamente críticas costuma produzir um custo muito maior posteriormente, quando erros precisam ser corrigidos manualmente ou provocam impactos sobre o negócio.

Performance e capacidade de processamento

O quarto eixo corresponde à **performance**, frequentemente associada ao throughput da aplicação.

Enquanto a latência mede quanto tempo uma requisição demora para ser concluída, o throughput mede quantas requisições conseguem ser processadas simultaneamente.

Esse aspecto torna-se particularmente importante porque provedores de modelos normalmente impõem limites de utilização.

Quando o número de solicitações ultrapassa essa capacidade, filas começam a se formar.

Como consequência, a latência cresce para todos os usuários, inclusive para aqueles que acabaram de iniciar uma nova requisição.

Nesse contexto, performance atua como um multiplicador dos demais fatores.

Uma arquitetura perfeitamente adequada para poucos usuários pode tornar-se completamente inadequada quando submetida a um grande volume de acessos.

Por isso, desempenho em escala precisa ser considerado desde o início do projeto.

Não é possível otimizar tudo ao mesmo tempo

Talvez o aspecto mais importante deste módulo seja compreender que esses quatro eixos competem continuamente entre si.

Não existe arquitetura capaz de maximizar simultaneamente baixa latência, baixo custo, alta precisão e elevada capacidade de processamento.

Toda decisão representa um compromisso.

Quando escolho um modelo menor para reduzir custos, normalmente sacrifico parte da precisão.

Quando adiciono diversas etapas de validação para aumentar a confiabilidade das respostas, inevitavelmente aumento também a latência.

Quando utilizo múltiplos modelos em paralelo para acelerar determinadas respostas, passo a consumir mais recursos financeiros.

Essas relações de dependência caracterizam aquilo que chamamos de **trade-off arquitetural**.

Arquitetura consiste justamente em decidir quais características são prioritárias e quais podem ser flexibilizadas.

Não existe resposta universal.

Cada componente possui necessidades diferentes.

Aplicando os trade-offs ao Trial Forge

Esse raciocínio torna-se bastante claro quando analisamos o diagrama de referência construído ao longo do módulo.

Gateway

O Gateway representa a porta de entrada de todo o sistema.

Sua principal responsabilidade é autenticar usuários, validar requisições e realizar o primeiro roteamento.

Como nenhuma chamada para modelos ocorre nesse ponto, seu custo operacional permanece muito baixo.

Da mesma forma, não existe preocupação relacionada à precisão do modelo.

Entretanto, a latência torna-se absolutamente crítica.

Qualquer atraso introduzido pelo Gateway afeta todas as requisições subsequentes.

Por esse motivo, esse componente deve priorizar velocidade acima de qualquer outro fator.

Orquestrador

O Orquestrador também exige baixa latência.

Embora execute apenas lógica determinística, ele coordena todo o fluxo da aplicação.

Decisões incorretas de roteamento podem enviar requisições para componentes inadequados, comprometendo completamente o funcionamento do sistema.

Nesse caso, custo permanece relativamente baixo, mas precisão passa a representar um fator importante, pois o comportamento determinístico precisa ser absolutamente confiável.

Modelo, Tools e RAG

O componente responsável pela Inteligência Artificial apresenta um perfil completamente diferente.

Aqui a latência torna-se mais flexível.

Gerar documentos regulatórios ou relatórios técnicos normalmente permite que o usuário aguarde alguns segundos adicionais.

Essa margem de tempo pode ser utilizada para aumentar significativamente a qualidade das respostas.

Da mesma forma, esse componente concentra praticamente todo o investimento financeiro da arquitetura.

É nele que ocorrem as chamadas aos modelos, recuperação de contexto, utilização de ferramentas externas e demais operações relacionadas ao raciocínio.

Por consequência, precisão passa a representar o principal objetivo desse componente.

Approval Gate

O Approval Gate apresenta um comportamento bastante peculiar.

Como existe intervenção humana, sua latência naturalmente deixa de ser medida em segundos.

Uma aprovação pode levar minutos ou até horas, sem comprometer necessariamente o funcionamento da aplicação.

Da mesma forma, custo e precisão deixam de ser avaliados sob a perspectiva do modelo.

Nesse ponto, quem toma a decisão é o especialista humano.

O contexto modifica o orçamento

Outro aspecto importante consiste em perceber que o orçamento arquitetural não é permanente.

O mesmo componente pode possuir prioridades completamente diferentes dependendo do contexto de utilização.

Imagine inicialmente um cenário no qual um especialista regulatório aguarda, durante uma reunião, a geração de um documento.

Nesse caso, tempo de resposta torna-se extremamente importante.

Talvez seja necessário utilizar modelos mais rápidos ou versões parcialmente armazenadas em cache para reduzir a espera.

Agora imagine um segundo cenário completamente diferente.

Durante a madrugada, a Vitalis Pharma precisa processar automaticamente dezenas de relatórios clínicos acumulados para preparar uma auditoria anual.

Nesse momento, praticamente ninguém está aguardando o resultado.

Latência deixa de ser prioridade.

Esse tempo disponível pode ser utilizado para executar múltiplas verificações, consultar mais documentos, utilizar modelos mais sofisticados e produzir respostas de maior qualidade.

Além disso, diversos provedores oferecem custos reduzidos para processamento em lote, tornando financeiramente mais interessante executar grandes volumes de trabalho fora dos períodos de maior utilização.

Observe que o componente continua sendo exatamente o mesmo.

O que mudou foi apenas o contexto de uso.

Essa percepção reforça que arquiteturas AI-First precisam ser suficientemente flexíveis para adaptar seus parâmetros conforme o cenário operacional.

Aplicando o conhecimento

Como encerramento deste primeiro módulo, proponho um exercício prático que reúne todos os conceitos estudados.

O primeiro passo consiste em desenhar um diagrama de referência para um caso real ou para um cenário hipotético suficientemente detalhado.

Em seguida, cada tarefa identificada deve ser analisada utilizando o framework das três perguntas apresentado anteriormente, permitindo classificá-la como responsabilidade de um agente inteligente ou de lógica determinística.

Sempre que existirem tarefas híbridas, elas deverão ser decompostas em subtarefas menores antes da classificação.

Por fim, cada componente recebe seu próprio orçamento arquitetural.

Para cada um deles é necessário decidir qual eixo será considerado inegociável e quais fatores poderão ser flexibilizados durante a implementação.

Esse exercício produz o primeiro documento de arquitetura efetivamente utilizável para uma solução AI-First.

Mais do que um diagrama, ele registra decisões técnicas, critérios de classificação e justificativas relacionadas aos trade-offs adotados.

Módulo 02: Arquiteturas Single-Agent

Capítulo 1: Arquiteturas Single-Agent (PT-1)

No módulo anterior, construímos o diagrama de referência do Trial Forge e identificamos os principais componentes de uma arquitetura AI-First: Gateway, Orquestrador, Modelo associado a ferramentas e RAG, Approval Gate e uma camada transversal de observabilidade. Também definimos, com base em um framework de três perguntas, quando uma tarefa deveria permanecer no domínio determinístico e quando poderia ser atribuída ao componente não determinístico da arquitetura.

Além disso, discutimos os principais trade-offs envolvidos nessa decisão, considerando latência, custo, precisão e performance. Com isso, conseguimos compreender onde a Inteligência Artificial deve ser utilizada e quais restrições precisam ser consideradas durante o desenho da solução.

Entretanto, até agora tratamos o componente de modelo como uma caixa fechada. Consideramos que uma entrada era enviada e uma resposta era produzida, sem analisar detalhadamente o que acontece dentro dessa estrutura. Neste módulo, começo a desmontar essa caixa para mostrar que um agente possui uma anatomia própria, composta por elementos que podem ser combinados em diferentes níveis de sofisticação.

Essas combinações definem o comportamento do agente e influenciam diretamente seu custo, sua latência, sua previsibilidade e sua capacidade de atuar em processos reais.

O que caracteriza um agente

Toda semana surgem novos produtos apresentados como agentes de Inteligência Artificial. Entretanto, uma parcela significativa dessas soluções consiste apenas em uma chamada a um modelo de linguagem acompanhada de um prompt bem elaborado.

Esses sistemas não possuem memória entre diferentes execuções, não conseguem planejar uma sequência de passos e também não utilizam ferramentas externas para consultar dados ou executar operações.

Isso não significa que sejam soluções ruins. Em muitos casos, uma chamada simples é exatamente o que a tarefa exige. O problema surge quando utilizamos o termo agente para descrever arquiteturas completamente diferentes.

Essa falta de precisão cria dificuldades durante o desenho da solução.

Uma pessoa da equipe pode imaginar que o agente se lembrará das interações anteriores com o usuário. Outra pode considerar que cada execução será completamente independente. Ambas utilizam a mesma palavra, mas estão descrevendo sistemas arquiteturalmente distintos.

Para evitar essa confusão, utilizo uma definição baseada em componentes concretos, e não em marketing.

Um agente single-agent pode ser analisado a partir de quatro componentes principais:

1. Memória.
2. Planejamento.
3. Ferramentas.
4. Ação.

Esses componentes não representam um checklist obrigatório. Eles formam um conjunto de possibilidades que deve ser dimensionado conforme a tarefa.

Um agente pode possuir apenas memória de curto prazo, planejamento simples, uma única ferramenta e uma ação limitada. Outro pode exigir memória persistente, planejamento de múltiplas etapas, várias ferramentas e capacidade de executar alterações reais em sistemas externos.

Os dois continuam sendo agentes válidos. O que muda é a quantidade de cada componente exigida pelo problema.

Memória de curto e longo prazo

O primeiro componente é a memória.

Costumo dividi-la em memória de curto prazo e memória de longo prazo.

A memória de curto prazo corresponde ao contexto da requisição atual. Ela inclui as mensagens recentes, os documentos recebidos e os resultados produzidos durante as etapas anteriores do fluxo.

Esse tipo de memória pode ser comparado à memória RAM de um computador. Ela é rápida, está disponível durante o processamento atual, mas possui capacidade limitada e tende a desaparecer quando a execução termina.

A memória de longo prazo, por outro lado, persiste entre diferentes sessões.

Ela permite que o agente recupere informações obtidas em interações anteriores, mesmo depois que uma execução foi encerrada e uma nova foi iniciada.

Podemos compará-la ao armazenamento em disco. Ela comporta um volume muito maior de informações, mas precisa ser consultada de forma seletiva quando algum dado anterior se torna relevante para a tarefa atual.

Essa seleção é fundamental.

Uma implementação adequada de memória de longo prazo não consiste em enviar todo o histórico novamente ao modelo. Isso aumentaria rapidamente o volume de contexto, elevaria o custo e poderia ultrapassar os limites suportados pelo modelo.

O comportamento correto consiste em recuperar apenas os registros relevantes para a interação atual. Essa recuperação costuma ser realizada por mecanismos de busca vetorial, aplicando o mesmo princípio utilizado em arquiteturas RAG.

Também podemos dividir a memória de longo prazo em dois tipos.

A memória episódica registra eventos específicos. Ela pode armazenar, por exemplo, que determinado usuário apresentou uma reclamação em uma interação realizada anteriormente.

A memória semântica mantém informações generalizadas, como preferências ou características persistentes do usuário. Ela poderia registrar que determinada pessoa prefere respostas mais objetivas ou que trabalha com um contexto regulatório específico.

A primeira se aproxima de um histórico de eventos. A segunda se aproxima de um perfil.

Memória e risco de compliance

A memória de longo prazo acrescenta uma preocupação que não existe na mesma intensidade na memória temporária: retenção de dados pessoais.

Quando uma requisição termina, a memória de curto prazo tende a desaparecer naturalmente. A memória persistente não.

Se um usuário solicitar a exclusão de seus dados, o sistema precisa possuir mecanismos reais para localizar e remover essas informações. Não basta simplesmente deixar de utilizá-las.

Essa exigência torna a memória de longo prazo um componente de arquitetura, segurança e governança.

Por esse motivo, o agente responsável pela geração do Termo de Consentimento Livre e Esclarecido no Trial Forge não utiliza memória persistente.

Cada protocolo é processado de forma independente. O agente não precisa recordar protocolos anteriores, usuários anteriores ou decisões tomadas em outras sessões.

Nesse caso, adicionar memória de longo prazo não melhoraria a tarefa. Apenas aumentaria o custo de armazenamento, a complexidade operacional e a superfície de risco relacionada ao tratamento de dados.

Isso demonstra um princípio importante: nem todo agente precisa lembrar.

Quando cada entrada pode ser processada de maneira independente, memória persistente pode ser desnecessária. Já em um assistente que acompanha o mesmo usuário durante semanas, a ausência desse componente pode comprometer completamente a experiência.

Planejamento

O segundo componente é o planejamento.

Planejar significa decompor uma tarefa complexa em etapas menores antes de executar uma ação.

Dentro desse componente, encontramos técnicas como raciocínio estruturado, decomposição em subobjetivos, autocrítica e reflexão.

O raciocínio estruturado permite que o modelo organize os passos necessários antes de produzir a resposta final.

A decomposição em subobjetivos divide uma meta ampla em tarefas menores, executáveis e mais fáceis de verificar.

A reflexão acrescenta uma nova etapa ao fluxo. Depois de produzir uma resposta inicial, o agente avalia criticamente o próprio resultado e gera uma versão revisada.

Esse comportamento pode aumentar a qualidade, mas também gera custo adicional.

Uma reflexão completa normalmente exige uma nova chamada ao modelo. O agente responde, analisa sua própria resposta e depois produz uma nova versão.

Já a decomposição de subobjetivos representa uma decisão mais ampla de fluxo. É necessário definir quantas etapas serão executadas, em que ordem elas ocorrerão e qual componente decidirá quando o processo estará concluído.

Por isso, o planejamento costuma ser um dos elementos mais caros em termos de latência.

Quanto mais o agente analisa antes de agir, maior tende a ser a qualidade da decisão. Entretanto, maior também será o tempo de resposta e o consumo de recursos.

Essa é a mesma relação de trade-off discutida anteriormente.

Ferramentas

O terceiro componente corresponde às ferramentas.

Ferramentas são funções ou serviços que o agente pode acionar para obter informações ou executar operações fora do próprio modelo de linguagem.

Uma ferramenta pode ser uma calculadora, uma busca em documentos, uma consulta a banco de dados, um calendário, uma API corporativa ou um sistema de emissão de documentos.

O ponto mais importante é compreender que utilizar uma ferramenta não significa que o agente está realizando o cálculo ou a consulta internamente.

Ele está delegando uma tarefa para um componente determinístico especializado.

Essa divisão é extremamente valiosa.

Modelos de linguagem são adequados para interpretação contextual e raciocínio sobre informações ambíguas. Sistemas tradicionais continuam sendo mais

confiáveis para cálculos exatos, consultas estruturadas e execução de regras objetivas.

Um agente que tenta resolver internamente um cálculo complexo, quando poderia utilizar uma calculadora, está desperdiçando a possibilidade de combinar o melhor dos dois mundos.

Cada ferramenta adicionada, entretanto, cria uma nova decisão.

O agente precisa reconhecer quando deve utilizá-la, quais parâmetros deve fornecer e como interpretar o resultado retornado.

Por isso, adicionar ferramentas indiscriminadamente aumenta a complexidade do planejamento e amplia a possibilidade de decisões incorretas.

Uma ferramenta deve possuir um caso de uso claro e frequente. Ela não deve ser incluída apenas como precaução.

Ação

O quarto componente é a ação.

A ação representa o passo executado no mundo real depois que o agente interpreta o contexto e decide o que fazer.

Responder ao usuário é uma ação.

Consultar uma ferramenta também é uma ação.

Atualizar um sistema corporativo, criar um registro, disparar um fluxo de trabalho ou gerar um documento são igualmente ações.

Entretanto, possuir capacidade de ação não significa atuar sempre de forma autônoma.

Um agente bem desenhado pode ter como uma de suas ações possíveis interromper o processo e solicitar aprovação humana.

Em situações de alto risco, essa pode ser a ação mais importante.

No Trial Forge, o agente responsável pelo termo de consentimento pode gerar um rascunho, mas não pode publicá-lo como documento oficial.

Sua ação termina quando o rascunho é produzido.

A transformação desse conteúdo em versão oficial depende do Approval Gate e da decisão do especialista regulatório.

Essa limitação não existe por falta de capacidade técnica. Ela representa uma decisão consciente de arquitetura.

Dimensionando o agente do Trial Forge

Ao aplicar esses quatro componentes ao agente responsável pelo Termo de Consentimento Livre e Esclarecido, obtemos uma arquitetura deliberadamente simples.

A memória é mínima e limitada à execução atual. O protocolo em processamento, as cláusulas recuperadas e o texto produzido formam o contexto temporário da tarefa.

Não existe necessidade de guardar informações entre sessões.

O planejamento também é simples.

O agente precisa executar duas etapas principais: recuperar as cláusulas regulatórias aplicáveis e gerar o rascunho do documento com base no protocolo aprovado.

Não há necessidade de múltiplos ciclos de reflexão, autocrítica ou decomposição sofisticada.

Adicionar essas etapas aumentaria o tempo de resposta sem garantir uma melhoria proporcional no resultado.

A única ferramenta necessária é a busca na base de cláusulas regulatórias, implementada por meio de RAG.

O agente não precisa acessar calculadoras, executar código ou consultar diversos sistemas externos.

Por fim, sua ação é restrita à geração do rascunho.

Ele nunca publica o documento autonomamente.

Essa arquitetura é enxuta, mas não incompleta. Ela está corretamente dimensionada para o problema.

O teste de realidade

Para verificar se um agente está bem dimensionado, utilizo um teste composto por cinco perguntas.

A primeira pergunta é: se a memória de longo prazo fosse removida, a tarefa continuaria funcionando adequadamente?

Se a resposta for positiva, provavelmente essa memória não deveria existir.

A segunda pergunta avalia o planejamento: se ele fosse reduzido pela metade, a qualidade cairia de forma perceptível?

Caso a resposta seja negativa, o agente pode estar realizando etapas desnecessárias.

A terceira pergunta analisa as ferramentas: cada ferramenta possui uma necessidade clara e recorrente ou foi adicionada apenas por precaução?

Ferramentas sem utilização concreta aumentam complexidade e ampliam a superfície de decisão.

A quarta pergunta avalia a ação final: ela é reversível?

Quando a resposta for negativa, deve existir um Approval Gate antes de qualquer efeito real.

A quinta pergunta retoma o orçamento de trade-offs: esse dimensionamento cabe nos limites de latência e custo definidos para o componente?

Essas perguntas ajudam a identificar tanto agentes superdimensionados quanto agentes insuficientes para a tarefa.

Agentes com necessidades diferentes

Para compreender essa diferença, podemos comparar o agente do Trial Forge com um agente de atendimento de uma companhia aérea.

O agente da companhia aérea pode precisar de memória persistente para recuperar o histórico do passageiro, suas viagens anteriores e preferências.

Seu planejamento pode envolver diversas etapas para decidir entre remarcação, reembolso ou compensação.

Ele também pode utilizar várias ferramentas, como consulta de voos, emissão de bilhetes, sistema de pagamentos e atualização de reservas.

Por fim, suas ações podem alterar diretamente uma viagem real.

Nesse cenário, os quatro componentes aparecem em níveis muito mais elevados de sofisticação.

Isso não significa que esse agente tenha uma arquitetura melhor do que a utilizada no Trial Forge.

Significa apenas que a tarefa possui exigências diferentes.

Quando utilizar um único agente

Uma arquitetura single-agent é especialmente adequada quando a tarefa pode ser concluída em poucas etapas, utiliza um conjunto limitado de ferramentas e não exige conhecimentos especializados de vários domínios independentes.

Ela também é apropriada quando o orçamento de latência e custo não justifica a coordenação entre múltiplos agentes.

Sistemas multiagentes acrescentam comunicação, sincronização, resolução de conflitos e troca de contexto. Essa complexidade somente se justifica quando um

único agente já não consegue executar adequadamente todas as responsabilidades.

O agente do Trial Forge representa um bom exemplo de arquitetura single-agent. Sua tarefa é específica, o planejamento é curto, a ferramenta utilizada é bem definida e a ação permanece limitada.

Quando uma solução começa a exigir muitos domínios de conhecimento, memória extensa e grande quantidade de ferramentas, passa a existir um sinal de que a arquitetura talvez precise evoluir para um modelo multiagente.

O ciclo do agente

Memória, planejamento, ferramentas e ação não funcionam de forma isolada.

Esses componentes operam dentro de um ciclo.

O agente utiliza a memória para compreender o contexto atual. Em seguida, planeja o próximo passo. Depois, decide se precisa acionar uma ferramenta ou executar outra ação. Por fim, observa o resultado obtido e verifica se a tarefa foi concluída.

Caso ainda exista trabalho pendente, o ciclo começa novamente.

É essa repetição que produz a impressão de raciocínio contínuo.

Arquiteturalmente, continuamos trabalhando com uma máquina de estados. A diferença é que existe um componente não determinístico decidindo qual será o próximo estado a cada iteração.

Ao analisar qualquer produto apresentado como agente, recomendo sempre procurar esses quatro elementos.

Que tipo de memória existe?

Quantas etapas de planejamento são executadas?

Quais ferramentas estão disponíveis?

Quais ações podem ser realizadas?

Existe aprovação humana antes de ações irreversíveis?

Essas perguntas permitem separar uma arquitetura real de uma simples classificação comercial. Mais importante, ajudam a dimensionar o agente de acordo com a tarefa, evitando tanto a falta de capacidade quanto o excesso de complexidade.

Capítulo 2: Arquiteturas Single-Agent (PT-2)

No capítulo anterior, apresentei a anatomia de um agente inteligente, composta por quatro elementos fundamentais: memória, planejamento, ferramentas e ação. Mostrei que um agente não deve ser definido por discursos de marketing, mas pela forma como esses componentes são organizados para resolver um problema específico. Um agente bem projetado não é aquele que possui mais recursos, mas aquele que utiliza exatamente os recursos necessários para cumprir sua função.

Entretanto, ainda falta compreender como esses componentes trabalham em conjunto. Até agora, analisamos cada um deles de forma isolada, mas, na prática, eles nunca operam separadamente. Eles fazem parte de um ciclo contínuo de tomada de decisão, no qual o agente avalia a situação, executa ações, observa os resultados e decide novamente qual deve ser o próximo passo.

Esse ciclo é conhecido como **ReAct**, sigla derivada das palavras *Reasoning* e *Acting*, representando a integração entre raciocínio e ação. Trata-se de um dos padrões mais importantes da arquitetura de agentes modernos, pois estabelece justamente o que diferencia um agente inteligente de uma simples chamada a um modelo de linguagem.

O que diferencia um agente de uma chamada ao modelo

Uma chamada tradicional para um modelo generativo possui um fluxo bastante simples.

O usuário fornece um prompt, o modelo processa esse contexto e produz imediatamente uma resposta.

Independentemente de essa resposta estar completa, incompleta ou até mesmo incorreta, o processo termina naquele instante.

O modelo não verifica se possui informações suficientes, não consulta novas fontes de dados e tampouco reconsidera seu próprio raciocínio antes de responder.

O comportamento de um agente baseado no padrão ReAct é completamente diferente.

Antes de concluir sua resposta, ele pode reconhecer que ainda não possui contexto suficiente. Em vez de responder imediatamente, decide executar alguma ação intermediária para obter novas informações. Somente depois de analisar esse novo contexto é que decide se já pode produzir uma resposta definitiva ou se ainda precisa realizar outra etapa de investigação.

Essa capacidade de interromper momentaneamente o fluxo, buscar informações adicionais e reavaliar a situação constitui uma das principais características de um agente autônomo.

É justamente essa diferença que faz com que muitos assistentes modernos aparentem ser significativamente mais inteligentes do que sistemas baseados apenas em uma única interação com o modelo.

A origem do padrão ReAct

O padrão ReAct foi formalizado a partir do trabalho *ReAct: Synergizing Reasoning and Acting in Language Models*, publicado em 2022 por pesquisadores liderados por Shunyu Yao.

Desde então, esse modelo tornou-se uma referência para praticamente todos os frameworks de agentes disponíveis atualmente.

Independentemente da tecnologia utilizada, seja ela baseada em plataformas proprietárias, bibliotecas de código aberto ou serviços em nuvem, quase todas implementam alguma variação desse mesmo ciclo.

Por esse motivo, considero muito mais importante compreender o funcionamento do padrão do que memorizar funcionalidades específicas de uma ferramenta.

Frameworks mudam rapidamente.

O ciclo arquitetural permanece.

As quatro etapas do ciclo

O funcionamento do ReAct pode ser dividido em quatro etapas principais.

A primeira delas corresponde ao **pensamento**.

Nesse momento, o agente analisa tudo o que já sabe sobre o problema e identifica quais informações ainda são necessárias para produzir uma resposta confiável.

Essa etapa está diretamente relacionada ao componente de planejamento apresentado anteriormente.

O agente avalia o contexto disponível, organiza suas hipóteses e define qual será a próxima ação.

A segunda etapa corresponde à **ação**.

Em vez de responder imediatamente, o agente decide utilizar uma ferramenta.

Essa ferramenta pode realizar uma busca em documentos, consultar uma API corporativa, acessar uma base de conhecimento utilizando RAG, executar um cálculo ou recuperar informações armazenadas em algum sistema.

O aspecto importante é que o agente não tenta resolver tudo sozinho.

Sempre que existe um componente determinístico capaz de executar determinada tarefa com maior precisão, ele delega essa responsabilidade.

A terceira etapa corresponde à **observação**.

Depois que a ferramenta executa sua função, o resultado retorna para o agente.

Entretanto, essa informação não representa ainda a resposta final destinada ao usuário.

Ela passa a integrar o contexto disponível para uma nova etapa de raciocínio.

Em outras palavras, o agente aprende algo novo durante a execução da tarefa.

Por fim, chega o momento da decisão.

Com todas as informações reunidas até aquele instante, o agente responde a uma pergunta bastante simples:

Já possuo contexto suficiente para concluir esta tarefa?

Caso a resposta seja positiva, o ciclo termina e o resultado é entregue ao usuário.

Caso contrário, uma nova iteração é iniciada.

O agente volta à etapa de pensamento utilizando um contexto mais rico do que possuía anteriormente.

Esse processo pode repetir-se diversas vezes até que exista confiança suficiente para finalizar a atividade.

Loops variáveis

Uma característica extremamente importante do padrão ReAct é que o número de iterações nunca é conhecido antecipadamente.

Em um algoritmo determinístico tradicional, o fluxo costuma ser previamente definido.

Independentemente da entrada recebida, as mesmas etapas são executadas na mesma sequência.

No ReAct isso não acontece.

Uma pergunta simples pode ser resolvida após apenas uma iteração.

Uma tarefa mais complexa pode exigir diversas consultas sucessivas antes que o agente reúna todas as evidências necessárias.

Essa diferença altera completamente a forma como projetamos aplicações baseadas em Inteligência Artificial.

Não estamos mais descrevendo uma sequência fixa de instruções.

Estamos construindo um mecanismo capaz de decidir dinamicamente qual será seu próximo passo.

Essa característica também explica a relação direta entre o padrão ReAct e o framework apresentado no módulo anterior.

Quando o caminho necessário para resolver uma tarefa já é conhecido antecipadamente, não existe justificativa para utilizar um agente.

Uma regra determinística continuará sendo mais simples, mais barata e mais previsível.

O ciclo ReAct somente faz sentido quando o próprio processo de resolução depende das informações descobertas ao longo da execução.

A evolução dos assistentes de desenvolvimento

Esse padrão também explica por que ferramentas modernas de desenvolvimento assistido por Inteligência Artificial parecem significativamente mais sofisticadas do que as primeiras gerações de assistentes.

Inicialmente, esses sistemas apenas recebiam uma solicitação e geravam código imediatamente.

Hoje o comportamento costuma ser diferente.

O agente primeiro analisa o projeto atual.

Depois consulta arquivos relacionados.

Executa testes.

Observa os resultados obtidos.

Identifica falhas.

Corrige os problemas encontrados.

Somente depois apresenta uma solução ao desenvolvedor.

A sensação de inteligência não decorre apenas da qualidade do modelo utilizado.

Ela surge principalmente porque o agente percorreu diversas voltas do ciclo antes de entregar sua resposta final.

Observabilidade do ciclo

Como cada iteração produz novas decisões, torna-se fundamental registrar tudo o que acontece durante esse processo.

Para isso utilizo um artefato simples chamado **React Loop Canvas**.

Seu objetivo não é documentar apenas a resposta produzida pelo agente.

O foco está em registrar toda a sequência de decisões intermediárias.

Cada linha representa uma iteração do ciclo.

São registradas as conclusões obtidas durante a etapa de pensamento, a ferramenta utilizada, o resultado retornado durante a observação e a decisão tomada ao final daquela iteração.

Esse histórico funciona como uma poderosa ferramenta de depuração.

Quando um agente produz um comportamento inesperado, raramente o problema está apenas no prompt inicial.

Na maioria das situações, alguma observação intermediária foi interpretada incorretamente, conduzindo o restante do raciocínio para um caminho inadequado.

Sem esse registro, torna-se praticamente impossível compreender por que determinada decisão foi tomada.

Esse conceito conecta-se diretamente à camada de observabilidade apresentada no módulo anterior.

Cada iteração do ReAct representa um conjunto adicional de eventos que precisa ser registrado para permitir auditoria e investigação posterior.

O ciclo aplicado ao Trial Forge

Para compreender melhor esse funcionamento, podemos analisar um exemplo utilizando novamente o Trial Forge.

Considere a geração automática de um Termo de Consentimento Livre e Esclarecido.

O protocolo clínico contém diversas seções opcionais.

Algumas dependem das características específicas do estudo.

Outras somente aparecem quando determinadas condições são satisfeitas.

Imagine, por exemplo, uma seção referente à realização de testes para HIV ou outra relacionada ao assentimento de participantes menores de idade.

O agente não sabe antecipadamente se essas seções devem ou não fazer parte do documento.

Ele precisa descobrir.

Na primeira etapa do ciclo, analisa o protocolo recebido e identifica quais informações ainda são insuficientes para decidir.

Em seguida, consulta a base regulatória por meio do mecanismo de RAG.

O resultado dessa consulta retorna ao agente durante a etapa de observação.

Agora ele já possui novas informações.

Nesse momento surge uma nova decisão.

As evidências recuperadas são suficientes para determinar a presença daquela seção?

Caso positivo, o documento continua sendo elaborado.

Caso negativo, o agente realiza outra consulta, talvez buscando informações adicionais em outra fonte.

Observe que não existe uma quantidade fixa de etapas.

Alguns protocolos serão resolvidos rapidamente.

Outros exigirão diversas consultas até que todas as decisões estejam fundamentadas.

Essa flexibilidade constitui justamente o principal benefício do padrão ReAct.

Quando o ciclo se torna um problema

Embora extremamente poderoso, o ReAct também introduz riscos importantes de arquitetura.

O primeiro deles está relacionado ao crescimento contínuo do custo.

Cada nova iteração normalmente representa novas chamadas para modelos, novas consultas em bases de conhecimento e maior volume de contexto acumulado.

Se não existir um limite adequado, uma tarefa simples pode transformar-se rapidamente em um processo caro e lento.

O segundo risco corresponde aos ciclos sem convergência.

Em algumas situações, o agente continua executando novas iterações sem jamais reunir informações suficientes para encerrar a tarefa.

Isso pode acontecer quando ferramentas retornam dados ambíguos, quando o modelo interpreta incorretamente as observações recebidas ou quando simplesmente não existe informação suficiente disponível.

Nesses casos, o agente permanece preso em um processo contínuo de investigação.

Arquiteturas maduras precisam prever mecanismos capazes de impedir esse comportamento.

Mecanismos de contenção

Existem diferentes estratégias para controlar esses riscos.

A mais simples consiste em estabelecer um limite máximo de iterações.

Depois de determinado número de voltas, o agente interrompe automaticamente o processo.

Outra estratégia bastante comum consiste em definir um orçamento máximo de custo ou de tempo.

Quando esse limite é atingido, o ciclo é encerrado independentemente de o agente considerar que ainda existam informações pendentes.

Também é possível utilizar critérios de confiança.

Se, após sucessivas iterações, a confiança da resposta permanecer abaixo do esperado, o agente deixa de insistir em novas buscas e encaminha a situação para um especialista humano.

Essa decisão conecta novamente o padrão ReAct ao conceito de Approval Gate estudado anteriormente.

Em alguns cenários, a melhor ação que um agente pode executar é reconhecer que não possui informações suficientes para prosseguir autonomamente.

O papel do arquiteto

Ao projetar um agente baseado em ReAct, o papel do arquiteto não consiste apenas em definir quais ferramentas estarão disponíveis.

É igualmente importante decidir quando o ciclo deve terminar, quais limites financeiros podem ser alcançados, qual tempo máximo será aceitável e quais situações exigirão intervenção humana.

Esses critérios fazem parte da arquitetura tanto quanto a escolha do modelo de linguagem.

Sem eles, um agente aparentemente inteligente pode transformar-se rapidamente em um sistema caro, imprevisível e difícil de controlar.

Ao compreender o funcionamento do ciclo ReAct, deixamos de enxergar agentes inteligentes como componentes misteriosos. Passamos a entendê-los como estruturas compostas por etapas bem definidas, capazes de raciocinar, agir, observar e decidir repetidamente até alcançar uma solução adequada. Essa visão arquitetural servirá de base para os próximos capítulos, nos quais aprofundaremos mecanismos de planejamento, uso de ferramentas e evolução para arquiteturas cada vez mais sofisticadas.

Capítulo 3: Arquiteturas Single-Agent (PT-3)

No capítulo anterior, apresentei o padrão ReAct, mostrando como um agente inteligente pode alternar entre raciocínio, ação, observação e decisão para

resolver problemas de maneira iterativa. Vimos que esse ciclo permite buscar novas informações antes de produzir uma resposta definitiva, reduzindo limitações presentes em uma simples chamada a um modelo de linguagem.

Entretanto, existe uma situação bastante comum que o ReAct, por si só, não resolve.

Imagine que o agente concluiu todas as etapas do ciclo. Ele reuniu informações, utilizou ferramentas corretamente, produziu uma resposta consistente e decidiu que já possuía elementos suficientes para finalizar a tarefa. Ainda assim, a resposta entregue pode estar incorreta.

Nesse caso, o problema não está na ausência de informação, mas na forma como essa informação foi sintetizada.

É justamente essa limitação que motiva o padrão conhecido como **Self-Reflection** ou simplesmente **Reflection**.

Por que revisar a própria resposta?

Qualquer pessoa que utilize assistentes baseados em Inteligência Artificial provavelmente já percebeu um comportamento curioso.

Em diversas situações, pedir ao modelo que revise a própria resposta produz uma versão significativamente melhor do que a primeira.

Esse resultado não ocorre porque o modelo adquiriu novos conhecimentos durante a revisão.

O conhecimento disponível continua exatamente o mesmo.

O que muda é o objetivo da tarefa.

Na primeira execução, o modelo recebe a instrução de responder a uma pergunta.

Na segunda execução, sua missão deixa de ser construir uma resposta e passa a ser identificar erros, inconsistências, omissões ou problemas existentes naquilo que ele próprio acabou de produzir.

Esses dois processos utilizam capacidades cognitivas diferentes.

Responder e revisar são atividades distintas.

Essa diferença permite transformar a revisão em um componente arquitetural independente, em vez de tratá-la apenas como um truque de engenharia de prompts.

O padrão Reflection

Na literatura, esse comportamento foi formalizado pelo padrão Reflection, apresentado em pesquisas sobre agentes baseados em linguagem natural, entre elas o trabalho **Reflexion: Language Agents with Verbal Reinforcement Learning**, publicado em 2023.

Esse padrão integra um conjunto de arquiteturas que procuram superar diferentes limitações dos agentes tradicionais.

Enquanto agentes puramente reativos respondem imediatamente às solicitações recebidas, agentes com memória preservam informações entre interações. Agentes baseados em ferramentas ampliam suas capacidades utilizando sistemas externos. O padrão ReAct organiza raciocínio e ação em ciclos sucessivos.

Já o Reflection resolve outro problema.

Ele procura reduzir erros de síntese e inconsistências produzidas mesmo quando todas as informações necessárias já estão disponíveis.

Não se trata de buscar novos dados.

Trata-se de analisar criticamente aquilo que já foi produzido.

As etapas do Reflection

O funcionamento desse padrão pode ser dividido em quatro etapas.

A primeira corresponde à **execução**.

Nessa fase, o agente realiza normalmente a tarefa solicitada, produzindo um resultado inicial.

Esse resultado pode assumir diferentes formas.

Pode ser um documento, uma classificação, um resumo, uma decisão ou qualquer outro artefato gerado pela Inteligência Artificial.

A segunda etapa consiste em transformar esse resultado em objeto de análise.

Ele deixa de representar a resposta final e passa a funcionar como entrada para uma nova execução do modelo.

Na terceira etapa ocorre propriamente a **reflexão**.

Agora o objetivo muda completamente.

O agente recebe instruções específicas para procurar inconsistências, erros factuais, contradições, informações ausentes ou qualquer outra categoria de problema previamente definida pela arquitetura.

É importante perceber que o modelo não está simplesmente relendo sua resposta.

Ele está executando uma tarefa completamente diferente.

Seu objetivo deixa de ser produzir conteúdo e passa a ser encontrar falhas.

Por fim, chega a etapa da **resposta refletida**.

Se nenhum problema relevante for encontrado, o sistema confirma que o resultado pode prosseguir para a próxima etapa da arquitetura.

Caso contrário, o agente produz uma nova versão corrigida ou devolve recomendações indicando exatamente quais aspectos precisam ser revistos.

Quando necessário, o fluxo retorna novamente para a etapa de execução, produzindo uma nova versão que será analisada outra vez pelo processo de reflexão.

Esse ciclo pode repetir-se diversas vezes até que a qualidade desejada seja atingida.

Reflection e ReAct não são a mesma coisa

Embora ambos utilizem ciclos iterativos, Reflection e ReAct resolvem problemas completamente diferentes.

No padrão ReAct, cada nova iteração procura obter informações adicionais.

O agente consulta documentos, executa ferramentas, acessa APIs ou recupera novos dados utilizando mecanismos como RAG.

Seu objetivo é ampliar o contexto disponível antes de responder.

No Reflection, isso não acontece.

Nenhuma ferramenta adicional é utilizada.

Nenhuma informação nova é buscada.

O agente trabalha exclusivamente sobre aquilo que ele mesmo produziu.

Seu objetivo é identificar problemas internos na resposta.

Esses dois padrões não competem entre si.

Na prática, eles costumam aparecer juntos.

Primeiro o agente utiliza o ReAct para reunir todas as informações necessárias.

Somente depois entra em funcionamento o Reflection para verificar a qualidade da resposta produzida.

Essa combinação reduz significativamente a quantidade de erros antes que o resultado seja entregue ao usuário.

Separando execução e crítica

Uma decisão arquitetural importante consiste em manter execução e reflexão como chamadas independentes.

Pode parecer tentador construir um único prompt solicitando ao modelo que responda e revise sua própria resposta ao mesmo tempo.

Na prática, entretanto, essa abordagem costuma produzir resultados inferiores.

Quando os dois objetivos aparecem simultaneamente, o modelo tende a priorizar a geração da resposta, dedicando pouca atenção à análise crítica.

Ao separar essas responsabilidades, cada chamada passa a possuir um objetivo cognitivo claro.

Na primeira execução, o modelo concentra seus esforços na solução do problema.

Na segunda, sua única missão consiste em procurar defeitos.

Essa divisão torna a revisão mais rigorosa e previsível.

Outro aspecto importante diz respeito ao próprio prompt utilizado durante a reflexão.

Solicitações genéricas como "revise este texto" normalmente produzem críticas superficiais ou simplesmente confirmam que tudo parece correto.

Já prompts específicos tendem a gerar resultados significativamente melhores.

Quando o sistema solicita explicitamente a identificação de divergências numéricas, inconsistências regulatórias, ausência de informações obrigatórias ou contradições entre diferentes seções do documento, a reflexão passa a concentrar seus esforços exatamente nesses aspectos.

Quanto mais clara for a categoria de erro procurada, maior tende a ser a qualidade da revisão.

Reflection aplicado ao Trial Forge

No Trial Forge existe um excelente exemplo da utilidade desse padrão.

A plataforma precisa gerar documentos regulatórios tanto em português quanto em inglês.

Esses documentos representam versões oficiais do mesmo conteúdo, mas destinadas a diferentes órgãos reguladores.

O agente inicialmente produz o relatório clínico em português e, posteriormente, gera sua versão em inglês para submissão internacional.

Até esse momento, as duas versões foram criadas independentemente.

A etapa de Reflection começa logo em seguida.

Em vez de simplesmente reler um dos documentos, o agente recebe uma nova tarefa: comparar as duas versões procurando divergências numéricas, diferenças em critérios regulatórios ou qualquer inconsistência relevante.

Imagine que a versão em português informe idade mínima de treze anos para participação em determinado estudo clínico, enquanto a versão em inglês indique doze anos.

Esse problema não representa apenas uma diferença de tradução.

Trata-se de uma inconsistência regulatória capaz de comprometer toda a documentação.

Nesse caso, o agente não tenta decidir autonomamente qual informação está correta.

Sua função consiste em identificar a divergência e encaminhá-la para revisão.

A decisão permanece sob responsabilidade do especialista regulatório por meio do Approval Gate.

Observe que o valor da reflexão não está em corrigir automaticamente todos os problemas.

Seu maior benefício consiste em detectar inconsistências antes que elas alcancem órgãos reguladores ou usuários finais.

O limite estrutural da autorreflexão

Apesar de extremamente útil, Reflection possui limitações importantes.

A principal delas decorre de um fato bastante simples.

Na maioria das implementações, o mesmo modelo responsável por gerar a resposta também realiza sua própria revisão.

Isso significa que muitos dos vieses presentes durante a geração permanecem existindo durante a reflexão.

Se determinado modelo possui tendência sistemática para cometer certo tipo de erro, existe uma probabilidade significativa de que ele também não consiga identificar esse mesmo erro durante a revisão.

Em outras palavras, Reflection reduz erros, mas não elimina completamente seus pontos cegos.

Essa situação lembra bastante o processo de revisão de código realizado por desenvolvedores.

Quando um profissional revisa o próprio código antes de submetê-lo para avaliação da equipe, normalmente consegue identificar erros simples, como omissões, problemas de sintaxe ou pequenos defeitos de lógica.

Entretanto, dificilmente perceberá falhas estruturais decorrentes do mesmo raciocínio utilizado durante o desenvolvimento.

É exatamente por isso que revisões por pares continuam sendo fundamentais em projetos de software.

A presença de um segundo avaliador acrescenta uma perspectiva diferente, reduzindo a probabilidade de ambos cometerem exatamente o mesmo erro.

O mesmo princípio aplica-se aos agentes inteligentes.

Reflection não substitui revisão humana

Essa limitação torna especialmente importante compreender o papel do Approval Gate.

Em tarefas críticas, Reflection jamais deve substituir a validação realizada por especialistas.

Seu objetivo consiste em reduzir o volume de erros que chegará ao revisor humano.

Quando a reflexão identifica problemas simples, esses erros deixam de consumir tempo da equipe responsável pela validação.

Entretanto, decisões envolvendo critérios regulatórios, segurança, aspectos legais ou impactos financeiros continuam exigindo confirmação humana.

A reflexão representa um filtro adicional de qualidade.

Não representa a etapa final de aprovação.

Utilizando modelos diferentes

Uma estratégia bastante eficiente para reduzir os pontos cegos consiste em utilizar modelos distintos durante geração e reflexão.

Nesse cenário, um primeiro modelo produz a resposta inicial.

Posteriormente, um segundo modelo, preferencialmente pertencente a outra família ou até mesmo a outro fornecedor, realiza a análise crítica.

Como esses modelos possuem processos de treinamento diferentes, também tendem a apresentar vieses distintos.

Consequentemente, aumenta a probabilidade de que um deles identifique erros ignorados pelo outro.

Naturalmente, essa estratégia implica maior custo operacional.

Existe uma nova chamada para outro modelo, aumentando consumo financeiro e tempo de processamento.

Entretanto, em componentes classificados como críticos durante o módulo anterior, esse custo adicional costuma representar um excelente investimento.

Mais uma vez aparece o conceito de trade-off arquitetural.

Aceita-se um pequeno aumento de custo para reduzir significativamente o risco de erros caros e irreversíveis.

Dois níveis de reflexão

Na prática, costumo dividir Reflection em dois níveis distintos.

O primeiro corresponde à **reflexão de superfície**.

Seu objetivo é verificar aspectos internos do próprio documento.

São analisadas estrutura, completude, consistência entre diferentes seções e presença de todos os elementos obrigatórios.

Esse tipo de revisão apresenta baixo custo computacional e costuma valer a pena em praticamente todas as tarefas.

O segundo nível corresponde à **reflexão de conteúdo**.

Nesse caso, a resposta gerada é comparada diretamente com uma fonte externa considerada confiável.

No contexto do Trial Forge, isso significa confrontar o documento produzido com o protocolo clínico original.

Essa comparação permite identificar desvios factuais, informações omitidas ou alterações indevidas que não seriam percebidas apenas pela análise interna do texto.

Embora muito mais poderosa, essa verificação também exige maior tempo de processamento e consumo adicional de recursos.

Por isso, nem toda tarefa necessita obrigatoriamente desses dois níveis.

Documentos de baixo risco podem utilizar apenas reflexão de superfície.

Já documentos destinados a processos regulatórios normalmente exigem ambas as verificações.

A importância da fonte de comparação

Existe ainda um aspecto frequentemente negligenciado.

Reflection somente consegue identificar erros presentes dentro do universo de informações ao qual possui acesso.

Se o agente comparar apenas duas versões geradas do mesmo documento, conseguirá detectar divergências entre elas.

Entretanto, caso ambas contenham exatamente o mesmo erro originado na etapa de geração, a reflexão dificilmente perceberá o problema.

Para identificar esse tipo de situação, torna-se indispensável comparar o documento produzido com sua fonte original de informação.

No Trial Forge, essa fonte corresponde ao protocolo clínico.

Somente essa comparação permite verificar se as informações presentes no relatório realmente representam aquilo que estava definido originalmente para o estudo.

Consequentemente, arquiteturas maduras costumam combinar múltiplas estratégias de reflexão.

Primeiro verificam consistência interna.

Depois confrontam o resultado com documentos de referência.

Cada abordagem detecta categorias diferentes de problemas.

Reflection como componente arquitetural

Ao final deste capítulo, torna-se evidente que Reflection não representa apenas uma técnica de engenharia de prompts.

Ele constitui um componente arquitetural responsável por aumentar a confiabilidade das respostas produzidas por agentes inteligentes.

Sua principal contribuição está em separar claramente duas atividades cognitivas distintas: produzir uma resposta e avaliá-la criticamente.

Essa separação permite reduzir inconsistências antes que elas avancem para etapas posteriores do fluxo.

Entretanto, também fica claro que Reflection possui limites bem definidos.

Ele reduz erros, melhora a qualidade das respostas e diminui o trabalho dos revisores humanos, mas não substitui mecanismos de governança nem elimina a necessidade de validação especializada em tarefas de alto risco.

Com essa base estabelecida, estamos preparados para avançar para o próximo componente da anatomia dos agentes inteligentes: a utilização de ferramentas externas. A partir desse ponto, o agente deixa de depender exclusivamente do conhecimento armazenado no modelo e passa a interagir com sistemas externos capazes de ampliar significativamente suas capacidades de decisão e execução.

Capítulo 4: Arquiteturas Single-Agent (PT-4)

Nos capítulos anteriores, analisei diferentes componentes que formam uma arquitetura single-agent. Primeiro, apresentei a anatomia do agente, considerando memória, planejamento, ferramentas e ação. Depois, mostrei como o ciclo ReAct combina raciocínio e execução, permitindo que o agente busque informações antes de concluir uma tarefa. Também apresentei o padrão Reflection, no qual o agente critica o próprio resultado antes de entregá-lo.

Entretanto, ainda falta detalhar uma peça essencial dessa arquitetura: o funcionamento das ferramentas externas.

Sempre que um agente consulta uma base de dados, realiza uma busca regulatória, verifica uma assinatura ou solicita uma operação em outro sistema, existe uma chamada de ferramenta. Essa chamada não pode depender de convenções improvisadas por cada desenvolvedor. Ela precisa seguir um contrato explícito, com parâmetros conhecidos, tipos bem definidos e retornos estruturados.

É essa estrutura que permite transformar a decisão probabilística do modelo em uma operação determinística e controlável.

O que é uma ferramenta em uma arquitetura de agentes

No contexto de agentes inteligentes, considero ferramenta qualquer função externa que o agente possa solicitar durante seu processo de planejamento.

Essa função pode realizar uma busca simples, consultar um banco de dados, acessar uma API corporativa, efetuar um cálculo ou executar algum processamento especializado.

A presença dessas ferramentas representa uma das diferenças mais importantes entre um chatbot convencional e um agente.

Um chatbot limita-se à geração de linguagem. Ele recebe uma entrada e produz uma resposta textual.

Um agente, por outro lado, pode utilizar recursos externos para obter informações e provocar ações. Ele não depende apenas do conhecimento armazenado no modelo.

Entretanto, é importante esclarecer um ponto técnico decisivo: o modelo de linguagem não executa diretamente a ferramenta.

O modelo não acessa o banco de dados por conta própria, não chama uma API real e não executa uma função dentro do ambiente da empresa.

O que ele faz é produzir uma proposta estruturada de chamada. Nessa proposta, informa qual ferramenta deseja utilizar e quais parâmetros devem ser enviados.

A execução real permanece sob responsabilidade do sistema determinístico que envolve o modelo.

Essa separação é fundamental para que o uso de ferramentas seja suficientemente seguro em produção. O modelo propõe uma ação, mas quem controla e executa essa ação é a aplicação.

As quatro etapas de uma chamada de ferramenta

Uma chamada de ferramenta pode ser organizada em quatro etapas: definição, decisão, execução e retorno.

Cada uma delas possui uma função específica e precisa utilizar formatos claros.

Definição da ferramenta

A primeira etapa corresponde à definição, também chamada de esquema.

Nesse momento, descrevo explicitamente o contrato da ferramenta. Esse contrato precisa informar seu nome, sua finalidade e os parâmetros aceitos.

Cada parâmetro deve possuir um tipo definido. Dependendo da necessidade, podemos trabalhar com texto, números, valores booleanos ou listas restritas de opções.

Esse esquema é apresentado ao modelo antes da decisão de chamada.

O modelo utiliza essas informações para compreender o que cada ferramenta faz, em quais situações ela deve ser utilizada e quais dados precisam ser fornecidos.

Uma descrição vaga aumenta a possibilidade de escolha incorreta. Um parâmetro mal tipado amplia a quantidade de valores inesperados que o modelo pode produzir.

Por esse motivo, a qualidade do esquema influencia diretamente a confiabilidade do agente.

Decisão do modelo

Na segunda etapa, o modelo decide se precisa utilizar uma ferramenta.

Essa decisão acontece durante o planejamento ou durante uma iteração do ciclo ReAct.

Com base no contexto atual, o agente escolhe uma das ferramentas disponíveis e preenche os parâmetros exigidos pelo esquema.

Nesse ponto existe comportamento não determinístico.

O modelo interpreta a situação e propõe a chamada que considera mais adequada.

Entretanto, sua liberdade permanece limitada pelo contrato fornecido. Quanto mais preciso for esse contrato, menor será o espaço para ambiguidades.

Execução externa

A terceira etapa corresponde à execução propriamente dita.

A aplicação recebe a proposta do modelo, valida os parâmetros e executa o código real.

Essa execução pode envolver uma consulta em banco de dados, uma chamada para uma API, uma busca vetorial ou qualquer outra operação previamente implementada.

Toda essa etapa é determinística.

O modelo não participa da execução interna da função. Ele apenas aguarda o resultado.

Essa separação permite aplicar as práticas tradicionais de engenharia de software: testes, validação de entrada, controle de acesso, tratamento de exceções, logs e monitoramento.

Retorno estruturado

Na quarta etapa, o resultado da execução volta para o agente.

Esse retorno também precisa obedecer a um formato explícito.

Não considero adequado devolver apenas um parágrafo em linguagem natural, pois o próximo passo do agente precisaria interpretar novamente aquele texto e poderia introduzir novas ambiguidades.

O ideal é retornar uma estrutura com campos nomeados e significados bem definidos.

Esse resultado passa a fazer parte da observação do ciclo ReAct. O agente analisa os dados recebidos e decide se já possui contexto suficiente ou se precisa realizar uma nova ação.

A importância da tipagem

O erro mais comum em esquemas de ferramentas está na ausência de tipagem rigorosa.

Considere um parâmetro destinado a indicar a jurisdição regulatória. Dentro do escopo do Trial Forge, esse campo pode aceitar apenas duas opções: Anvisa ou FDA.

Caso eu declare esse parâmetro como texto livre, o modelo poderá produzir diferentes variações para representar o mesmo valor. Pode utilizar letras maiúsculas, minúsculas, abreviações ou formatos inesperados.

Para uma pessoa, todas essas variações podem parecer equivalentes. Para um sistema determinístico que espera um valor exato, elas representam entradas diferentes.

Esse problema pode não aparecer na primeira execução. Talvez também não apareça nas primeiras dezenas de chamadas.

Entretanto, em algum momento o modelo produzirá uma variação não prevista e o código de execução poderá falhar.

Por isso, costumo afirmar que um esquema ruim não necessariamente quebra na primeira chamada. Ele quebra quando o volume de uso revela uma combinação que ninguém havia considerado.

Quando o universo de valores possíveis é limitado, o esquema deve representar essa limitação de forma explícita.

Essa prática reduz a liberdade desnecessária do modelo e simplifica a implementação determinística.

Esquemas como contratos de interface

Um esquema de ferramenta funciona como um contrato de interface.

Em uma linguagem estaticamente tipada, a assinatura de uma função impede que um valor textual seja enviado quando a operação espera um número.

No tool calling, o objetivo é semelhante.

A diferença está no componente localizado do outro lado do contrato.

Em uma integração tradicional, normalmente temos dois sistemas determinísticos. No caso de agentes, um dos participantes é um modelo que interpreta linguagem natural e trabalha com probabilidades.

Por isso, o contrato precisa ser ainda mais explícito.

Não posso depender de inferências implícitas ou assumir que o modelo sempre produzirá a mesma grafia.

Quanto mais crítico for o processo, mais importante será eliminar interpretações desnecessárias.

Ferramentas de leitura e ferramentas de escrita

Além do formato do esquema, também preciso avaliar o risco associado à ferramenta.

Costumo separar ferramentas em duas categorias principais: leitura e escrita.

Ferramentas de leitura consultam informações sem alterar o estado do mundo real.

Uma busca regulatória, uma consulta a banco de dados, um cálculo ou uma verificação de assinatura pertencem a essa categoria.

Quando ocorre um erro em uma chamada de leitura, normalmente é possível corrigir o problema e executar a operação novamente.

Já as ferramentas de escrita alteram algum estado externo.

Elas podem criar registros, modificar documentos, enviar notificações, atualizar sistemas ou iniciar processos.

Nesse caso, o erro pode ser caro ou irreversível.

Essa classificação retoma diretamente o framework de decisão apresentado no primeiro módulo.

Quando uma ferramenta de escrita possui alto impacto e sua ação não pode ser desfeita, ela deve passar por um Approval Gate antes da execução.

A aprovação posterior não resolve o problema, pois o efeito já terá acontecido.

Notificação de evento adverso

Um exemplo claro de ferramenta de escrita seria a notificação de um evento adverso regulatório.

Essa ferramenta poderia enviar informações para a Anvisa ou para o FDA quando determinado evento precisasse ser reportado dentro de um prazo.

A execução altera o mundo real. Uma vez transmitida, a notificação chega ao órgão regulador e passa a produzir consequências formais.

O esquema dessa ferramenta precisa declarar explicitamente a necessidade de aprovação.

Antes da execução, um profissional responsável pela farmacovigilância deve confirmar os dados, a jurisdição, a classificação e a necessidade de notificação.

Nesse caso, a obrigatoriedade do gate não depende da complexidade técnica da ferramenta.

Uma operação pode ser simples de implementar e, ainda assim, possuir risco elevado.

O critério relevante é a consequência da ação.

Notificar incorretamente ou deixar de notificar quando necessário pode produzir impactos regulatórios, multas e problemas de conformidade. Por isso, o agente pode propor a chamada, mas não deve executá-la autonomamente.

O tratamento estruturado de falhas

O retorno de uma ferramenta precisa ser estruturado não apenas em situações de sucesso, mas também quando a execução falha.

Imagine que a busca de cláusulas regulatórias não encontre nenhum resultado para determinado tema.

A resposta não deveria ser apenas uma mensagem genérica informando que ocorreu um erro.

O retorno deve indicar claramente que nenhum conteúdo foi encontrado e, quando possível, sugerir um próximo passo, como consultar outra jurisdição ou reformular o tema.

Essa distinção é importante porque ausência de resultado não significa necessariamente ausência da cláusula.

Talvez a consulta tenha sido restrita demais. Talvez a informação exista em outra fonte. Talvez o índice precise ser pesquisado com uma expressão diferente.

Quando a falha retorna como texto ambíguo, o agente pode interpretar incorretamente que a informação simplesmente não existe.

Um retorno estruturado permite que ele reconheça a situação e decida de forma mais adequada.

Busca de cláusulas regulatórias no Trial Forge

No Trial Forge, uma das ferramentas principais é a busca de cláusulas regulatórias.

Ela recebe dois parâmetros.

O primeiro é o tema da consulta. Nesse caso, faz sentido utilizar texto livre, pois o universo de assuntos regulatórios é amplo demais para ser enumerado previamente.

O segundo parâmetro é a jurisdição. Como o escopo do projeto considera apenas Anvisa e FDA, esse campo deve aceitar exclusivamente essas duas opções.

Durante a geração de uma seção de assentimento para menores, por exemplo, o agente pode identificar que precisa consultar os requisitos da Anvisa.

O modelo produz a chamada utilizando os valores permitidos pelo esquema.

Em seguida, o sistema executa uma busca vetorial na base regulatória. Essa etapa é completamente determinística e segue o mesmo princípio utilizado em uma arquitetura RAG.

O retorno contém o texto da cláusula e a referência exata de sua origem, incluindo resolução e artigo.

Essa fonte não representa um detalhe adicional.

Ela faz parte do contrato da ferramenta.

A rastreabilidade permite que o especialista responsável pela aprovação verifique rapidamente se a cláusula está correta e atualizada, sem precisar repetir manualmente toda a pesquisa.

Verificação de assinatura eletrônica

Outro exemplo importante é a ferramenta de verificação de assinatura eletrônica.

Ela confirma se a assinatura digital de um participante é criptograficamente válida e se corresponde ao documento analisado.

Essa ferramenta não altera nenhuma informação. Portanto, continua sendo uma ferramenta de leitura.

Entretanto, seu retorno exige mais detalhes do que um simples valor verdadeiro ou falso.

Caso a assinatura seja considerada inválida, o sistema precisa informar o motivo.

Pode existir uma assinatura corrompida, um documento alterado após a assinatura ou um certificado expirado.

Cada uma dessas situações exige um tratamento diferente no fluxo.

Um retorno puramente booleano eliminaria informações importantes e obrigaria o agente a tentar inferir qual problema ocorreu.

O esquema deve justamente impedir esse tipo de ambiguidade.

Esse exemplo demonstra que a classificação entre leitura e escrita não está relacionada à complexidade do retorno.

Uma ferramenta de leitura pode possuir um contrato sofisticado.

O que define seu risco é a existência ou não de alteração em um estado externo.

O problema das integrações artesanais

Até aqui, cada ferramenta possui seu próprio esquema.

Entretanto, surge um problema quando diferentes equipes começam a criar formatos incompatíveis.

Um projeto pode utilizar estruturas em JSON. Outro pode adotar texto separado por vírgulas. Um terceiro pode criar uma sintaxe interna completamente própria.

Nesse cenário, o agente precisa compreender um padrão diferente para cada integração.

Quanto mais ferramentas são adicionadas, maior se torna a variedade de formatos e maior a probabilidade de erro.

Essa abordagem não escala em uma arquitetura corporativa.

Cada nova integração exige desenvolvimento específico, adaptação do agente e manutenção de novos contratos.

É exatamente esse problema que protocolos padronizados procuram resolver.

Model Context Protocol

Um dos protocolos mais relevantes para esse cenário é o Model Context Protocol, conhecido como MCP.

Trata-se de um padrão aberto que define uma forma comum de descrever ferramentas, fornecer contexto ao modelo e devolver resultados estruturados.

Com um protocolo desse tipo, o agente aprende uma linguagem de integração.

Novas ferramentas que seguem o mesmo padrão podem ser conectadas sem a necessidade de criar uma sintaxe completamente diferente para cada uma delas.

O valor não está em uma implementação específica.

Ele está na interoperabilidade.

Esse princípio é semelhante ao funcionamento dos protocolos de rede. Sistemas desenvolvidos por organizações diferentes conseguem comunicar-se porque obedecem a contratos compartilhados.

Em uma arquitetura enterprise como a da Vitalis Pharma, essa padronização torna-se especialmente importante.

Hoje, o Trial Forge utiliza uma ferramenta de busca regulatória.

No futuro, pode precisar consultar o sistema de gestão de estudos clínicos, verificar assinaturas eletrônicas, acessar bases de eventos adversos ou integrar-se a serviços externos de saúde.

Sem um protocolo comum, cada nova ferramenta representa um projeto de integração independente.

Com um padrão compartilhado, boa parte desse esforço transforma-se em configuração e adaptação controlada.

Além disso, a adoção de um protocolo padronizado amplia a compatibilidade com ferramentas produzidas por terceiros.

Conectores para bancos de dados, sistemas de assinatura e plataformas regulatórias podem oferecer suporte nativo ao mesmo protocolo.

Caso a empresa utilize apenas um formato proprietário, será necessário construir adaptadores para cada serviço externo.

Portanto, padronizar a comunicação não serve apenas para organizar o desenvolvimento interno. Serve também para reduzir o trabalho de integração com todo o ecossistema.

Uma ferramenta bem projetada não é apenas uma função que o agente consegue chamar.

Ela precisa possuir um contrato explícito, parâmetros tipados, execução externa controlada e retorno estruturado.

Também precisa comunicar adequadamente seus erros, preservar rastreabilidade e declarar o risco associado às ações que realiza.

Ferramentas de leitura e escrita exigem tratamentos diferentes, principalmente quando existe possibilidade de produzir efeitos caros ou irreversíveis.

Nesses casos, o agente deve limitar-se a propor a operação, mantendo a execução condicionada à aprovação humana.

Por fim, protocolos padronizados como o MCP permitem que esse modelo cresça de forma sustentável. Eles evitam que cada nova ferramenta introduza uma linguagem diferente e reduzem o esforço necessário para integrar serviços internos e externos.

Com essa peça, completamos os principais elementos isolados de uma arquitetura single-agent: memória, raciocínio iterativo, reflexão e utilização de ferramentas. O próximo passo consiste em reunir todos esses componentes em um protótipo único, observando como eles colaboram durante a execução completa de uma tarefa.

Capítulo 5: Arquiteturas Single-Agent (PT-5)

Ao longo deste módulo, construí os principais elementos que compõem uma arquitetura single-agent. Inicialmente, apresentei a anatomia de um agente, formada por memória, planejamento, ferramentas e ação. Em seguida, mostrei como o padrão ReAct organiza o ciclo entre raciocínio e execução, estabelecendo limites para impedir que o agente permaneça indefinidamente tentando resolver um problema.

Depois, apresentei o padrão Reflection, mostrando como um agente pode revisar criticamente sua própria resposta antes de entregá-la, sempre como um mecanismo complementar ao Approval Gate e nunca como um substituto da validação humana em tarefas críticas. Por fim, analisei o uso de ferramentas externas, destacando a importância de contratos tipados e protocolos padronizados para que o agente possa interagir com sistemas externos de maneira segura e previsível.

Neste capítulo, deixo de analisar cada componente individualmente e passo a integrá-los em um único protótipo funcional. O objetivo é compreender como esses elementos trabalham em conjunto para resolver uma tarefa real.

Um agente é resultado de calibragem

Existe um equívoco bastante comum quando começamos a desenvolver agentes inteligentes.

Muitas pessoas imaginam que um agente robusto deve possuir memória permanente, múltiplas ferramentas, reflexão obrigatória em todas as respostas e um ciclo de raciocínio praticamente ilimitado.

Na prática, isso raramente representa uma boa arquitetura.

Um agente não deve ser construído para utilizar todos os recursos possíveis. Ele deve possuir exatamente os recursos necessários para resolver a tarefa para a qual foi projetado.

Toda funcionalidade adicional aumenta custo, tempo de resposta, consumo computacional, superfície de auditoria e complexidade operacional.

Por esse motivo, a arquitetura de um agente não consiste em adicionar componentes continuamente, mas em calibrar cuidadosamente a intensidade com que cada um deles será utilizado.

Cada tarefa exige uma combinação diferente de memória, planejamento, reflexão e ferramentas.

O papel do arquiteto é encontrar esse equilíbrio.

Calibrando o agente do Trial Forge

Podemos observar essa calibragem utilizando novamente o Trial Forge como referência.

O agente responsável pela geração da seção de assentimento do Termo de Consentimento Livre e Esclarecido foi deliberadamente projetado para ser simples.

A memória utilizada é exclusivamente temporária.

Cada protocolo clínico é processado de forma independente. O agente não mantém informações persistentes entre diferentes estudos porque isso não produz benefícios reais para essa tarefa.

Ao contrário, armazenar protocolos anteriores poderia até aumentar o risco de utilização acidental de informações desatualizadas.

O planejamento também foi mantido sob controle.

O ciclo ReAct possui limite explícito de quatro iterações.

Esse limite não é decidido dinamicamente pelo modelo.

Ele faz parte da arquitetura implementada pelo orquestrador.

Sempre que esse limite é atingido sem que exista uma conclusão satisfatória, o agente interrompe automaticamente sua execução.

Outro aspecto importante diz respeito à reflexão.

Embora o padrão Reflection tenha sido apresentado anteriormente como um recurso importante para aumentar a qualidade das respostas, ele não aparece neste protótipo.

Essa ausência não representa uma limitação.

Ela é uma decisão consciente de arquitetura.

A tarefa executada pelo agente é suficientemente direta para que a validação humana realizada posteriormente pelo Approval Gate seja considerada suficiente.

Adicionar uma etapa de autorreflexão aumentaria custo e latência sem produzir benefícios proporcionais.

Por fim, existe apenas uma ferramenta disponível.

O agente pode consultar a base regulatória para recuperar cláusulas específicas necessárias durante a elaboração do documento.

Nenhuma outra ferramenta foi adicionada.

Essa decisão reduz complexidade, simplifica auditorias e mantém o comportamento do agente mais previsível.

O Blueprint do protótipo

Sempre que projeto um novo agente, utilizo um artefato denominado Prototype Blueprint Canvas.

Esse documento organiza as principais decisões relacionadas à calibragem dos componentes.

Para cada elemento da arquitetura, registro não apenas sua configuração, mas principalmente a justificativa da escolha.

No caso da memória, procuro responder por que determinada tarefa necessita ou não de persistência entre diferentes sessões.

No planejamento, defino se haverá ciclos iterativos, qual será o limite máximo de interações e quais condições determinam o encerramento do processo.

Na reflexão, justifico se ela realmente agrega valor à tarefa ou se representa apenas consumo adicional de recursos.

Por fim, documento todas as ferramentas disponíveis, seus contratos e os critérios que justificam sua utilização.

Esse processo transforma decisões subjetivas em decisões arquiteturais explícitas.

Mais importante do que copiar a configuração utilizada no Trial Forge é compreender por que cada componente recebeu exatamente aquele nível de intensidade.

Percorrendo o fluxo completo

Depois de calibrar o agente, podemos acompanhar sua execução do início ao fim.

A tarefa utilizada como exemplo continua sendo a geração da seção de assentimento para estudos clínicos envolvendo participantes menores de idade.

Tudo começa quando o protocolo do estudo chega ao Gateway da arquitetura AI-First.

Após autenticação e validação inicial, a solicitação é encaminhada ao orquestrador.

Nesse momento, o orquestrador identifica qual tarefa deve ser executada.

Ele reconhece que o objetivo consiste em gerar a seção de assentimento correspondente ao protocolo recebido.

A partir desse ponto inicia-se o ciclo ReAct.

Na primeira etapa, o agente analisa o protocolo e procura identificar se existem participantes menores de idade.

Essa análise corresponde ao momento de raciocínio.

Em seguida, decide consultar o próprio protocolo utilizando os mecanismos de recuperação de contexto disponíveis.

Após essa consulta, entra na etapa de observação.

O resultado confirma que o estudo realmente contempla participantes com idade entre doze e dezessete anos.

Com essa informação disponível, o agente identifica a necessidade de recuperar a cláusula regulatória correspondente.

Nesse momento ocorre a chamada da ferramenta.

A solicitação é realizada utilizando o contrato previamente definido.

O tema informado corresponde ao assentimento de menores e a jurisdição selecionada é a Anvisa.

A ferramenta executa a busca na base regulatória e devolve tanto o conteúdo da cláusula quanto sua referência oficial.

Com essas informações disponíveis, o agente produz a seção correspondente do documento.

Por fim, antes que esse conteúdo passe a integrar oficialmente o Termo de Consentimento, entra em funcionamento o Approval Gate.

Um especialista regulatório revisa o texto gerado, confirma sua adequação e somente então autoriza sua utilização.

Esse fluxo reúne todos os componentes estudados ao longo do módulo.

Memória temporária, ciclo ReAct, ferramenta externa e aprovação humana passam a trabalhar de forma integrada.

Flexibilidade durante a execução

Uma característica importante desse fluxo é que ele não representa uma sequência fixa de etapas.

Caso o protocolo analisado não envolvesse participantes menores de idade, o comportamento do agente seria diferente.

Durante a etapa de observação, o sistema identificaria que nenhuma cláusula específica de assentimento seria necessária.

Como consequência, a ferramenta responsável pela busca regulatória sequer seria acionada.

O agente encerraria sua execução após um número menor de iterações.

Esse comportamento demonstra claramente a diferença entre um agente e um fluxo determinístico tradicional.

As etapas executadas dependem das informações descobertas durante a própria execução.

O agente adapta seu comportamento conforme o contexto observado.

A importância das decisões de ausência

Outro aspecto frequentemente negligenciado consiste em observar aquilo que propositalmente não aparece na arquitetura.

O agente não mantém memória de longo prazo.

Também não utiliza múltiplas ferramentas apenas porque elas poderiam existir.

Não executa reflexão em todas as respostas.

Essas ausências são tão importantes quanto os componentes presentes.

Cada funcionalidade adicionada produz algum custo.

Memória persistente exige armazenamento e recuperação de contexto.

Cada iteração adicional do ReAct aumenta latência e consumo de tokens.

Reflection normalmente implica novas chamadas ao modelo.

Cada ferramenta representa mais uma superfície potencial de erro.

Um bom agente não é aquele que acumula recursos.

É aquele que utiliza apenas os componentes realmente necessários para sua tarefa.

O orçamento continua existindo

Essa calibragem conecta-se diretamente aos quatro eixos de trade-off apresentados anteriormente.

Adicionar memória persistente aumenta armazenamento e tempo de recuperação.

Permitir mais iterações do ciclo ReAct amplia tanto a latência quanto o custo operacional.

Reflection praticamente dobra a quantidade de chamadas realizadas ao modelo.

Novas ferramentas aumentam complexidade e exigem novos contratos, testes e mecanismos de observabilidade.

Consequentemente, calibrar um agente representa uma aplicação direta do orçamento arquitetural estabelecido anteriormente.

Cada componente consome parte dos recursos disponíveis.

O papel da arquitetura consiste em distribuir esse orçamento da maneira mais eficiente possível.

Implementando o agente

Uma vez definido o Blueprint, o próximo passo consiste em transformar essas decisões em software.

Independentemente da linguagem utilizada, a implementação segue exatamente a estrutura arquitetural definida anteriormente.

O ciclo ReAct passa a ser controlado por um laço de execução com limite máximo de iterações.

Esse limite permanece sob responsabilidade do orquestrador.

O modelo nunca decide sozinho quando interromper sua execução.

Durante cada iteração, o agente analisa o histórico acumulado e decide se já possui informações suficientes para concluir a tarefa ou se precisa acionar alguma ferramenta adicional.

Caso nenhuma ferramenta seja necessária, a resposta é produzida imediatamente.

Quando uma ferramenta é solicitada, o sistema executa a operação determinística correspondente, incorpora o resultado ao histórico da conversa e inicia uma nova etapa de raciocínio.

Se o número máximo de iterações for atingido sem convergência, o processo não termina silenciosamente.

O sistema produz explicitamente um resultado indicando que o caso deve ser encaminhado para revisão humana.

Esse comportamento preserva previsibilidade e evita que o agente permaneça indefinidamente tentando resolver um problema.

Critérios de parada

Embora o limite de iterações seja bastante comum, ele não representa a única possibilidade.

Dependendo da tarefa, o encerramento do ciclo pode ser baseado em diferentes critérios.

Uma arquitetura pode estabelecer um tempo máximo de execução.

Caso esse tempo seja ultrapassado, o agente interrompe automaticamente sua atividade.

Outra possibilidade consiste em definir um orçamento financeiro máximo para cada requisição.

Quando o consumo estimado ultrapassar determinado limite, novas chamadas deixam de ser realizadas.

Também é possível combinar diferentes critérios simultaneamente.

O aspecto realmente importante é que o critério de parada permaneça explícito.

Ele não deve depender exclusivamente da decisão probabilística do modelo.

Essa responsabilidade pertence à arquitetura.

A missão prática

Ao construir seu próprio agente, recomendo seguir quatro etapas principais.

Primeiro, escolha uma tarefa real e desenhe seu Blueprint, justificando cuidadosamente a intensidade atribuída a cada componente.

Em seguida, descreva completamente o esquema da ferramenta mais importante utilizada pelo agente.

Depois, documente explicitamente o critério de parada adotado.

Por fim, implemente o ciclo completo utilizando um modelo real e pelo menos uma ferramenta externa.

A implementação pode utilizar modelos executados localmente, permitindo experimentar toda a arquitetura sem necessidade de depender de serviços pagos durante a fase de aprendizado.

Essa escolha reduz barreiras de entrada e permite concentrar a atenção na arquitetura propriamente dita.

Naturalmente, ambientes de produção exigirão decisões diferentes, considerando requisitos de escalabilidade, conformidade, disponibilidade e desempenho.

Evitando superdimensionamento

Ao longo da minha experiência, percebo um padrão recorrente.

Entre todos os componentes estudados neste módulo, o mais frequentemente superdimensionado é a memória de longo prazo.

Muitas equipes adicionam persistência simplesmente porque acreditam que poderá ser útil futuramente.

Entretanto, cada informação armazenada gera custos permanentes de recuperação, auditoria, conformidade e proteção de dados.

Por isso, costumo seguir um princípio bastante simples.

Começo sempre pelo agente mais enxuto possível.

Novos componentes são adicionados apenas quando uma necessidade concreta aparece durante a evolução do sistema.

Essa estratégia reduz complexidade sem comprometer futuras expansões.

Preparando a evolução

Ao concluir este módulo, já somos capazes de construir um agente único completamente funcional.

Sabemos calibrar memória, controlar ciclos de raciocínio, integrar ferramentas externas, utilizar mecanismos de reflexão quando necessário e estabelecer critérios claros para encerramento da execução.

Entretanto, também começamos a perceber uma limitação importante.

O Trial Forge não precisa produzir apenas um documento.

Além do Termo de Consentimento, também gera protocolos clínicos, relatórios finais de estudos e diversos outros artefatos que precisam permanecer consistentes entre si.

Concentrar todas essas responsabilidades em um único agente faria com que ele acumulasse conhecimento sobre múltiplos domínios, diferentes formatos documentais e diversos processos regulatórios.

Com o tempo, esse agente se tornaria excessivamente generalista, difícil de auditar e igualmente difícil de evoluir.

A alternativa natural consiste em distribuir essas responsabilidades entre múltiplos agentes especializados.

Cada um deles permanece responsável por um domínio específico, aplicando exatamente os mesmos princípios estudados neste módulo, mas colaborando com outros agentes durante a execução de processos mais complexos.

Essa evolução marca a transição para o próximo módulo da disciplina, no qual deixaremos de estudar agentes individuais para compreender como arquiteturas multiagentes coordenam conhecimento, compartilham responsabilidades e colaboram de forma organizada para resolver problemas corporativos de maior escala.

Módulo 03: Arquiteturas Multi-Agent

Capítulo 01: Arquiteturas Multi-Agent (PT-1)

Ao longo dos dois módulos anteriores, construímos a base necessária para compreender e projetar um agente único. Estudamos sua anatomia, o ciclo de raciocínio, os mecanismos de reflexão, o uso de ferramentas e os protocolos responsáveis por estruturar essas interações.

Agora, porém, o problema do Trial Forge se torna maior.

Na prática, a plataforma não precisa gerar apenas um tipo de documento. Ela deve produzir o Termo de Consentimento Livre e Esclarecido, o protocolo do estudo e o relatório final do estudo clínico. Esses documentos possuem objetivos

diferentes, públicos distintos, formatos próprios e exigências regulatórias específicas.

Mais importante do que isso, todos eles precisam permanecer consistentes entre si.

É nesse ponto que surge a necessidade de uma arquitetura multi-agent. O desafio já não consiste apenas em fazer um agente executar bem uma tarefa individual. Passa a ser necessário coordenar múltiplos agentes especializados, compartilhar estado entre eles e impedir que diferentes partes do sistema produzam versões incompatíveis da mesma realidade.

O limite natural de um agente único

Um agente único, mesmo quando corretamente calibrado, possui um limite natural de escopo.

Ele pode dominar profundamente uma tarefa, como gerar um documento específico ou responder a uma categoria bem definida de perguntas. Entretanto, quando o sistema passa a lidar com múltiplas atividades especializadas, cada uma com vocabulário, ferramentas, público e risco próprios, concentrar tudo em um único agente cria um problema estrutural.

Não se trata apenas de performance.

Também não se trata apenas de escolher um modelo maior ou permitir mais iterações de raciocínio.

O problema está na acumulação de responsabilidades.

No Trial Forge, um estudo clínico completo produz pelo menos três documentos principais.

O primeiro é o Termo de Consentimento Livre e Esclarecido, também conhecido como ICF. Esse documento precisa utilizar linguagem acessível ao participante do estudo. Ele deve explicar procedimentos, riscos e direitos de forma

compreensível para uma pessoa que não necessariamente possui conhecimento médico ou regulatório.

Também pode conter seções condicionais. Dependendo das características do estudo, pode ser necessário incluir informações sobre testes de HIV, participação de menores de idade e assentimento, entre outras situações previstas pelas normas aplicáveis.

O segundo documento é o protocolo do estudo.

Diferentemente do termo de consentimento, o protocolo utiliza linguagem altamente técnica. Ele descreve metodologia, critérios de inclusão e exclusão, cronograma, desenho experimental e diferentes procedimentos do estudo.

Seu público é formado por pesquisadores, especialistas e membros de comitês de ética. Portanto, o nível de detalhamento e o vocabulário empregados são completamente diferentes daqueles utilizados no documento direcionado ao participante.

O terceiro documento é o relatório de estudo clínico, conhecido como CSR.

Esse relatório apresenta a síntese final dos resultados, incluindo análises estatísticas e informações estruturadas conforme padrões internacionais, como o ICH E3.

Além disso, pode precisar ser produzido em diferentes idiomas para submissão simultânea a órgãos reguladores distintos, como a Anvisa e o FDA.

Um agente generalista responsável por todos esses documentos precisaria lidar, ao mesmo tempo, com três vocabulários, três públicos e diferentes níveis de risco regulatório.

Essa concentração de responsabilidades torna o raciocínio mais complexo e dificulta a identificação da origem dos erros.

O problema de auditoria

Imagine que um agente generalista produza uma informação incorreta em um relatório.

Como identificar exatamente qual parte de sua lógica falhou?

O erro pode ter surgido durante a interpretação da metodologia do estudo, durante a adaptação da linguagem para o paciente, durante a síntese estatística ou na aplicação das regras regulatórias.

Como todas essas responsabilidades permanecem dentro do mesmo ciclo de raciocínio, a origem da falha torna-se difícil de isolar.

Um agente que faz tudo também é um agente no qual tudo pode dar errado ao mesmo tempo.

Essa ausência de fronteiras claras aumenta significativamente o custo de auditoria e depuração.

A alternativa arquitetural consiste em dividir o trabalho entre agentes especialistas.

Podemos criar um agente responsável pelo ICF, outro responsável pelo protocolo e um terceiro dedicado ao CSR.

Cada agente passa a possuir um conjunto próprio de ferramentas, vocabulário, nível de reflexão e regras de validação compatíveis com o risco do documento que produz.

Essa divisão melhora a especialização e cria fronteiras claras de responsabilidade.

Entretanto, ela também introduz um novo problema: a coordenação.

O custo de dividir

A divisão em múltiplos agentes não é gratuita.

Um agente único não precisa coordenar seu trabalho com outras entidades autônomas. Quando criamos vários especialistas, adicionamos um novo custo arquitetural.

Esse custo envolve comunicação, sincronização, compartilhamento de estado, resolução de conflitos e tratamento de falhas distribuídas.

Três agentes excelentes em seus próprios domínios ainda podem formar um sistema ruim quando a coordenação entre eles é inadequada.

Por isso, antes de dividir uma tarefa, é necessário comparar dois custos.

De um lado está o custo de coordenação entre múltiplos agentes.

Do outro está o custo de auditoria de um agente generalista.

Um agente único é mais simples de coordenar, mas costuma ser muito mais difícil de investigar quando algo sai errado.

Já uma arquitetura com especialistas exige mais comunicação, mas reduz o espaço de busca durante a depuração.

Se o CSR apresentar um problema, sabemos imediatamente qual agente deve ser analisado primeiro.

Não precisamos desembaraçar um único raciocínio que tentou resolver três problemas diferentes ao mesmo tempo.

Quando a especialização vale a pena

A pergunta central não é simplesmente se o sistema possui tarefas diferentes.

Praticamente todo sistema corporativo possui.

A questão relevante é verificar se essas tarefas exigem vocabulários, ferramentas ou níveis de risco suficientemente distintos para justificar agentes especializados.

Utilizo três dimensões principais para avaliar essa decisão:

1. Vocabulário do domínio.
2. Ferramentas necessárias.
3. Nível de risco regulatório ou operacional.

Quando pelo menos duas dessas dimensões diferem significativamente entre as tarefas, a divisão em múltiplos agentes normalmente compensa o custo adicional de coordenação.

Essa regra, entretanto, não deve ser tratada como uma fórmula rígida.

A divergência de risco, por si só, pode justificar a separação.

Duas tarefas podem utilizar vocabulários e ferramentas semelhantes, mas possuir consequências completamente diferentes em caso de erro.

Uma delas pode tolerar correção posterior, enquanto a outra exige aprovação humana antes de produzir qualquer efeito.

Nesse cenário, separar responsabilidades permite aplicar níveis de reflexão, observabilidade e controle compatíveis com cada risco.

Agentes não são ferramentas

Antes de avançar para os padrões de orquestração, existe uma distinção conceitual que precisa permanecer muito clara: um agente não é uma ferramenta.

Essa diferença continua válida mesmo quando, tecnicamente, um agente chama outro utilizando uma estrutura semelhante àquela usada no tool calling.

Uma ferramenta executa uma função determinística.

Ela possui um esquema fixo, recebe parâmetros, realiza uma operação previsível e devolve um resultado.

Uma ferramenta não possui objetivo próprio, não decide alterar a sequência de execução e não chama outra ferramenta por iniciativa própria.

Um agente é fundamentalmente diferente.

Ele interpreta contexto, toma decisões, escolhe ferramentas e persegue um objetivo.

Também pode identificar que a solicitação recebida está incompleta, pedir esclarecimentos ou recusar a tarefa quando ela ultrapassa seu domínio declarado.

Essa diferença importa porque, em uma arquitetura multi-agent, a comunicação entre dois agentes pode parecer tecnicamente idêntica a uma chamada de ferramenta.

Entretanto, o componente localizado do outro lado da comunicação também possui autonomia.

Ele pode interpretar a solicitação de maneira diferente do esperado ou concluir que precisa de mais contexto antes de prosseguir.

Tratar um agente como uma função pura e previsível é um erro de arquitetura.

Comunicação entre agentes

A diferença entre ferramentas e agentes motivou a criação de protocolos específicos para comunicação agente a agente.

Enquanto protocolos de ferramentas padronizam o acesso a APIs, bancos de dados e outras capacidades externas, protocolos de comunicação entre agentes organizam interações nas quais ambos os lados possuem alguma capacidade de julgamento.

Um desses exemplos é o protocolo Agent-to-Agent, conhecido como A2A.

Seu objetivo é definir uma estrutura comum para mensagens entre agentes, incluindo identificação de remetente e destinatário, manutenção de estado durante conversas e controle explícito das permissões de cada participante.

A existência de um protocolo dedicado demonstra que a comunicação entre agentes representa um problema estruturalmente diferente da chamada de uma ferramenta.

Quando utilizo uma ferramenta, quero incorporar uma capacidade externa determinística ao fluxo.

Quando utilizo outro agente, estou interagindo com um componente que possui autonomia, pode discordar, solicitar esclarecimentos ou negociar antes de executar.

Essa distinção orientará todo o restante do módulo.

Os especialistas do Trial Forge

No Trial Forge, a arquitetura multi-agent começa com três especialistas.

O agente ICF é responsável pela linguagem acessível ao participante e pelas seções condicionais do termo de consentimento.

O agente de protocolo concentra-se na metodologia técnica, nos critérios de inclusão e exclusão e na estrutura formal do estudo clínico.

O agente CSR atua sobre a síntese estatística, os resultados finais e a conformidade com o padrão ICH E3.

Cada um deles, isoladamente, continua sendo um agente relativamente simples.

Sua arquitetura interna pode seguir os mesmos princípios apresentados no módulo anterior: memória calibrada, ciclo de raciocínio limitado, ferramentas específicas e reflexão compatível com o risco da tarefa.

O desafio não está em aumentar a inteligência individual desses agentes.

Está em garantir que todos trabalhem sobre os mesmos fatos.

Consistência compartilhada

Imagine que o agente de protocolo registre idade mínima de doze anos para participação no estudo.

Meses depois, o agente responsável pelo CSR pode utilizar uma versão desatualizada e sintetizar os resultados considerando idade mínima de treze anos.

Nesse caso, cada agente pode ter executado corretamente sua tarefa com base no contexto que recebeu.

Ainda assim, o sistema como um todo produziu uma inconsistência regulatória.

Esse tipo de falha é particularmente perigoso porque não nasce necessariamente dentro de um agente.

Ele surge na comunicação e na sincronização entre agentes diferentes.

Portanto, o problema central de uma arquitetura multi-agent não consiste apenas em produzir bons especialistas.

É necessário garantir que todos compartilhem uma visão consistente do estado atual do trabalho.

Essa necessidade aparece em diversos outros contextos corporativos.

Em atendimento ao cliente, agentes especializados em triagem, suporte técnico e questões financeiras precisam concordar sobre o histórico do mesmo consumidor.

Em um pipeline de análise de dados, agentes responsáveis por extração, transformação e geração de relatórios precisam trabalhar sobre a mesma versão dos dados.

Em qualquer desses casos, o sucesso depende menos da inteligência isolada de cada especialista e mais da qualidade da arquitetura de coordenação.

O padrão orquestrador e trabalhadores

Um exemplo prático de arquitetura multi-agent aparece em sistemas de pesquisa aprofundada.

Nesse tipo de solução, um agente principal recebe a pergunta do usuário, decompõe o problema em subtarefas e distribui essas atividades entre agentes especializados.

Cada subagente pesquisa fontes diferentes e devolve seus resultados para o agente principal, que consolida o material em uma resposta final.

Esse desenho é conhecido como padrão orquestrador e trabalhadores.

O número de agentes pode variar conforme a complexidade da tarefa.

Uma busca factual simples pode utilizar apenas um agente.

Uma comparação direta pode exigir dois ou quatro especialistas.

Pesquisas mais amplas podem acionar mais de dez agentes simultaneamente.

Além disso, cada subagente pode utilizar múltiplas ferramentas dentro da própria subtarefa.

Nesse cenário, existe paralelismo em dois níveis.

O primeiro acontece entre os subagentes.

O segundo ocorre dentro de cada agente, durante o acesso às ferramentas necessárias.

Essa abordagem pode melhorar significativamente a qualidade e reduzir o tempo de conclusão de tarefas complexas.

Entretanto, seu custo é elevado.

Arquiteturas multi-agent consomem muito mais tokens do que interações simples ou mesmo do que agentes únicos.

Isso confirma um princípio já discutido anteriormente: mais capacidade nunca é gratuita.

Capacidade, custo e depuração

A divisão entre especialistas pode produzir resultados melhores e mais rápidos, principalmente quando atividades independentes são executadas em paralelo.

Por outro lado, o consumo de recursos aumenta de maneira significativa.

Cada agente precisa receber contexto, executar seu próprio ciclo de raciocínio, utilizar ferramentas e devolver resultados.

Além disso, o orquestrador precisa consolidar todas essas respostas.

O custo de coordenação aparece tanto no consumo de tokens quanto na complexidade operacional.

Ainda assim, o isolamento entre especialistas produz uma vantagem importante.

Quando uma parte do resultado está incorreta, torna-se muito mais fácil identificar qual agente ou subtarefa deve ser investigado.

Voltamos, portanto, ao mesmo trade-off: múltiplos agentes custam mais para coordenar, mas normalmente custam menos para auditar.

Essa decisão precisa ser tomada de acordo com o contexto, o risco e o valor produzido pela especialização.

Uma arquitetura multi-agent torna-se necessária quando um único agente começa a acumular domínios, ferramentas e níveis de risco incompatíveis dentro do mesmo ciclo de raciocínio.

Dividir responsabilidades entre especialistas melhora a qualidade, facilita auditorias e permite calibrar cada agente de acordo com sua tarefa.

Entretanto, a divisão também introduz comunicação, sincronização, controle de estado e novos pontos de falha.

Por isso, o objetivo não deve ser multiplicar agentes indiscriminadamente.

A decisão correta consiste em comparar o benefício da especialização com o custo de coordenação.

No Trial Forge, a separação entre os agentes de ICF, protocolo e CSR é justificada pelas diferenças de público, vocabulário, estrutura e risco regulatório.

A partir dessa divisão, surge o verdadeiro desafio do módulo: garantir que agentes relativamente independentes permaneçam consistentes sobre os mesmos fatos.

Nos próximos capítulos, aprofundarei os padrões responsáveis por organizar essa comunicação, começando pelas arquiteturas sequencial, paralela e supervisionada.

Capítulo 02: Arquiteturas Multi-Agent (PT-2)

No capítulo anterior, apresentei os motivos que levam uma arquitetura AI-First a evoluir de um único agente para um conjunto de agentes especializados. Utilizando o Trial Forge como referência, mostrei que documentos como o Termo de Consentimento Livre e Esclarecido, o protocolo clínico e o Clinical Study Report possuem objetivos, públicos e requisitos regulatórios diferentes. Essa diferença torna mais eficiente distribuir responsabilidades entre especialistas do que concentrar todo o conhecimento em um único agente.

Entretanto, dividir responsabilidades resolve apenas parte do problema.

Depois que passamos a trabalhar com múltiplos agentes, surge uma nova preocupação: como coordenar a comunicação entre eles? Como garantir que cada agente execute sua parte no momento correto? Como impedir conflitos, retrabalho ou inconsistências entre resultados produzidos por especialistas diferentes?

Neste capítulo, apresento os primeiros padrões de orquestração utilizados em arquiteturas multiagentes. O objetivo não é apenas conhecer cada padrão

isoladamente, mas compreender que eles podem coexistir na mesma solução, resolvendo problemas distintos dentro de um mesmo fluxo de trabalho.

Orquestração não significa escolher um único padrão

Quando começamos a estudar arquiteturas multiagentes, existe uma tendência natural de procurar "o melhor padrão".

Na prática, essa pergunta normalmente está incorreta.

Os padrões de coordenação não competem entre si. Cada um resolve um tipo diferente de dependência existente entre tarefas.

Em uma aplicação corporativa é perfeitamente normal que uma parte do fluxo seja executada de maneira sequencial, outra aconteça em paralelo e determinadas decisões permaneçam sob responsabilidade de um agente supervisor.

A arquitetura deixa de ser uma escolha única e passa a ser uma composição de estratégias.

Essa percepção é importante porque evita decisões excessivamente simplificadas.

Uma arquitetura madura dificilmente utiliza apenas um único modelo de coordenação do início ao fim.

O padrão Sequential

O primeiro padrão é o **Sequential**.

Nesse modelo, cada agente depende diretamente do resultado produzido pelo agente anterior.

O fluxo ocorre em uma sequência claramente definida.

Nenhuma etapa começa antes que a anterior tenha sido concluída.

Esse padrão é adequado quando existe uma relação de dependência lógica entre as atividades.

O segundo agente somente consegue executar corretamente sua tarefa depois que o primeiro disponibiliza todas as informações necessárias.

A principal vantagem dessa abordagem está na previsibilidade.

O fluxo torna-se fácil de compreender, depurar e auditar.

Cada etapa possui uma entrada conhecida e produz uma saída igualmente conhecida para o próximo componente da cadeia.

Por outro lado, existe uma limitação evidente.

Como todas as tarefas aguardam umas pelas outras, a latência total corresponde aproximadamente à soma dos tempos individuais de execução.

Quanto maior a cadeia, maior tende a ser o tempo necessário para concluir todo o processo.

Essa característica faz do Sequential um excelente padrão para atividades fortemente dependentes, mas pouco eficiente quando diferentes tarefas poderiam ocorrer simultaneamente.

Aplicando o Sequential ao Trial Forge

No Trial Forge encontramos diversos exemplos naturais desse comportamento.

Antes de produzir o Termo de Consentimento Livre e Esclarecido, o agente precisa conhecer corretamente o protocolo clínico.

Não faz sentido gerar cláusulas de consentimento antes que os critérios do estudo estejam definidos.

Da mesma forma, o relatório final depende dos documentos produzidos anteriormente.

Existe uma sequência lógica que não pode ser ignorada.

Primeiro o protocolo é estruturado.

Depois são gerados os documentos derivados.

Somente ao final surgem os relatórios consolidados.

Nesse contexto, tentar paralelizar todas essas etapas apenas introduziria inconsistências.

O Sequential preserva a ordem natural do processo.

O padrão Parallel

O segundo padrão corresponde ao **Parallel**.

Diferentemente do Sequential, aqui diferentes agentes executam suas tarefas ao mesmo tempo.

Essa abordagem somente faz sentido quando existe independência entre as atividades.

Se dois agentes não dependem diretamente do trabalho um do outro, não existe motivo para obrigá-los a esperar.

A principal vantagem desse modelo é a redução da latência total.

Em vez de executar três tarefas consecutivas, a arquitetura permite que todas sejam iniciadas simultaneamente.

O tempo de resposta deixa de corresponder à soma das execuções e aproxima-se do tempo da tarefa mais demorada.

Essa característica torna o Parallel especialmente interessante para atividades de pesquisa, recuperação de documentos ou análises independentes.

Entretanto, paralelismo também produz desafios.

Quando todos os agentes terminam sua execução, alguém precisa consolidar os resultados.

Esse processo exige sincronização.

Também pode surgir conflito quando dois agentes produzem conclusões incompatíveis sobre o mesmo assunto.

Por isso, paralelismo normalmente exige um componente adicional responsável pela agregação das respostas.

Paralelismo na prática

Imagine um processo de pesquisa regulatória.

Em vez de utilizar um único agente para consultar sucessivamente diferentes bases de conhecimento, podemos distribuir essa responsabilidade.

Um agente consulta documentos da Anvisa.

Outro pesquisa regulamentações do FDA.

Um terceiro recupera referências científicas.

Todos trabalham simultaneamente.

Ao final, os resultados são reunidos e organizados para a etapa seguinte.

Observe que nenhum desses agentes interfere diretamente no trabalho dos demais.

Cada um executa sua própria investigação.

A arquitetura apenas reúne todas as evidências posteriormente.

Esse comportamento reduz significativamente o tempo necessário para concluir tarefas que originalmente seriam executadas de forma sequencial.

O padrão Supervisor

O terceiro padrão apresentado neste capítulo é o **Supervisor**.

Nesse modelo existe um agente responsável por coordenar o trabalho dos demais especialistas.

É importante compreender que esse agente não substitui os especialistas.

Sua função consiste em distribuir tarefas, acompanhar o progresso e decidir quando cada agente deve ser acionado.

O Supervisor atua como um coordenador.

Ele recebe uma solicitação ampla, decompõe o problema em partes menores, encaminha cada sub tarefa ao especialista adequado e posteriormente consolida os resultados obtidos.

Essa abordagem torna especialmente simples incorporar novos agentes especializados ao longo da evolução da arquitetura.

O supervisor não precisa conhecer profundamente cada domínio.

Ele apenas precisa saber qual especialista deve ser acionado para cada tipo de tarefa.

O Supervisor não faz tudo

Um erro comum consiste em transformar o supervisor em um agente excessivamente inteligente.

Quando isso acontece, ele deixa de coordenar especialistas e passa a competir com eles.

A consequência é o retorno ao problema que tentávamos resolver inicialmente: excesso de responsabilidades concentradas em um único componente.

O supervisor deve permanecer responsável pela coordenação.

O conhecimento especializado continua pertencendo aos agentes de domínio.

Essa separação facilita manutenção, evolução e auditoria.

Quando um novo requisito regulatório surgir para o Clinical Study Report, apenas o agente responsável por esse documento precisará ser atualizado.

O supervisor continuará executando exatamente o mesmo papel de coordenação.

Combinando os três padrões

A principal mensagem deste capítulo é que Sequential, Parallel e Supervisor não representam escolhas mutuamente exclusivas.

Eles podem aparecer simultaneamente dentro da mesma arquitetura.

Considere novamente o Trial Forge.

O supervisor recebe uma nova solicitação para produzir toda a documentação de um estudo clínico.

Inicialmente, identifica quais agentes precisarão participar do processo.

Algumas atividades seguem obrigatoriamente uma sequência lógica.

O protocolo precisa existir antes da geração de determinados documentos derivados.

Nesse trecho da arquitetura, o padrão Sequential é aplicado.

Depois que o protocolo está consolidado, diferentes especialistas podem trabalhar simultaneamente.

Enquanto um agente prepara o Termo de Consentimento, outro organiza partes do relatório clínico e outro executa verificações regulatórias independentes.

Nesse momento entra em funcionamento o Parallel.

Ao final dessas execuções, o supervisor reúne todos os resultados, verifica se existem inconsistências entre eles e encaminha o fluxo para as etapas seguintes.

Observe que os três padrões aparecem naturalmente dentro da mesma solução.

Cada um resolve um tipo diferente de dependência.

Critérios para escolher cada padrão

Durante o desenho de uma arquitetura multiagente costumo responder três perguntas bastante simples.

A primeira é: uma tarefa depende obrigatoriamente da saída produzida por outra?

Se a resposta for positiva, existe um forte indicativo para utilização do Sequential.

A segunda pergunta é: duas tarefas podem ser executadas ao mesmo tempo sem produzir interferência entre si?

Nesse caso, o Parallel normalmente oferece melhor desempenho.

Por fim, pergunto quem será responsável por distribuir trabalho, acompanhar execução e consolidar resultados.

Sempre que essa coordenação existir, aparece naturalmente a necessidade de um Supervisor.

Essas perguntas não escolhem apenas um padrão.

Elas identificam exatamente onde cada estratégia deve aparecer dentro da arquitetura.

Coordenação como responsabilidade arquitetural

Outro aspecto importante consiste em perceber que coordenação não representa um detalhe de implementação.

Ela faz parte da arquitetura.

A forma como os agentes colaboram influencia diretamente latência, custo operacional, capacidade de evolução e facilidade de auditoria.

Uma divisão inadequada pode produzir agentes extremamente competentes trabalhando de maneira ineficiente.

Da mesma forma, uma boa estratégia de coordenação permite que especialistas relativamente simples resolvam problemas bastante complexos quando atuam em conjunto.

Esse é justamente o objetivo das arquiteturas multiagentes.

Não aumentar indefinidamente a inteligência de um único agente, mas organizar especialistas para que cada um execute exatamente aquilo que faz melhor.

Os padrões Sequential, Parallel e Supervisor representam os primeiros blocos de construção utilizados para organizar arquiteturas multiagentes.

O Sequential preserva dependências naturais entre tarefas.

O Parallel reduz tempo de execução ao permitir atividades independentes em simultâneo.

O Supervisor coordena especialistas, distribui responsabilidades e consolida resultados.

Mais importante do que conhecer cada padrão isoladamente é compreender que eles podem coexistir dentro da mesma arquitetura. No Trial Forge, essa combinação permite equilibrar previsibilidade, desempenho e especialização, criando uma base sólida para sistemas corporativos compostos por diversos agentes colaborando de maneira organizada.

Nos próximos capítulos aprofundarei outros padrões de coordenação multiagente e mostrarei como arquiteturas mais sofisticadas resolvem problemas de consenso, colaboração distribuída e tomada de decisão entre especialistas independentes.

Capítulo 03: Arquiteturas Multi-Agent (PT-3)

No capítulo anterior, apresentei os três primeiros padrões de orquestração utilizados em arquiteturas multiagentes: Sequential, Parallel e Supervisor. Esses padrões resolvem problemas relacionados à dependência entre tarefas, execução concorrente e coordenação centralizada. Também mostrei como o Trial Forge combina essas estratégias para organizar a geração de protocolos clínicos, Termos de Consentimento Livre e Esclarecido e Clinical Study Reports dentro de um fluxo único.

Entretanto, esses três padrões não resolvem todos os problemas encontrados em arquiteturas corporativas.

Existem situações nas quais agentes precisam debater antes de chegar a uma decisão. Em outros casos, um agente percebe que a tarefa recebida pertence a outro domínio de conhecimento e precisa transferir completamente a responsabilidade para outro especialista.

Esses cenários exigem padrões diferentes de coordenação.

Neste capítulo, amplio o conjunto de estratégias de orquestração apresentando os padrões **Hierarchical**, **Group Chat** e **Handoff**, completando os seis padrões fundamentais utilizados no framework arquitetural da disciplina.

A evolução das arquiteturas multiagentes

O crescimento das arquiteturas compostas por agentes especializados deixou de representar apenas um tema de pesquisa.

Cada vez mais organizações utilizam agentes distribuídos para resolver problemas complexos, nos quais diferentes domínios de conhecimento precisam colaborar de maneira coordenada.

Esse movimento tornou a escolha do padrão de orquestração uma decisão arquitetural importante.

Não basta decidir quantos agentes existirão.

Também é necessário definir como eles serão organizados, como compartilharão responsabilidades e de que maneira ocorrerá a comunicação entre eles.

Essa decisão influencia diretamente aspectos como escalabilidade, facilidade de manutenção, isolamento de responsabilidades e capacidade de evolução da solução.

O padrão Hierarchical

O primeiro padrão apresentado neste capítulo é o **Hierarchical**.

Ele representa uma evolução natural do padrão Supervisor estudado anteriormente.

Enquanto o Supervisor tradicional concentra toda a coordenação em um único agente central, a arquitetura hierárquica distribui essa responsabilidade em diferentes níveis.

Em vez de um único supervisor responsável por todos os especialistas, passamos a trabalhar com uma árvore de coordenação.

No topo encontra-se um orquestrador principal.

Esse componente não conversa diretamente com todos os agentes especializados.

Sua função consiste em encaminhar cada solicitação para um supervisor de domínio.

Cada supervisor administra exclusivamente os especialistas pertencentes ao seu próprio contexto de atuação.

Essa divisão cria um isolamento bastante importante.

O supervisor responsável por um domínio não precisa conhecer detalhes internos dos demais domínios.

Da mesma forma, o orquestrador principal não necessita compreender como cada supervisor distribui internamente suas tarefas.

Ele precisa apenas reconhecer qual domínio é responsável pela solicitação recebida.

Essa estrutura reduz acoplamento e facilita significativamente a expansão da arquitetura.

Isolamento de responsabilidades

Uma das principais vantagens do padrão Hierarchical está no isolamento do estado.

Cada supervisor trabalha apenas com informações pertencentes ao seu domínio.

Essa característica lembra bastante arquiteturas corporativas baseadas em separação de responsabilidades e controle de acesso.

Cada camada possui uma visão limitada do sistema.

Nenhum componente precisa conhecer detalhes que não sejam relevantes para sua função.

Esse princípio reduz complexidade e melhora aspectos relacionados à segurança e governança.

Também facilita auditorias.

Quando um problema ocorre em determinado domínio, a investigação pode concentrar-se naquele ramo específico da árvore sem necessidade de analisar toda a arquitetura.

Aplicando o Hierarchical ao Trial Forge

No Trial Forge, esse padrão surge naturalmente.

O orquestrador principal recebe solicitações relacionadas à geração de documentos clínicos.

Sua primeira responsabilidade consiste em identificar a natureza da tarefa.

Depois dessa classificação inicial, a solicitação é encaminhada para o supervisor correspondente.

Existe um supervisor responsável pelos protocolos clínicos.

Outro supervisiona os documentos de consentimento.

Um terceiro coordena os agentes relacionados ao Clinical Study Report.

Cada supervisor administra seus próprios especialistas.

Caso um domínio evolua e passe a possuir novos agentes internos, essa mudança permanece completamente transparente para o orquestrador principal.

A única responsabilidade do nível superior continua sendo encaminhar corretamente cada solicitação.

Essa característica permite que a arquitetura cresça sem aumentar proporcionalmente a complexidade do componente central.

O padrão Group Chat

Os próximos padrões resolvem um problema completamente diferente.

Até este ponto, todos os modelos de coordenação estudados assumiam que existia alguma forma clara de autoridade.

Mesmo quando diversos agentes participavam da execução, havia um responsável pela distribuição das tarefas.

O padrão **Group Chat** rompe essa lógica.

Nesse modelo, todos os agentes compartilham uma mesma conversa.

Cada participante pode analisar as informações disponíveis, responder, complementar argumentos e até discordar das conclusões apresentadas pelos demais.

A decisão final deixa de ser consequência de uma delegação hierárquica.

Ela emerge da própria discussão entre os agentes.

Existe normalmente um moderador responsável apenas por organizar a ordem das interações.

Sua função não é decidir o conteúdo da resposta.

Ele apenas controla qual agente participa em cada momento e distribui as mensagens entre todos os envolvidos.

Debate entre especialistas

O principal diferencial do Group Chat está justamente na possibilidade de debate.

Cada agente possui autonomia para defender seu ponto de vista.

Quando diferentes especialistas analisam um mesmo problema sob perspectivas distintas, torna-se possível construir decisões mais robustas do que aquelas produzidas por um único coordenador.

Esse padrão é especialmente útil quando não existe um especialista capaz de reunir sozinho todas as informações necessárias.

Em vez de concentrar a responsabilidade em um supervisor, a arquitetura permite que diferentes conhecimentos sejam confrontados antes da decisão final.

Naturalmente, esse benefício possui um custo.

Cada rodada de discussão representa novas chamadas ao modelo, maior consumo de recursos computacionais e aumento da latência.

Quanto mais agentes participam da conversa, maior tende a ser o tempo necessário para alcançar consenso.

Por isso, Group Chat deve ser reservado para situações nas quais o benefício da colaboração supera claramente o custo adicional de coordenação.

Group Chat e Supervisor não desempenham o mesmo papel

Embora ambos coordenem múltiplos agentes, Group Chat e Supervisor resolvem problemas diferentes.

No Supervisor, os especialistas trabalham isoladamente.

O coordenador recebe os resultados individuais e realiza a consolidação.

Os especialistas não discutem entre si.

No Group Chat acontece exatamente o contrário.

Todos compartilham o mesmo contexto.

Cada nova contribuição pode modificar a direção da conversa.

A decisão final surge da interação coletiva.

O moderador apenas administra a dinâmica da discussão.

Essa distinção ajuda a escolher corretamente o padrão de acordo com a natureza do problema.

Quando a tarefa consiste apenas em agregar resultados independentes, o Supervisor normalmente é suficiente.

Quando diferentes interpretações precisam ser reconciliadas por meio de debate, o Group Chat tende a produzir resultados superiores.

O padrão Handoff

O terceiro padrão deste capítulo é o **Handoff**.

Enquanto o Group Chat trata da colaboração entre especialistas, o Handoff resolve um cenário completamente diferente.

Durante a execução de uma tarefa, um agente pode perceber que o problema ultrapassa seu domínio de conhecimento.

Em vez de continuar produzindo uma resposta para a qual não foi projetado, ele transfere completamente o controle da execução para outro especialista.

Essa transferência representa muito mais do que uma consulta.

O agente original deixa de participar do processo.

O novo agente assume integralmente a responsabilidade pela tarefa.

O contexto acumulado até aquele momento acompanha essa transição.

O novo especialista não reinicia o trabalho.

Ele continua exatamente do ponto onde o processo foi interrompido.

Transferindo responsabilidade

A principal característica do Handoff é a mudança de responsabilidade.

Não existe colaboração simultânea entre dois agentes.

Existe substituição.

Essa diferença é importante porque evita sobreposição de competências.

Cada agente permanece responsável apenas pelo domínio para o qual foi projetado.

Quando identifica uma situação fora desse escopo, transfere imediatamente a tarefa.

Esse comportamento melhora qualidade, reduz riscos e impede que agentes tentem resolver problemas para os quais não possuem conhecimento suficiente.

O Handoff aplicado ao Trial Forge

Um bom exemplo aparece durante a elaboração do Termo de Consentimento Livre e Esclarecido.

Imagine que o agente responsável pelo documento esteja descrevendo os riscos de um estudo clínico.

Durante essa atividade, ele identifica que o protocolo envolve uma terapia gênica experimental relacionada à edição da linha germinativa.

Esse tema ultrapassa significativamente o domínio tradicional de um agente especializado em consentimento.

Em vez de improvisar uma resposta ou utilizar conhecimento limitado sobre bioética, o agente executa um Handoff.

O controle da tarefa é transferido para um agente especializado em bioética.

Esse novo agente recebe todo o contexto acumulado durante a execução.

Ele conhece o protocolo analisado, compreende as decisões tomadas anteriormente e continua a elaboração do documento exatamente do ponto em que o processo foi interrompido.

A transferência não representa perda de informações.

Representa apenas uma mudança consciente de especialidade.

Escolhendo entre os padrões

Depois de conhecer os seis padrões de orquestração, torna-se possível construir um processo de decisão arquitetural mais consistente.

Quando uma tarefa exige apenas dependência entre etapas, o Sequential continua sendo a escolha mais natural.

Quando diferentes atividades podem ocorrer simultaneamente, o Parallel oferece melhor aproveitamento do tempo de execução.

Quando existe necessidade de coordenação centralizada, utiliza-se o Supervisor.

Se a arquitetura cresce e passa a possuir diversos domínios independentes, o Hierarchical permite distribuir responsabilidades em múltiplos níveis.

Quando especialistas precisam discutir para chegar a uma decisão compartilhada, o Group Chat torna-se mais adequado.

Por fim, quando uma tarefa muda de domínio durante sua execução, o Handoff permite transferir completamente o controle para o especialista correto.

O seletor de padrões

Ao longo da evolução do Trial Forge, organizei essas decisões em um artefato de arquitetura utilizado durante o desenho de novas soluções.

Esse seletor de padrões funciona como uma árvore de decisão.

Em vez de perguntar apenas qual padrão utilizar, ele conduz uma sequência de perguntas relacionadas à natureza da tarefa.

Existe dependência obrigatória entre etapas?

As atividades podem ocorrer simultaneamente?

Há necessidade de coordenação centralizada?

Os agentes possuem autoridade equivalente para discordar entre si?

A responsabilidade muda de domínio durante a execução?

Cada resposta conduz naturalmente ao padrão mais adequado para aquela situação.

Esse processo reduz decisões baseadas exclusivamente em percepção individual e transforma a escolha da estratégia de orquestração em uma decisão arquitetural estruturada e repetível.

Capítulo 04: Arquiteturas Multi-Agent (PT-4)

Nos capítulos anteriores, apresentei os principais padrões de coordenação utilizados em arquiteturas multiagentes. Estudamos estratégias como Sequential, Parallel, Supervisor, Hierarchical, Group Chat e Handoff, compreendendo como cada uma delas organiza a colaboração entre agentes especializados dentro de um sistema corporativo.

Entretanto, até esse momento partimos de uma hipótese bastante otimista: a de que todos os agentes executariam suas tarefas corretamente e responderiam dentro do tempo esperado.

Na prática, sistemas distribuídos não funcionam dessa maneira.

Agentes podem ficar indisponíveis, chamadas de rede podem falhar, APIs externas podem ultrapassar o tempo limite de resposta e modelos de linguagem podem demorar muito mais do que o previsto para concluir uma inferência.

Quando isso acontece, a arquitetura precisa estar preparada para decidir como reagir. Não basta apenas distribuir responsabilidades entre especialistas. Também é necessário estabelecer mecanismos de recuperação capazes de preservar a estabilidade do sistema mesmo diante de falhas inevitáveis.

Neste capítulo, apresento dois conceitos fundamentais para lidar com esse cenário: a aplicação do Teorema CAP em arquiteturas multiagentes e o padrão Saga para controle de estado e ações compensatórias.

Falhas fazem parte da arquitetura

Um dos maiores erros ao projetar sistemas distribuídos consiste em tratar falhas como exceções.

Em ambientes reais, falhas não representam acontecimentos extraordinários. Elas fazem parte do comportamento normal da arquitetura.

Cada agente pode estar executando em uma máquina diferente.

Pode utilizar um modelo distinto.

Pode depender de serviços externos sobre os quais não existe qualquer controle.

Consequentemente, sempre existe a possibilidade de que algum componente fique indisponível, responda lentamente ou simplesmente deixe de responder.

O papel da arquitetura não é impedir que essas situações aconteçam.

Seu objetivo é definir antecipadamente qual comportamento o sistema adotará quando elas ocorrerem.

O Teorema CAP

Para compreender esse problema, utilizo um dos conceitos clássicos de sistemas distribuídos: o **Teorema CAP**.

Proposto por Eric Brewer e posteriormente formalizado por Gilbert e Lynch, esse teorema demonstra que, diante de uma partição de comunicação, um sistema distribuído precisa escolher entre dois objetivos concorrentes.

O primeiro deles é a **consistência**.

Nesse cenário, o sistema prefere aguardar até possuir certeza de que está trabalhando com a informação correta.

O segundo é a **disponibilidade**.

Aqui, a prioridade consiste em continuar respondendo, mesmo que parte das informações ainda esteja indisponível.

Quando uma falha de comunicação realmente acontece, não é possível maximizar simultaneamente esses dois objetivos.

Essa observação possui enorme impacto sobre arquiteturas baseadas em agentes.

O dilema dos agentes distribuídos

Imagine que, durante a geração de um Clinical Study Report, o agente responsável pela síntese estatística deixe de responder.

O supervisor passa imediatamente a enfrentar uma decisão arquitetural.

A primeira possibilidade consiste em esperar indefinidamente até que o agente conclua sua tarefa.

Essa abordagem preserva consistência.

Nenhum documento será produzido utilizando informações incompletas.

Entretanto, existe um risco evidente.

Se aquele agente nunca voltar a responder, todo o fluxo permanecerá bloqueado.

A segunda possibilidade consiste em prosseguir com o trabalho disponível naquele momento, registrando explicitamente que determinada parte do relatório ainda não pôde ser concluída.

Nesse caso, a disponibilidade do sistema é preservada.

O processo continua avançando, mas uma lacuna precisará ser resolvida posteriormente.

Observe que essa decisão não pertence ao modelo de linguagem.

Ela faz parte da arquitetura.

Cada componente deve possuir uma política claramente definida para situações de indisponibilidade.

CAP como decisão arquitetural

Existe uma simplificação bastante comum ao apresentar o Teorema CAP.

Muitas vezes afirma-se que um sistema é permanentemente classificado como CP ou AP.

Na realidade, essa interpretação não representa corretamente a aplicação prática do conceito.

A partição de comunicação não é uma escolha.

Falhas de rede acontecem independentemente da vontade da equipe de desenvolvimento.

A decisão arquitetural consiste em definir qual comportamento determinado componente adotará quando essa situação ocorrer.

Isso significa que diferentes partes da mesma arquitetura podem tomar decisões distintas.

Algumas operações críticas podem privilegiar consistência.

Outras podem priorizar disponibilidade para evitar interrupções prolongadas.

A escolha depende do risco associado a cada processo.

Três mecanismos fundamentais

Independentemente da estratégia adotada, existem três mecanismos que considero indispensáveis em qualquer arquitetura multiagente.

O primeiro deles é o **timeout explícito**.

Nenhum agente deve permanecer aguardando indefinidamente pela resposta de outro componente.

Toda comunicação precisa possuir um limite de tempo claramente definido.

Quando esse limite é atingido, a arquitetura toma uma decisão previamente estabelecida.

O segundo mecanismo corresponde ao **retry com limite**.

Uma falha temporária não significa necessariamente que o serviço permanecerá indisponível.

Por isso, faz sentido realizar novas tentativas.

Entretanto, essas tentativas também precisam possuir um limite máximo.

Esse comportamento lembra o controle de iterações utilizado anteriormente no ciclo ReAct.

A diferença é que agora o controle acontece entre agentes distribuídos.

O terceiro mecanismo é a **idempotência**.

Sempre que uma operação for repetida, seu resultado deve permanecer consistente.

Imagine que um timeout ocorra logo após um agente concluir sua tarefa.

Sem confirmação da resposta, o supervisor decide repetir a solicitação.

Se essa operação não for idempotente, o sistema poderá gerar dois documentos distintos para exatamente o mesmo estudo clínico.

A repetição de uma chamada jamais deve produzir efeitos duplicados.

Esse princípio reduz significativamente problemas decorrentes de falhas temporárias de comunicação.

O desafio do controle de estado

Até este ponto analisamos situações nas quais um agente falha durante sua execução.

Existe, entretanto, outro cenário igualmente importante.

Imagine que diversos agentes executaram corretamente suas responsabilidades.

Somente nas etapas finais do fluxo surge uma inconsistência capaz de invalidar parte do trabalho realizado anteriormente.

Nesse caso, simplesmente reiniciar todo o processo representa uma solução extremamente ineficiente.

Grande parte das atividades executadas continua correta.

O problema encontra-se apenas em uma parte específica da arquitetura.

É exatamente esse tipo de situação que motivou o desenvolvimento do padrão Saga.

O padrão Saga

O padrão **Saga**, originalmente proposto para lidar com transações longas em sistemas distribuídos, parte de uma ideia bastante simples.

Em vez de tratar todo o fluxo como uma única transação indivisível, cada etapa passa a representar uma transação independente.

Se alguma atividade posterior identificar um problema, não é necessário reiniciar todo o processo.

Em vez disso, executam-se **ações compensatórias** responsáveis por desfazer apenas os efeitos das etapas afetadas.

Essas ações ocorrem na ordem inversa da execução original.

O aspecto mais importante é compreender que compensação não representa um rollback automático semelhante ao realizado por bancos de dados tradicionais.

Cada etapa precisa possuir sua própria estratégia de desfazer.

Essa estratégia depende diretamente do significado daquela operação.

Compensação versus reinício completo

Considere novamente o Trial Forge.

Imagine que o agente responsável pelo Clinical Study Report descubra, ao final do fluxo, uma inconsistência regulatória proveniente de uma alteração recente no protocolo clínico.

Os agentes responsáveis pelo protocolo e pelo Termo de Consentimento já concluíram corretamente suas atividades.

Reiniciar todo o processo significaria descartar documentos que permanecem válidos apenas por precaução.

Com o padrão Saga, a arquitetura identifica exatamente quais etapas precisam ser revistas.

Somente essas atividades executam suas ações compensatórias.

Todo o restante permanece preservado.

Essa abordagem reduz tempo de processamento, evita desperdício de recursos computacionais e diminui significativamente o custo operacional.

Coordenação e sincronização

Durante o desenho do Trial Forge, dois outros desafios também aparecem: coordenação e sincronização.

A coordenação já foi resolvida pelos padrões estudados anteriormente, especialmente pelo Supervisor.

É esse componente que distribui tarefas, acompanha sua execução e consolida os resultados produzidos pelos especialistas.

A sincronização representa um problema diferente.

Na arquitetura atual, cada agente trabalha sobre documentos distintos.

O agente responsável pelo protocolo não modifica simultaneamente o mesmo arquivo utilizado pelo agente do Clinical Study Report.

Por esse motivo, ainda não existe conflito de escrita.

Entretanto, essa situação pode mudar no futuro.

Imagine dois agentes revisando simultaneamente o mesmo protocolo clínico.

Sem mecanismos de sincronização, a última gravação realizada poderá sobrescrever silenciosamente o trabalho produzido anteriormente.

Controle otimista de versão

Uma estratégia bastante utilizada para evitar esse tipo de conflito é o **controle otimista de versão**.

Cada agente informa qual versão do documento utilizou durante sua leitura.

Quando tenta gravar uma alteração, o sistema verifica se aquela versão continua sendo a mais recente.

Caso outro agente já tenha atualizado o documento nesse intervalo, a gravação é rejeitada.

O responsável pela última alteração precisa então reconciliar seu trabalho antes de tentar novamente.

Esse comportamento é bastante semelhante ao funcionamento de sistemas de controle de versão distribuído.

Em vez de permitir sobrescritas silenciosas, a arquitetura força a resolução explícita de conflitos.

O Distributed Failure Canvas

Para facilitar a análise desses cenários, utilizo um artefato denominado **Distributed Failure Canvas**.

Esse canvas organiza o fluxo completo da arquitetura e solicita que cada etapa seja analisada sob duas perspectivas.

A primeira consiste em identificar o comportamento esperado quando um agente deixa de responder.

Nesse ponto registram-se políticas relacionadas ao Teorema CAP, timeout, retry e idempotência.

A segunda perspectiva concentra-se nas ações compensatórias.

Para cada etapa do fluxo, é necessário responder se existe uma estratégia clara para desfazer seus efeitos caso uma falha tardia invalide o trabalho realizado.

Esse exercício costuma revelar rapidamente quais partes da arquitetura realmente possuem um plano de recuperação e quais simplesmente pressupõem que nenhuma falha ocorrerá.

O Trial Forge sob a perspectiva do Saga

Ao aplicar esse raciocínio ao Trial Forge, torna-se evidente que nem toda falha exige reinício completo do processo.

Imagine que os agentes responsáveis pelo protocolo e pelo Termo de Consentimento concluíram corretamente suas atividades.

Posteriormente, durante a elaboração do Clinical Study Report, uma inconsistência regulatória é identificada.

Nesse cenário, a arquitetura não descarta automaticamente todos os documentos produzidos.

Primeiro identifica exatamente quais artefatos dependem da informação incorreta.

Depois executa apenas as ações compensatórias necessárias para corrigir esses elementos específicos.

Essa abordagem preserva o trabalho válido já realizado e reduz significativamente tanto o custo quanto o tempo necessário para restabelecer a consistência do sistema.

Ao incorporar timeout, retry, idempotência, decisões explícitas relacionadas ao Teorema CAP e mecanismos de compensação baseados em Saga, uma arquitetura multiagente deixa de depender da hipótese de funcionamento perfeito dos seus componentes. Ela passa a assumir que falhas acontecerão inevitavelmente e organiza seu comportamento para continuar operando de forma previsível, auditável e resiliente mesmo diante desses cenários.

Capítulo 05: Arquiteturas Multi-Agent (PT-5)

Nos capítulos anteriores, construímos gradualmente uma arquitetura multiagente completa. Inicialmente, discutimos por que um único agente deixa de ser suficiente quando diferentes documentos, domínios de conhecimento e responsabilidades passam a coexistir dentro do mesmo sistema. Em seguida, estudamos os seis padrões fundamentais de orquestração, os mecanismos de comunicação entre agentes e as estratégias de tolerância a falhas baseadas no Teorema CAP, no padrão Saga e no controle otimista de versão.

Neste capítulo, reúno todos esses elementos em um único protótipo funcional. O objetivo não é apresentar novos conceitos, mas demonstrar como todas as decisões arquiteturais estudadas anteriormente se integram em uma implementação coerente, documentada e preparada para operar em cenários reais.

Integrando todas as decisões arquiteturais

Um protótipo multiagente bem construído não surge pela simples combinação de vários agentes especializados. O verdadeiro diferencial está na disciplina utilizada para justificar cada decisão da arquitetura.

Cada escolha precisa possuir um motivo claro.

Por que determinado agente utiliza o padrão Sequential e não Parallel?

Por que um timeout específico foi adotado para uma determinada tarefa?

Por que uma etapa exige compensação explícita enquanto outra não necessita desse mecanismo?

Essas respostas precisam existir antes mesmo da implementação. Quando a arquitetura é documentada dessa forma, o código deixa de representar apenas uma sequência de instruções e passa a refletir decisões técnicas previamente analisadas.

Consolidando a arquitetura do Trial Forge

No Trial Forge, a arquitetura completa pode ser representada por uma tabela de calibração que reúne todos os agentes participantes do sistema e associa a cada um deles os padrões de orquestração, estratégias de recuperação e mecanismos de compensação definidos ao longo do módulo.

O agente responsável pelo protocolo ocupa naturalmente o início da cadeia de execução.

Seu funcionamento segue o padrão Sequential, pois todos os demais agentes dependem das informações produzidas nessa etapa.

Como essa tarefa é relativamente rápida e representa a base para todo o restante do fluxo, foi definido um timeout de trinta segundos, acompanhado por até três tentativas de execução.

Caso alguma inconsistência seja identificada posteriormente, a estratégia de compensação não consiste em eliminar o protocolo existente, mas criar uma nova versão preservando completamente o histórico anterior.

Essa decisão garante rastreabilidade e evita perda de informações importantes durante auditorias.

O agente responsável pelo Termo de Consentimento Livre e Esclarecido opera em paralelo com o agente responsável pelo Clinical Study Report.

Além disso, esse agente representa um ponto potencial para utilização do padrão Handoff.

Sempre que identificar uma situação que ultrapasse seu domínio de conhecimento, como uma questão complexa de bioética ou um tema altamente especializado, a responsabilidade poderá ser transferida para outro agente mais adequado.

Como sua tarefa envolve geração de linguagem natural e inclusão de seções condicionais, o tempo de execução tende a ser maior do que o do protocolo.

Por esse motivo, o timeout definido é de quarenta e cinco segundos.

Caso alguma alteração posterior afete apenas parte do documento, a compensação limita-se à regeneração da seção correspondente, preservando todo o restante do conteúdo.

O agente responsável pelo Clinical Study Report também trabalha em paralelo.

Entre todos os especialistas do sistema, ele executa uma das atividades mais pesadas, pois precisa consolidar resultados estatísticos e garantir conformidade com padrões regulatórios internacionais.

Por essa razão, recebe o timeout mais amplo da arquitetura, definido em sessenta segundos.

Além disso, costuma ser esse agente que identifica inconsistências capazes de exigir compensações em etapas anteriores do fluxo, desencadeando revisões no protocolo ou em documentos já produzidos.

No topo dessa estrutura encontra-se o agente supervisor.

Seu papel continua sendo exclusivamente coordenar.

Ele não produz conteúdo, não executa análises regulatórias nem participa da geração dos documentos.

Sua responsabilidade consiste em acompanhar a execução dos especialistas, consolidar resultados e decidir quais mecanismos de compensação devem ser acionados quando algum problema é identificado.

Timeouts devem refletir a realidade

Existe um erro bastante comum durante o desenvolvimento de arquiteturas multiagentes.

Muitas equipes definem exatamente o mesmo timeout para todos os agentes simplesmente para simplificar a implementação.

Embora pareça uma solução prática, essa decisão normalmente produz efeitos indesejados.

Cada agente executa uma tarefa diferente.

Algumas atividades terminam rapidamente.

Outras exigem consultas extensas, recuperação de contexto ou processamento significativamente mais pesado.

Consequentemente, o tempo esperado para cada execução também é diferente.

Quando todos utilizam o mesmo limite, agentes naturalmente mais lentos passam a produzir falsos alarmes de indisponibilidade.

Por outro lado, tarefas muito rápidas podem permanecer tempo demais aguardando antes que uma falha seja identificada.

A definição do timeout deve refletir o comportamento real de cada responsabilidade arquitetural, sempre acompanhada por uma margem de segurança coerente com o contexto operacional.

Documentando contratos de comunicação

Antes da implementação propriamente dita, existe outro artefato importante: a documentação dos eventos trocados entre agentes.

Para isso utilizo um canvas específico destinado à definição das mensagens da arquitetura.

Cada transição do fluxo precisa declarar explicitamente três elementos.

Primeiro, o nome do evento.

Depois, quais informações esse evento transporta.

Por fim, quais agentes são responsáveis por reagir àquela mensagem.

Essa documentação pode parecer burocrática em um primeiro momento.

Entretanto, ela elimina um dos problemas mais frequentes em sistemas distribuídos: agentes diferentes assumindo formatos incompatíveis para representar exatamente a mesma informação.

Ao documentar previamente os contratos de comunicação, todos os participantes passam a compartilhar a mesma expectativa sobre o conteúdo transportado por cada evento.

Comunicação assíncrona entre agentes

A implementação do protótipo utiliza comunicação assíncrona baseada em eventos.

O princípio é bastante simples.

Quando um agente conclui sua tarefa, publica um evento informando que determinada etapa foi finalizada.

Esse agente não permanece aguardando a reação dos demais componentes.

Sua responsabilidade termina no momento em que a mensagem é publicada.

Os demais agentes permanecem inscritos nesse evento.

Assim que ele acontece, cada especialista inicia sua própria execução.

Essa abordagem elimina chamadas bloqueantes desnecessárias e reduz o acoplamento entre os componentes da arquitetura.

Do ponto de vista conceitual, esse mecanismo permanece válido tanto para implementações simples quanto para ambientes corporativos de grande escala.

A principal diferença encontra-se nas garantias oferecidas pela infraestrutura responsável pelo transporte das mensagens.

Barramento de eventos

No protótipo apresentado durante a aula, a comunicação utiliza um barramento de eventos simples.

Essa escolha atende plenamente ao objetivo didático da implementação.

Entretanto, esse tipo de mecanismo não preserva mensagens caso o processo seja interrompido durante sua execução.

Em ambientes corporativos, normalmente utilizamos plataformas especializadas capazes de registrar os eventos antes mesmo de confirmar sua entrega.

Esse comportamento impede que mensagens importantes sejam perdidas durante falhas inesperadas da infraestrutura.

Outro aspecto importante diz respeito ao isolamento dos fluxos.

No protótipo, cada estudo clínico possui seu próprio barramento de eventos.

Em ambientes reais, diversos estudos compartilham simultaneamente a mesma infraestrutura de mensageria.

Nessas situações, cada mensagem precisa transportar um identificador de correlação que permita distinguir claramente os eventos pertencentes a cada execução.

Esse identificador deixa de ser apenas um detalhe de implementação.

Ele passa a representar um requisito essencial para o roteamento correto das mensagens.

Eventos ricos

Outro aspecto discutido durante a implementação refere-se ao conteúdo transportado pelos eventos.

Uma abordagem bastante comum consiste em publicar apenas uma notificação indicando que determinada tarefa terminou.

Os agentes interessados precisam então consultar outra fonte para recuperar os dados necessários.

No protótipo do Trial Forge, adota-se uma estratégia diferente.

Cada evento transporta diretamente as informações relevantes produzidas pelo agente anterior.

Esse modelo, conhecido como evento rico, reduz a necessidade de consultas adicionais e simplifica parte da comunicação.

Entretanto, também aumenta o acoplamento entre o formato do evento e a estrutura dos dados transportados.

Como qualquer decisão arquitetural, trata-se de um trade-off que precisa ser documentado explicitamente.

Comunicação não resolve falhas

A comunicação assíncrona resolve o problema do desacoplamento entre agentes.

Entretanto, ela não elimina automaticamente problemas relacionados a falhas de execução.

Imagine que dois agentes estejam processando simultaneamente uma mesma solicitação.

Se um deles concluir sua tarefa normalmente enquanto o outro lança uma exceção durante a execução, o supervisor precisa ser capaz de distinguir exatamente qual agente falhou.

Caso contrário, todo o trabalho realizado pelo componente que terminou corretamente poderá ser descartado desnecessariamente.

Esse cenário demonstra que comunicação e tolerância a falhas representam preocupações arquiteturais distintas.

A primeira organiza o fluxo das mensagens.

A segunda determina como a arquitetura reage quando parte desse fluxo deixa de funcionar conforme o esperado.

Falhas parciais

Uma arquitetura madura deve tratar falhas parciais de maneira explícita.

Quando apenas um agente apresenta problemas, não faz sentido descartar automaticamente o resultado produzido pelos demais especialistas.

O supervisor precisa receber o estado individual de cada execução para decidir quais componentes realmente necessitam de uma nova tentativa.

Essa estratégia reduz desperdício de processamento e evita repetir tarefas que já foram concluídas corretamente.

Além disso, torna possível aplicar mecanismos de retry apenas aos agentes efetivamente afetados pela falha.

Retry e idempotência

A repetição de uma operação somente é segura quando a arquitetura garante idempotência.

Considere o agente responsável pelo Termo de Consentimento.

É possível que o documento tenha sido produzido corretamente, mas que a confirmação da conclusão tenha sido perdida durante a comunicação.

Se a arquitetura simplesmente repetir toda a operação sem qualquer controle, poderão surgir documentos duplicados para o mesmo estudo.

Por esse motivo, cada nova tentativa precisa produzir exatamente o mesmo estado final esperado.

O retry não pode criar efeitos colaterais adicionais.

Essa relação entre retry e idempotência reforça diretamente os princípios apresentados anteriormente durante a discussão sobre o Teorema CAP.

Timeout supervisionado

Outra situação importante ocorre quando um agente não produz qualquer resposta dentro do tempo esperado.

Nessa circunstância, a arquitetura não depende de uma notificação enviada pelo próprio agente.

O supervisor acompanha externamente o tempo de execução de cada tarefa.

Caso o limite previamente definido seja ultrapassado, a falha é detectada automaticamente.

A partir desse momento entram em funcionamento as políticas de retry e os limites máximos de tentativas estabelecidos durante a calibração da arquitetura.

Se a indisponibilidade persistir após todas as tentativas previstas, o sistema aplica a estratégia escolhida anteriormente entre consistência e disponibilidade, permitindo que o fluxo prossiga ou interrompendo sua execução conforme o nível de criticidade daquela atividade.

Comunicação, recuperação e consistência

Ao reunir mensageria assíncrona, contratos explícitos de eventos, timeouts individualizados, retries controlados, idempotência e mecanismos de compensação, a arquitetura multiagente deixa de depender do chamado caminho feliz, no qual todos os componentes funcionam perfeitamente.

O sistema passa a considerar falhas como parte natural de sua operação.

Cada agente permanece responsável apenas pelo seu domínio de conhecimento, enquanto o supervisor coordena a execução e aplica as estratégias de recuperação previamente definidas.

É exatamente essa integração disciplinada entre padrões de orquestração, comunicação e tolerância a falhas que transforma um conjunto de agentes independentes em uma plataforma corporativa capaz de operar de maneira previsível, auditável e resiliente.

Módulo 04: Padrões de Design AI-Específicos

Capítulo 01: Padrões de Design AI-Específicos (PT-1)

Ao longo dos módulos anteriores, acompanhei a evolução do Trial Forge desde uma arquitetura AI-First composta por um único agente até uma plataforma formada por diversos agentes especializados, coordenados por diferentes padrões de orquestração e preparados para lidar com falhas, sincronização de estado e recuperação distribuída.

Entretanto, existe um componente que esteve presente praticamente em todos esses exemplos e que, até agora, apareceu apenas como um detalhe da arquitetura: a busca por informações.

Sempre que um agente precisava recuperar uma cláusula regulatória, consultar documentação técnica ou enriquecer seu contexto antes de gerar uma resposta, utilizávamos uma estratégia baseada em **Retrieval Augmented Generation**, normalmente chamada apenas de **RAG**.

Durante muito tempo tratamos o RAG como uma única técnica, quase como uma caixa-preta capaz de resolver qualquer necessidade de recuperação de informação.

Neste módulo, abandono essa visão simplificada.

O objetivo passa a ser compreender que existem diferentes padrões de RAG, cada um projetado para resolver limitações específicas dos mecanismos tradicionais de busca. No contexto do Trial Forge, veremos que uma plataforma corporativa madura não depende de apenas um padrão. Ela combina diferentes estratégias conforme o tipo de documento, a natureza da consulta e os requisitos de precisão exigidos pelo processo.

Ao longo deste estudo trabalharei com quatro padrões principais: **Basic RAG**, **Hybrid Search**, **Multi-Index** e **Agentic RAG**.

Cada um deles representa uma evolução arquitetural sobre o anterior.

A busca como componente arquitetural

Quando pensamos em Inteligência Artificial Generativa, existe uma tendência natural de concentrar toda a atenção no modelo de linguagem.

Entretanto, na prática, o modelo raramente possui sozinho todas as informações necessárias para responder corretamente às solicitações recebidas.

Grande parte do sucesso de um agente depende justamente da qualidade das informações recuperadas antes da geração da resposta.

Em outras palavras, um excelente modelo trabalhando sobre um contexto inadequado continuará produzindo respostas inadequadas.

Por esse motivo, a busca deixa de representar apenas uma etapa auxiliar do fluxo e passa a constituir um componente arquitetural com responsabilidades próprias.

Ela influencia diretamente precisão, custo, latência e até mesmo a confiabilidade de todo o sistema.

O problema da busca ingênua

Nos primeiros módulos do Trial Forge, sempre que o agente precisava recuperar uma cláusula regulatória, assumíamos que existia um mecanismo capaz de localizar automaticamente o trecho mais relevante.

Essa simplificação foi suficiente para compreender a arquitetura geral do sistema.

Entretanto, uma implementação corporativa precisa responder perguntas mais específicas.

Como os documentos são organizados?

Qual estratégia será utilizada para localizar o conteúdo?

Como decidir quais trechos realmente serão enviados ao modelo?

Como evitar recuperar informações irrelevantes?

Responder a essas perguntas exige abrir a chamada "caixa-preta" do RAG.

É justamente essa análise que inicia este módulo.

O funcionamento do Basic RAG

O primeiro padrão estudado é o **Basic RAG**.

Apesar do nome, ele continua sendo extremamente importante e permanece presente em grande parte das aplicações corporativas.

Seu funcionamento pode ser dividido em quatro etapas principais.

Tudo começa com uma base de documentos.

Esses documentos não são enviados integralmente ao modelo de linguagem.

Antes disso, passam por um processo de preparação.

Essa preparação consiste na divisão do conteúdo em pequenos fragmentos, normalmente chamados de **chunks**.

Essa etapa é necessária porque modelos de linguagem possuem limites para a quantidade de contexto processado em cada interação.

Além disso, trabalhar com documentos inteiros reduziria significativamente a eficiência da busca.

Depois da fragmentação, cada trecho é convertido em uma representação vetorial.

Esse processo transforma texto em uma estrutura numérica capaz de representar seu significado semântico.

Esses vetores são então armazenados em uma base especializada para consultas posteriores.

Quando o usuário realiza uma pergunta, acontece exatamente o mesmo processo.

A consulta também é convertida para uma representação vetorial.

Em seguida, o sistema compara esse vetor com todos os vetores armazenados anteriormente, procurando aqueles que apresentam maior proximidade semântica.

Os fragmentos considerados mais relevantes são recuperados e enviados juntamente com a pergunta original para o modelo de linguagem.

Somente nesse momento ocorre a geração da resposta.

Observe que o modelo não consulta diretamente toda a base documental.

Ele trabalha apenas sobre um subconjunto previamente selecionado pelo mecanismo de recuperação.

A anatomia do pipeline

Ao analisar esse fluxo, percebemos que o Basic RAG é composto por componentes claramente separados.

Existe uma etapa responsável pela preparação dos documentos.

Outra dedicada ao armazenamento vetorial.

Uma terceira realiza a recuperação das informações.

Por fim, o modelo de linguagem utiliza esse contexto enriquecido para produzir a resposta.

Essa separação é importante porque cada etapa pode evoluir independentemente.

Podemos modificar a estratégia de fragmentação sem alterar o modelo.

Podemos substituir a base vetorial sem modificar os documentos.

Também podemos alterar a forma de recuperação mantendo exatamente o mesmo processo de geração.

Essa modularidade transforma o RAG em um verdadeiro pipeline arquitetural, e não apenas em uma funcionalidade isolada.

A importância da fragmentação

Entre todas essas etapas, uma das mais críticas é justamente a fragmentação dos documentos.

Existe uma tendência inicial de imaginar que basta dividir o texto em blocos de tamanho fixo.

Na prática, essa abordagem produz resultados bastante limitados.

Quando um documento é fragmentado sem considerar sua estrutura lógica, informações diretamente relacionadas podem acabar separadas em partes diferentes.

Como consequência, o mecanismo de recuperação poderá localizar apenas um trecho parcial da informação necessária.

Esse problema torna-se ainda mais evidente em documentos regulatórios, nos quais definições, exceções e observações frequentemente aparecem distribuídas entre seções consecutivas.

Por esse motivo, uma boa estratégia de fragmentação procura preservar unidades naturais de significado, mantendo contexto suficiente para que cada fragmento permaneça compreensível mesmo quando recuperado isoladamente.

O papel dos embeddings

Outro componente essencial do Basic RAG é a geração dos embeddings.

São eles que permitem realizar buscas baseadas em significado e não apenas em correspondência literal de palavras.

Esse comportamento representa uma das maiores diferenças em relação aos mecanismos tradicionais de pesquisa.

Imagine que um usuário procure informações sobre "consentimento para menores de idade".

Mesmo que determinado documento utilize exclusivamente a expressão "assentimento", o mecanismo vetorial poderá identificar a proximidade semântica entre esses conceitos e recuperar corretamente o trecho relevante.

Essa capacidade amplia significativamente a qualidade das buscas em ambientes compostos por documentos extensos e linguagem especializada.

Recuperação antes da geração

Uma característica importante desse padrão é a separação entre recuperação e geração.

Primeiro o sistema procura informações.

Somente depois o modelo de linguagem inicia a produção da resposta.

Essa sequência reduz significativamente o risco de respostas produzidas apenas com base no conhecimento previamente aprendido pelo modelo.

Em vez de depender exclusivamente de sua memória paramétrica, o agente passa a fundamentar sua resposta utilizando informações efetivamente existentes na base documental da organização.

Essa estratégia melhora rastreabilidade e permite trabalhar com conteúdos específicos que não fazem parte do treinamento original do modelo.

Aplicando o Basic RAG ao Trial Forge

No Trial Forge, o Basic RAG aparece inicialmente na recuperação de cláusulas regulatórias.

As resoluções da Anvisa e os documentos utilizados como referência são previamente preparados, fragmentados e indexados.

Quando o agente responsável pelo Termo de Consentimento precisa gerar uma seção específica, sua primeira ação consiste em consultar essa base.

A pergunta é transformada em embedding.

Os fragmentos semanticamente mais próximos são recuperados.

Esses trechos passam a compor o contexto enviado ao modelo.

O resultado final deixa de depender apenas do conhecimento interno da Inteligência Artificial.

Ele passa a incorporar diretamente o conteúdo oficial das regulamentações utilizadas pela organização.

As limitações do Basic RAG

Embora extremamente eficiente para diversos cenários, o Basic RAG apresenta limitações importantes.

Seu principal desafio surge quando diferentes formas de busca precisam coexistir.

Nem toda consulta depende apenas de similaridade semântica.

Em muitos casos, palavras específicas, códigos regulatórios, números de resolução, identificadores de documentos ou nomes próprios possuem importância decisiva.

Uma busca baseada exclusivamente em embeddings pode deixar de recuperar resultados importantes quando determinados termos precisam corresponder exatamente ao texto pesquisado.

Essa limitação torna-se especialmente evidente em ambientes regulatórios, nos quais uma única referência normativa pode alterar completamente a interpretação de um documento.

É justamente para resolver esse problema que surgem padrões mais avançados de recuperação, começando pelo Hybrid Search, que combina diferentes estratégias de busca para ampliar simultaneamente precisão e cobertura dos resultados.

Capítulo 02: Padrões de Design AI-Específicos (PT-2)

No capítulo anterior, apresentei os quatro principais padrões de Retrieval Augmented Generation utilizados ao longo da arquitetura do Trial Forge. Vimos que Basic RAG, Hybrid Search, Multi-Index e Agentic RAG representam diferentes estratégias para recuperar informações antes que o modelo de linguagem produza sua resposta. Também discutimos como cada padrão atende necessidades específicas relacionadas à precisão, organização da informação e capacidade de recuperação de contexto.

Entretanto, mesmo quando a arquitetura já possui múltiplos índices e mecanismos sofisticados de busca, ainda existe uma decisão importante que precisa ser tomada antes de qualquer recuperação acontecer.

Qual modelo será utilizado para processar a requisição?

E, antes mesmo disso, para qual fluxo aquela requisição deverá ser encaminhada?

Essas duas perguntas parecem semelhantes, mas resolvem problemas completamente diferentes. Neste capítulo, apresento dois padrões fundamentais para responder essas questões: **Model Router** e **Intent-Based Routing**.

O problema do roteamento

Toda arquitetura baseada em Inteligência Artificial que chega ao ambiente de produção enfrenta um desafio inevitável.

Com o amadurecimento do ecossistema de modelos generativos, torna-se comum que uma organização tenha acesso simultâneo a modelos com capacidades, velocidades e custos bastante diferentes.

Alguns são rápidos e baratos, ideais para tarefas simples.

Outros possuem maior capacidade de raciocínio, porém apresentam custo significativamente superior.

Diante desse cenário, existe uma tendência bastante comum: utilizar sempre o modelo mais poderoso disponível.

À primeira vista, essa parece ser a alternativa mais segura.

Se o modelo mais avançado consegue resolver qualquer tarefa, por que não utilizá-lo para tudo?

Na prática, essa decisão representa exatamente o tipo de escolha por omissão discutido anteriormente durante o estudo dos trade-offs arquiteturais.

Não houve uma decisão consciente.

A equipe simplesmente nunca definiu uma estratégia diferente.

Como consequência, o custo cresce proporcionalmente ao volume de chamadas, independentemente da complexidade real de cada tarefa.

Em sistemas corporativos executando milhares de requisições diariamente, pequenas diferenças de custo por chamada transformam-se rapidamente em valores relevantes no orçamento operacional.

É justamente para resolver esse problema que surge o padrão Model Router.

Model Router

O **Model Router** tem uma responsabilidade bastante específica.

Sua função consiste em decidir qual modelo deve processar determinada tarefa.

Essa decisão acontece antes da inferência principal.

Em vez de encaminhar automaticamente toda solicitação para o modelo mais caro, a arquitetura realiza uma classificação prévia da complexidade da tarefa.

Quando a solicitação é simples, o roteador direciona a execução para um modelo menor, mais rápido e menos custoso.

Quando identifica necessidade de raciocínio mais elaborado, escolhe um modelo mais sofisticado.

Observe que o roteador não responde à pergunta do usuário.

Ele apenas decide qual modelo possui o melhor equilíbrio entre custo, desempenho e qualidade para aquela situação.

Essa decisão acontece continuamente.

Não se trata de uma configuração realizada apenas uma vez durante a implantação da arquitetura.

Cada nova requisição percorre novamente esse processo de classificação.

O custo da decisão por omissão

Uma arquitetura que utiliza permanentemente o modelo mais sofisticado desperdiça recursos em tarefas que poderiam ser resolvidas por modelos significativamente menores.

Grande parte das solicitações recebidas por sistemas corporativos envolve classificações simples, consultas diretas ou operações de baixa complexidade.

Reservar o modelo mais caro apenas para situações que realmente exigem maior capacidade de raciocínio representa uma otimização arquitetural importante.

Esse comportamento também reduz latência.

Modelos menores normalmente produzem respostas mais rapidamente.

Consequentemente, além da redução financeira, melhora-se também a experiência do usuário em tarefas simples.

Essa lógica conecta-se diretamente ao framework de trade-offs apresentado anteriormente.

A decisão continua baseada no equilíbrio entre custo, precisão, latência e desempenho.

A diferença é que agora esse equilíbrio acontece automaticamente durante cada nova requisição.

Evidências práticas

Esse padrão deixou de ser apenas uma proposta teórica.

Diversos trabalhos recentes demonstram que o roteamento inteligente permite reduzir significativamente custos sem comprometer a qualidade das respostas.

Um dos exemplos apresentados durante a aula é o **RouteLLM**, framework desenvolvido por pesquisadores da Universidade da Califórnia em Berkeley em parceria com a Anyscale e a Canva.

O objetivo do projeto consiste justamente em selecionar dinamicamente qual modelo deve atender cada solicitação.

Os experimentos mostram que o sistema consegue preservar aproximadamente noventa e cinco por cento da qualidade obtida pelo modelo mais sofisticado, acionando esse modelo apenas em uma parcela das requisições.

O resultado é uma redução expressiva no custo operacional sem perda significativa de desempenho percebido.

Independentemente dos números específicos obtidos durante o benchmark, o aspecto mais importante encontra-se na estratégia arquitetural.

O modelo mais caro deixa de representar a escolha padrão e passa a ser utilizado apenas quando realmente necessário.

O roteamento em escala

Outro exemplo discutido durante a aula refere-se ao próprio GPT-5.

Segundo o material apresentado, a arquitetura do sistema deixou de ser composta por um único modelo e passou a utilizar diferentes modelos especializados em conjunto.

Um modelo mais rápido responde à maior parte das solicitações.

Outro, orientado para raciocínio aprofundado, permanece reservado para problemas mais complexos.

Entre ambos existe exatamente um roteador responsável por decidir qual deles será acionado em cada conversa.

Essa decisão considera diferentes fatores, como o tipo da solicitação, sua complexidade, a necessidade de utilização de ferramentas externas e até mesmo preferências explicitamente informadas pelo usuário.

Outro aspecto importante mencionado durante a aula é que esse classificador não permanece estático.

Ele é continuamente recalibrado utilizando sinais obtidos durante o uso real da plataforma.

Essa observação reforça uma característica importante dos sistemas AI-First.

O roteador também representa um componente sujeito à evolução contínua.

Quando o roteamento se torna um problema

Apesar dos benefícios, o roteamento também introduz riscos.

Um classificador inadequado pode encaminhar tarefas complexas para modelos simplificados.

Nesse caso, o custo diminui, mas a qualidade percebida pelo usuário também.

Esse tipo de problema demonstra que o roteador não representa apenas um mecanismo técnico de otimização financeira.

Ele influencia diretamente a experiência de utilização do sistema.

Uma decisão incorreta tomada antes da inferência pode comprometer completamente o resultado produzido, mesmo que o modelo selecionado funcione exatamente como esperado.

Por isso, calibrar corretamente o roteador torna-se tão importante quanto escolher os próprios modelos utilizados pela arquitetura.

Como construir um Model Router

Existe uma percepção equivocada de que um roteador inteligente precisa necessariamente utilizar outro modelo sofisticado.

Na prática, isso não é obrigatório.

Uma implementação simples pode utilizar regras determinísticas bastante objetivas.

Características como tamanho da entrada, tipo da operação solicitada ou presença de determinadas palavras-chave já permitem separar uma parcela significativa das tarefas.

Uma segunda possibilidade consiste em utilizar um modelo pequeno e barato para classificar a solicitação.

Esse modelo compara a requisição atual com exemplos previamente conhecidos e determina qual nível de complexidade ela apresenta.

Em arquiteturas mais sofisticadas, o roteador pode ser treinado especificamente para prever qual modelo conseguirá resolver determinada tarefa sem perda perceptível de qualidade.

O aspecto importante é compreender que não existe obrigação de começar pela estratégia mais avançada.

Em muitos projetos, regras simples já capturam boa parte do benefício esperado.

Reutilizando o framework de trade-offs

Durante o módulo inicial, apresentei um framework baseado em custo, latência, precisão e desempenho.

Esse mesmo raciocínio continua válido aqui.

Quando o custo de um erro é elevado ou suas consequências são irreversíveis, o roteador tende naturalmente a favorecer modelos mais sofisticados.

Por outro lado, tarefas de baixo risco podem ser encaminhadas para modelos menores sem comprometer significativamente a operação do sistema.

Essa decisão não depende apenas da dificuldade da pergunta.

Também considera o impacto que um eventual erro produziria sobre o negócio.

Dessa forma, o limiar utilizado pelo roteador não deve ser uniforme para toda a arquitetura.

Cada categoria de tarefa pode possuir critérios próprios de classificação.

O classificador não precisa ser perfeito

Existe outro aspecto importante discutido durante a aula.

O objetivo do roteador não consiste em classificar corretamente absolutamente todas as requisições.

Sua função principal é evitar o uso desnecessário do modelo mais caro.

Consequentemente, o erro mais perigoso não é encaminhar ocasionalmente uma tarefa simples para um modelo mais sofisticado.

O verdadeiro risco consiste em enviar uma tarefa complexa para um modelo incapaz de resolvê-la adequadamente.

Por esse motivo, o limiar de confiança utilizado pelo classificador normalmente é calibrado de maneira conservadora.

Sempre que existir dúvida relevante, a arquitetura tende a privilegiar qualidade em vez de economia.

Essa estratégia reduz significativamente a probabilidade de degradação perceptível da experiência do usuário.

Intent-Based Routing

Até este ponto, todas as decisões analisadas concentraram-se na escolha do modelo.

Entretanto, existe uma decisão que acontece ainda antes dessa etapa.

Nem toda requisição precisa apenas escolher um modelo.

Primeiro é necessário compreender qual é a intenção da solicitação.

Esse problema é resolvido pelo padrão **Intent-Based Routing**.

Enquanto o Model Router responde à pergunta "qual modelo deve executar esta tarefa?", o Intent-Based Routing responde outra questão completamente diferente:

Para onde esta requisição deve seguir?

Essa distinção é fundamental.

Uma solicitação pode representar uma geração de documento, uma consulta regulatória, uma classificação, uma busca em base de conhecimento ou uma operação administrativa.

Cada uma dessas intenções percorre um fluxo arquitetural diferente.

O roteamento por intenção identifica esse objetivo antes mesmo que qualquer modelo seja selecionado.

Direção antes da execução

No Trial Forge, essa etapa acontece logo após a entrada da requisição.

O sistema identifica se o usuário pretende elaborar um protocolo clínico, gerar um Termo de Consentimento Livre e Esclarecido, produzir um Clinical Study Report ou executar qualquer outro fluxo disponível.

Somente depois dessa classificação inicial a solicitação alcança o agente especializado correspondente.

A partir desse ponto, entra em funcionamento o Model Router para decidir qual modelo deverá atender aquela tarefa específica.

Essa sequência mostra claramente que os dois padrões não competem entre si.

Eles atuam em momentos diferentes da arquitetura.

Primeiro identifica-se a intenção.

Depois escolhe-se o modelo mais adequado para executá-la.

Dois roteadores, duas responsabilidades

Embora utilizem o termo "roteamento", esses padrões possuem objetivos completamente distintos.

O Intent-Based Routing resolve direção.

Seu papel consiste em encaminhar corretamente cada solicitação para o fluxo apropriado.

O Model Router resolve custo e qualidade.

Sua responsabilidade é selecionar o modelo mais adequado para aquela execução específica.

Uma arquitetura AI-First madura normalmente utiliza ambos em sequência.

Primeiro identifica o destino correto da requisição.

Depois otimiza a execução escolhendo dinamicamente o modelo mais apropriado para aquela tarefa.

Essa separação mantém a arquitetura modular e permite evoluir cada mecanismo independentemente conforme surgem novos modelos, novos agentes e novos fluxos de negócio.

Capítulo 03: Padrões de Design AI-Específicos (PT-3)

No capítulo anterior, apresentei dois padrões fundamentais de roteamento utilizados em arquiteturas AI-First. O **Intent-Based Routing** permite identificar qual fluxo de negócio deve ser executado, enquanto o **Model Router** seleciona dinamicamente o modelo mais adequado para cada tarefa, equilibrando custo, desempenho e qualidade.

Esses dois mecanismos garantem que a requisição correta seja encaminhada para o fluxo correto e processada pelo modelo mais apropriado.

Entretanto, ainda existe um desperdício importante que nenhum deles resolve.

Mesmo quando uma solicitação chega ao agente correto e utiliza o modelo ideal, ainda é possível que exatamente a mesma informação já tenha sido processada anteriormente. Em sistemas corporativos, esse cenário é muito mais comum do que parece.

No Trial Forge, diversos pesquisadores, monitores clínicos e equipes diferentes consultam repetidamente o mesmo protocolo de estudo utilizando perguntas formuladas de maneiras distintas. Processar novamente essas solicitações significa consumir recursos desnecessariamente.

É justamente esse problema que motiva os padrões apresentados neste capítulo: **Semantic Cache**, **Prompt Cache** e **Response Streaming**.

Embora normalmente sejam estudados em conjunto, eles resolvem problemas completamente diferentes e atuam em momentos distintos da arquitetura.

O desperdício invisível

Uma arquitetura eficiente não deve apenas responder corretamente.

Ela também precisa evitar trabalho desnecessário.

Grande parte das perguntas realizadas em um sistema corporativo não representa conhecimento novo.

São consultas repetidas, apenas formuladas com palavras diferentes.

Perguntas como "Quais são os critérios de inclusão do estudo?" e "Quem pode participar desta pesquisa?" possuem praticamente a mesma intenção.

Entretanto, um mecanismo tradicional de cache baseado apenas em igualdade textual não consegue perceber essa equivalência.

Como consequência, a arquitetura executa novamente toda a inferência, consome novos tokens, consulta novamente a base de conhecimento e produz uma resposta praticamente idêntica.

Esse comportamento representa desperdício de processamento.

Semantic Cache

O primeiro padrão apresentado é o **Semantic Cache**.

Seu objetivo consiste em identificar perguntas semanticamente equivalentes antes que qualquer chamada ao modelo seja realizada.

O funcionamento segue um princípio bastante semelhante ao utilizado em arquiteturas RAG.

A pergunta recebida é transformada em um embedding.

Esse vetor é comparado com embeddings correspondentes a perguntas respondidas anteriormente.

Caso a similaridade ultrapasse um limiar previamente definido, o sistema devolve diretamente a resposta armazenada no cache.

Nenhuma nova chamada ao modelo acontece.

Observe que o ganho ocorre antes mesmo da inferência.

Todo o processamento é eliminado.

Essa característica diferencia completamente o Semantic Cache de outras formas tradicionais de otimização.

Comparando significado, não texto

O aspecto mais importante desse padrão está justamente na comparação baseada em significado.

Não interessa se duas perguntas utilizam exatamente as mesmas palavras.

O que importa é verificar se ambas representam essencialmente a mesma solicitação.

Essa abordagem permite reutilizar respostas mesmo quando diferentes usuários formulam perguntas utilizando vocabulários distintos.

Em aplicações corporativas, nas quais centenas de pessoas consultam continuamente a mesma base documental, esse comportamento reduz significativamente tanto o custo quanto a latência.

Evidências práticas

Durante a aula foram apresentados exemplos concretos da utilização desse padrão em ambientes reais.

Um dos casos discutidos foi o **GPTCache**, projeto de código aberto apresentado em 2023, cujo objetivo é justamente implementar cache semântico utilizando embeddings para comparar perguntas previamente respondidas.

Segundo os resultados apresentados, o mecanismo pode proporcionar ganhos expressivos de velocidade quando ocorre reutilização frequente de respostas.

Também foi mencionado um relato de utilização em produção pelo Walmart Global Tech, no qual o cache semântico atingiu taxas de reutilização superiores às inicialmente previstas pela equipe responsável.

Esses exemplos demonstram que o Semantic Cache deixou de representar apenas um conceito acadêmico e passou a integrar arquiteturas utilizadas em larga escala.

O problema do limiar de similaridade

Apesar dos benefícios, existe uma decisão arquitetural particularmente importante nesse padrão: a definição do limiar de similaridade.

Se esse valor for excessivamente permissivo, perguntas apenas parecidas poderão compartilhar exatamente a mesma resposta.

Nesse cenário, o sistema entrega respostas incorretas com elevado grau de confiança.

Por outro lado, um limiar extremamente conservador faz com que praticamente nenhuma pergunta seja considerada equivalente.

O cache deixa de produzir benefícios relevantes.

Encontrar esse ponto de equilíbrio exige experimentação contínua.

Não existe um valor universal capaz de atender igualmente todas as aplicações.

Cada domínio apresenta características próprias, diferentes níveis de risco e padrões distintos de utilização.

O isolamento entre tenants

Outro aspecto enfatizado durante a aula diz respeito ao isolamento dos dados.

Quando uma mesma infraestrutura atende múltiplos estudos clínicos ou diferentes clientes, o cache nunca deve atravessar essas fronteiras.

Uma resposta produzida para determinado estudo não pode ser reutilizada automaticamente em outro.

Esse problema ultrapassa a simples incorreção da informação.

Ele representa também uma questão de confidencialidade.

Por esse motivo, o isolamento entre tenants torna-se um requisito obrigatório durante a implementação do cache semântico.

Esse mesmo princípio voltará a aparecer posteriormente ao discutirmos arquiteturas multi-tenant.

Benchmarks recentes

A aula também apresentou resultados divulgados pela AWS utilizando o Amazon ElastiCache para cache semântico aplicado a assistentes baseados em RAG.

Os experimentos utilizaram milhares de perguntas reais e mostraram claramente a relação entre limiar de similaridade, taxa de reutilização, precisão e economia operacional.

Os resultados reforçam uma conclusão importante.

A calibração do limiar representa uma atividade contínua de arquitetura.

Ela depende do tipo de pergunta analisada, do risco associado ao erro e das características específicas da aplicação.

Não existe uma configuração única válida para todos os sistemas.

Prompt Cache

O segundo padrão resolve um problema completamente diferente.

Enquanto o Semantic Cache evita repetir uma pergunta já respondida anteriormente, o **Prompt Cache** procura eliminar outro desperdício.

Em muitas aplicações, o usuário faz perguntas diferentes sobre exatamente o mesmo documento.

O protocolo clínico permanece igual.

O System Prompt continua idêntico.

As definições das ferramentas também permanecem inalteradas.

Mesmo assim, todo esse contexto é reenviado ao modelo a cada nova interação.

Reprocessar continuamente esse mesmo conteúdo representa um custo elevado.

O Prompt Cache procura justamente evitar essa repetição.

Reutilizando o contexto

Em vez de reutilizar respostas, o Prompt Cache reutiliza o contexto previamente processado.

Quando um documento longo ou um System Prompt já foi analisado anteriormente, a arquitetura reaproveita esse processamento nas próximas interações.

Apenas a nova pergunta precisa ser efetivamente processada.

Essa diferença é fundamental.

No Semantic Cache toda a chamada ao modelo desaparece.

No Prompt Cache a chamada continua existindo.

O que deixa de acontecer é o reprocessamento completo do contexto.

Os dois padrões não competem entre si.

Eles atuam em camadas diferentes e podem ser utilizados simultaneamente.

Implementações em produção

Durante a aula foram citadas implementações disponibilizadas pelos principais fornecedores de modelos de linguagem.

A Anthropic introduziu seu mecanismo de Prompt Caching em 2024, direcionado principalmente para prompts extensos compostos por documentos, definições de ferramentas e instruções permanentes.

Também foi apresentada a implementação automática disponibilizada pela OpenAI para reutilização de partes repetidas do contexto em prompts longos.

Além disso, foi mencionado um trabalho acadêmico desenvolvido por pesquisadores de Yale em colaboração com o Google, propondo mecanismos ainda mais avançados de reutilização parcial do contexto.

Independentemente da implementação utilizada, todas essas abordagens procuram resolver exatamente o mesmo problema: impedir que o mesmo contexto seja processado repetidamente.

Comparando Semantic Cache e Prompt Cache

Embora ambos utilizem a palavra cache, seus objetivos são completamente diferentes.

O Semantic Cache pergunta:

"Essa pergunta já foi respondida anteriormente?"

Caso a resposta seja positiva, o modelo sequer é chamado.

O Prompt Cache faz outra pergunta:

"Esse contexto já foi processado antes?"

Nesse caso, a inferência continua acontecendo.

O que deixa de ser recalculado é apenas a parte repetida do prompt.

Essa distinção é importante porque evita aplicar o mecanismo incorreto para um problema diferente.

Response Streaming

O terceiro padrão apresentado possui natureza completamente distinta.

Enquanto Semantic Cache e Prompt Cache reduzem custo e processamento, o **Response Streaming** não produz nenhuma economia direta.

O tempo total necessário para gerar a resposta permanece praticamente o mesmo.

O consumo de tokens também não muda.

Mesmo assim, Streaming representa um padrão extremamente importante.

Sua função não consiste em reduzir tempo real.

Seu objetivo é reduzir a percepção de espera.

Latência percebida

Existe uma diferença importante entre latência real e latência percebida.

Uma resposta que demora vinte segundos para ser completamente produzida continuará demorando vinte segundos.

Entretanto, quando o usuário começa a visualizar os primeiros resultados logo no início da geração, a sensação subjetiva de espera muda completamente.

Durante a aula foi utilizada como referência a clássica pesquisa de Jakob Nielsen sobre percepção de tempo em interfaces.

Segundo esses estudos, respostas quase instantâneas mantêm a sensação de fluidez da interação.

Quando o usuário permanece muitos segundos observando uma tela sem qualquer retorno visual, a percepção de lentidão aumenta significativamente.

O Streaming atua exatamente nesse ponto.

Ele permite que o usuário acompanhe a geração do conteúdo à medida que ela acontece.

Streaming e Approval Gate

No contexto do Trial Forge, existe uma preocupação adicional.

Muitos documentos produzidos pelo sistema ainda precisam passar pelo Approval Gate antes de se tornarem oficiais.

Consequentemente, o texto exibido durante o streaming representa apenas um rascunho em construção.

A interface precisa deixar isso claro.

Visualizar o documento sendo produzido não significa que ele já foi aprovado ou validado por um especialista.

Essa distinção evita interpretações incorretas por parte do usuário.

O Cache & Streaming Decision Canvas

Durante a aula também foi apresentado um artefato utilizado para orientar a aplicação desses três padrões.

O processo segue uma sequência lógica de perguntas.

Primeiro verifica-se se a própria pergunta tende a repetir solicitações anteriores.

Quando isso acontece, avalia-se a utilização de Semantic Cache.

Depois analisa-se se existe reutilização frequente de um mesmo contexto documental.

Nesse caso, considera-se a adoção de Prompt Cache.

Por fim, verifica-se se a geração da resposta é suficientemente longa para justificar Response Streaming.

Esse fluxo demonstra que os três padrões não competem.

Cada um atua sobre uma etapa diferente da arquitetura.

Aplicando os padrões ao Trial Forge

No Trial Forge, o Semantic Cache é utilizado principalmente para perguntas rotineiras realizadas repetidamente sobre um mesmo protocolo clínico.

Consultas frequentes realizadas por pesquisadores ou monitores representam candidatos naturais para reutilização de respostas.

Entretanto, existe uma exceção importante.

A geração do Clinical Study Report nunca utiliza respostas provenientes do cache semântico.

Como esse documento possui elevado impacto regulatório, o risco de reutilizar informações potencialmente desatualizadas supera qualquer benefício financeiro obtido pela reutilização.

O Prompt Cache aparece justamente nos protocolos clínicos.

Esses documentos são utilizados continuamente pelos diferentes agentes da plataforma.

Reprocessar integralmente o mesmo protocolo a cada nova pergunta representaria um desperdício significativo de recursos.

Já o Response Streaming torna-se particularmente útil durante a geração do Clinical Study Report, que corresponde ao processo mais demorado de toda a arquitetura.

Nessa situação, acompanhar a construção gradual do documento melhora significativamente a experiência do usuário, mesmo sem alterar o tempo total de processamento.

A invalidação do cache

Antes de encerrar este conjunto de padrões, existe um último aspecto arquitetural importante.

Todo cache precisa responder a uma pergunta aparentemente simples:

Quando uma informação deixa de ser válida?

No Trial Forge, protocolos clínicos podem sofrer emendas ao longo do ciclo de vida do estudo.

Quando isso acontece, respostas produzidas anteriormente deixam de refletir a versão atual do documento.

Consequentemente, o cache correspondente precisa ser invalidado.

Caso contrário, a arquitetura continuará distribuindo respostas desatualizadas com a mesma confiança utilizada para informações corretas.

Esse comportamento transforma o cache de mecanismo de otimização em fonte permanente de inconsistências.

Por esse motivo, estratégias de invalidação fazem parte da própria arquitetura do sistema e devem ser planejadas com o mesmo cuidado dedicado à implementação do cache propriamente dito.

Capítulo 04: Padrões de Design AI-Específicos (PT-4)

Nos capítulos anteriores, apresentei diferentes padrões voltados para otimização de arquiteturas AI-First. Estudamos mecanismos de recuperação de contexto, estratégias de roteamento entre modelos, técnicas de cache semântico, reutilização de prompts e streaming de respostas. Todos esses padrões possuem um objetivo comum: tornar o sistema mais eficiente, reduzindo custo, latência e consumo de recursos.

Entretanto, quanto mais eficiente uma arquitetura se torna, maior passa a ser a tentação de automatizar completamente suas decisões.

Se uma resposta pode ser produzida rapidamente, se o custo caiu e se o sistema já consegue recuperar informações com elevada precisão, surge naturalmente a pergunta: ainda é necessário que alguém revise esse resultado antes de transformá-lo em uma ação oficial?

É justamente essa lacuna que este capítulo procura responder.

Até agora tratamos o Approval Gate apenas como um conceito presente no diagrama de referência da arquitetura AI-First. Sabíamos que determinadas decisões deveriam passar por validação humana, principalmente quando envolviam alto risco. Neste momento, porém, esse conceito deixa de ser apenas uma ideia arquitetural e passa a ser formalizado como um conjunto de componentes técnicos claramente definidos.

Essa formalização é composta por três elementos complementares:

- o mecanismo responsável por interromper a execução;
- o critério que determina quando essa interrupção deve ocorrer;
- a trilha de auditoria que comprova todo o processo de decisão.

Esses três componentes trabalham juntos para transformar a revisão humana em parte integrante da arquitetura, e não apenas em um procedimento operacional.

O Approval Gate como componente arquitetural

Desde o início da disciplina, o Approval Gate apareceu como um ponto de controle antes de ações consideradas críticas.

Naquele momento, bastava compreender que determinadas decisões não deveriam ser tomadas exclusivamente por um modelo generativo.

Agora essa ideia passa a ser tratada de forma muito mais precisa.

Um Approval Gate não representa uma lentidão acidental da arquitetura nem uma etapa esquecida durante a otimização do sistema.

Trata-se de uma pausa deliberadamente inserida antes de uma ação cuja consequência pode ser cara, sensível ou irreversível.

A função desse componente não consiste em reduzir velocidade.

Sua responsabilidade é impedir que determinadas decisões avancem automaticamente quando o risco ultrapassa o limite definido pela arquitetura.

Essa distinção é importante.

Enquanto diversos padrões estudados anteriormente procuram eliminar esperas desnecessárias, o Approval Gate introduz conscientemente uma interrupção no fluxo.

Essa interrupção representa uma decisão arquitetural e não um problema de desempenho.

Formalizando a interrupção

Quando o Approval Gate deixa de ser apenas um conceito e passa a fazer parte da implementação, a arquitetura precisa oferecer mecanismos capazes de interromper o fluxo de maneira controlada.

Essa interrupção não significa encerrar o processamento.

O sistema preserva todo o estado da execução e permanece aguardando uma decisão externa antes de continuar.

Na prática, diferentes plataformas já oferecem recursos específicos para esse comportamento.

Durante a aula foram apresentados exemplos como o mecanismo de interrupção existente no LangGraph, capaz de pausar um fluxo mantendo seu estado completo até que uma aprovação seja recebida.

Também foi discutido o funcionamento dos agentes do AWS Bedrock, que implementam etapas explícitas de confirmação antes da execução de determinadas ações.

Independentemente da tecnologia utilizada, o princípio arquitetural permanece o mesmo.

O fluxo precisa ser capaz de interromper sua execução exatamente no ponto em que a validação humana se torna necessária.

O contexto regulatório

No domínio do Trial Forge, essa decisão possui um significado ainda mais importante.

O sistema produz documentos regulatórios utilizados durante estudos clínicos.

Consequentemente, a necessidade de revisão humana deixa de representar apenas uma boa prática de engenharia.

Ela passa a constituir um requisito associado ao próprio processo regulatório.

Durante a aula foi discutido um documento publicado pelo FDA em 2025 relacionado ao uso de Inteligência Artificial em decisões regulatórias para medicamentos.

Esse material apresenta um framework de classificação de risco no qual sistemas capazes de tomar decisões críticas sem qualquer revisão humana são enquadrados em categorias de influência significativamente mais elevadas.

Em contrapartida, quando existe validação humana antes da decisão final, o risco atribuído ao sistema diminui formalmente.

Esse exemplo reforça um princípio importante.

O Approval Gate não existe apenas para reduzir erros.

Ele também altera a forma como o próprio sistema é classificado do ponto de vista regulatório.

Gates síncronos e assíncronos

Nem toda aprovação precisa interromper completamente o fluxo.

A escolha depende diretamente das consequências produzidas por um eventual erro.

Quando a decisão gera efeitos caros, mas ainda reversíveis, a arquitetura pode utilizar um gate assíncrono.

Nesse modelo, a execução prossegue normalmente, mas permanece marcada para revisão posterior.

Caso algum problema seja identificado durante essa análise, a ação pode ser revertida utilizando mecanismos semelhantes aos estudados anteriormente com o padrão Saga.

Entretanto, quando o erro produz consequências irreversíveis, a situação muda completamente.

Nesse caso, torna-se necessário utilizar um gate síncrono.

Nenhuma etapa posterior é executada enquanto a aprovação explícita não for concedida.

No Trial Forge, a síntese final do Clinical Study Report representa um exemplo típico dessa situação.

Como esse documento pode ser submetido oficialmente para avaliação regulatória, a arquitetura exige validação antes da conclusão definitiva do processo.

O problema da aprovação indiscriminada

Depois de compreender como interromper a execução, surge outra questão igualmente importante.

Se toda decisão passar obrigatoriamente por revisão humana, a automação deixa de produzir benefícios relevantes.

A arquitetura transforma-se apenas em um sistema que organiza filas de aprovação.

Essa abordagem elimina praticamente todas as vantagens obtidas pelos padrões estudados anteriormente.

Por esse motivo, a arquitetura também precisa decidir quando o Approval Gate será realmente acionado.

Essa decisão não pertence ao mecanismo de interrupção.

Ela depende de outro componente.

O limiar de confiança

O segundo padrão apresentado neste capítulo corresponde ao **Confidence Threshold**, ou limiar de confiança.

Seu objetivo consiste em transformar o nível de confiança produzido pelo sistema em uma decisão arquitetural objetiva.

Sempre que a confiança ultrapassa determinado valor previamente definido, o fluxo pode prosseguir automaticamente.

Quando esse valor permanece abaixo do limite estabelecido, o sistema interrompe a execução e encaminha a decisão para validação humana.

O limiar funciona como uma régua.

Ele converte um sinal probabilístico em uma regra operacional claramente definida.

Essa abordagem impede tanto a automatização excessiva quanto a necessidade de revisão manual em todas as tarefas.

Aprendendo a delegar

Durante a aula também foi apresentado o conceito de **Learning to Defer**, formalizado em trabalhos acadêmicos que estudam sistemas capazes de reconhecer quando não devem responder autonomamente.

Em vez de produzir uma resposta incerta, esses modelos aprendem a transferir a decisão para um especialista humano.

Esse comportamento altera significativamente a forma como avaliamos sistemas baseados em Inteligência Artificial.

O objetivo deixa de ser responder todas as perguntas.

Passa a ser responder apenas aquelas para as quais existe confiança suficiente.

As demais são deliberadamente encaminhadas para validação especializada.

Um exemplo de produção

Para ilustrar esse comportamento, a aula utilizou como exemplo o funcionamento do Stripe Radar.

Nesse sistema, cada pagamento recebe uma pontuação de risco.

Dependendo da faixa em que essa pontuação se encontra, três comportamentos diferentes podem ocorrer.

Transações consideradas seguras seguem automaticamente.

Casos intermediários são encaminhados para revisão manual.

Situações classificadas como de risco muito elevado podem ser bloqueadas imediatamente.

O aspecto mais importante desse exemplo não está nos valores específicos utilizados pela plataforma.

O ponto central é a existência de múltiplas faixas de decisão.

A arquitetura deixa de trabalhar apenas com as alternativas "aprovar" ou "rejeitar".

Ela passa a incorporar também o encaminhamento para análise humana.

Recalibrando continuamente

Outro aspecto enfatizado durante a aula é que o limiar de confiança nunca deve ser considerado definitivo.

Sempre que o modelo utilizado pela arquitetura muda de versão ou quando o perfil das solicitações recebidas sofre alterações relevantes, torna-se necessário recalibrar esse limite.

Caso contrário, o sistema pode começar a encaminhar um número excessivo de tarefas para revisão humana ou, no extremo oposto, automatizar decisões que anteriormente exigiam validação.

Esse comportamento lembra diretamente a necessidade de recalibração apresentada anteriormente para outros componentes da arquitetura AI-First.

Audit Trail

Mesmo quando o sistema interrompe corretamente sua execução e encaminha decisões para especialistas, ainda existe uma pergunta importante.

Como comprovar posteriormente que essa aprovação realmente aconteceu?

Essa necessidade introduz o terceiro padrão estudado neste capítulo: a **Audit Trail**.

Uma trilha de auditoria transforma uma aprovação em um fato verificável.

Ela registra permanentemente quem tomou determinada decisão, quando ela ocorreu e quais condições estavam presentes naquele momento.

Sem esse registro, qualquer processo de aprovação passa a depender exclusivamente da memória das pessoas envolvidas.

O requisito regulatório

A necessidade de trilhas de auditoria não surgiu com a Inteligência Artificial.

Durante a aula foi apresentada a regulamentação **21 CFR Part 11**, publicada pelo FDA, que exige registros eletrônicos capazes de documentar alterações realizadas em sistemas regulatórios.

Também foi discutido o **EU AI Act**, que amplia essa necessidade para sistemas classificados como de alto risco.

Embora pertençam a contextos diferentes, ambos os documentos convergem para o mesmo princípio.

Sistemas capazes de produzir decisões relevantes precisam registrar automaticamente seus eventos de forma permanente.

O que deve ser registrado

Registrar apenas que uma aprovação aconteceu não é suficiente.

Uma trilha de auditoria útil precisa responder às perguntas normalmente realizadas durante uma investigação.

Entre as informações consideradas essenciais estão:

- qual agente ou qual pessoa tomou a decisão;
- data e horário da execução;
- versão do prompt utilizada;
- versão do modelo responsável pela geração;
- limiar de confiança aplicado naquele momento;

- resultado final da decisão, indicando se a solicitação foi aprovada, rejeitada ou encaminhada para revisão.

Sem esse conjunto mínimo de informações, a trilha existe apenas formalmente, mas não oferece suporte adequado para auditorias reais.

Registros imutáveis

Um princípio igualmente importante diz respeito à imutabilidade da trilha.

Quando uma decisão precisa ser corrigida posteriormente, o registro anterior não deve ser apagado.

A arquitetura cria um novo evento informando a revisão da decisão anterior, preservando integralmente o histórico.

Esse comportamento segue exatamente a mesma lógica apresentada anteriormente durante o estudo do versionamento e das ações compensatórias.

A história do processo permanece completa.

Não existe reescrita do passado.

Essa característica garante rastreabilidade mesmo quando erros humanos acontecem durante o processo de aprovação.

Formalizando o Approval Gate

Durante a aula foi apresentado um artefato denominado **Human-in-the-Loop Formalization Canvas**, utilizado para estruturar a implementação desses três padrões.

O processo inicia identificando qual sinal de confiança estará disponível para determinada tarefa.

Em seguida, define-se o limiar que separa execução automática de revisão humana.

Depois determina-se se o Approval Gate deverá funcionar de forma síncrona ou assíncrona.

Por fim, especifica-se quais informações obrigatoriamente passarão a compor a trilha de auditoria.

Esse canvas transforma um conceito arquitetural relativamente abstrato em um conjunto organizado de decisões técnicas, permitindo que o Approval Gate seja implementado de maneira consistente em diferentes componentes da arquitetura.

Aplicando ao Trial Forge

No Trial Forge, esses três padrões aparecem de forma integrada durante a geração do Clinical Study Report.

O modelo produz inicialmente a síntese do documento e calcula seu nível de confiança.

Esse valor é comparado com o limiar previamente calibrado para esse tipo de tarefa.

Como a publicação desse documento possui elevado impacto regulatório, a arquitetura aciona obrigatoriamente um Approval Gate síncrono antes da conclusão do processo.

O especialista responsável realiza a revisão, registra sua decisão e todas as informações correspondentes passam a integrar permanentemente a trilha de auditoria.

Dessa forma, a arquitetura não apenas reduz riscos durante a geração dos documentos, mas também produz evidências verificáveis capazes de demonstrar, posteriormente, que todo o processo ocorreu conforme as exigências técnicas e regulatórias definidas para o sistema.

Capítulo 05: Padrões de Design AI-Específicos (PT-5)

Ao longo deste módulo, apresentei quatro grupos de padrões fundamentais para arquiteturas AI-First. Inicialmente, estudamos os diferentes modelos de Retrieval Augmented Generation e como estratégias como Multi-Index, Hybrid Search e Agentic RAG ampliam a capacidade de recuperação de informações. Depois, vimos como o roteamento permite escolher tanto o fluxo correto quanto o modelo mais adequado para cada solicitação. Em seguida, analisei mecanismos de otimização baseados em cache semântico, reutilização de contexto e streaming de respostas. Por fim, formalizei o Approval Gate, o limiar de confiança e a trilha de auditoria como componentes permanentes da arquitetura.

Até esse momento, cada grupo de padrões foi estudado isoladamente.

Essa abordagem foi necessária para compreender a função individual de cada componente.

Entretanto, nenhuma aplicação de produção executa esses mecanismos de maneira completamente independente.

Em uma arquitetura real, todos eles participam da mesma requisição, cada um assumindo sua responsabilidade em um momento específico do fluxo.

Neste capítulo, reúno esses quatro grupos de padrões em uma única arquitetura integrada, utilizando novamente o Trial Forge como referência para demonstrar como essas decisões trabalham juntas durante a execução de uma solicitação.

A diferença entre conhecer padrões e construir uma arquitetura

Aprender um padrão isoladamente é relativamente simples.

O verdadeiro desafio aparece quando diversos padrões precisam colaborar dentro da mesma requisição.

Nesse momento, a ordem em que cada decisão acontece torna-se tão importante quanto a existência do próprio padrão.

Não basta simplesmente adicionar Semantic Cache, Model Router, RAG e Approval Gate ao mesmo sistema.

É necessário definir exatamente quando cada um deles será executado.

Essa sequência determina tanto a eficiência quanto a previsibilidade da arquitetura.

Alterar essa ordem pode aumentar custos, introduzir processamento desnecessário ou comprometer a qualidade da decisão produzida pelo sistema.

Por esse motivo, a integração dos padrões representa uma decisão arquitetural independente da implementação de cada componente.

O gateway como ponto central

Toda a arquitetura apresentada neste módulo converge para um único componente: o gateway.

É nesse ponto que cada requisição entra no sistema.

O gateway deixa de ser apenas um mecanismo de entrada e passa a concentrar todas as decisões responsáveis por determinar como aquela solicitação será tratada.

Ele não produz diretamente a resposta.

Sua responsabilidade consiste em organizar o fluxo.

Cada requisição percorre exatamente a mesma sequência de decisões.

Dependendo das características da solicitação, alguns caminhos podem ser interrompidos antecipadamente enquanto outros exigirão processamento completo.

Essa abordagem mantém a arquitetura consistente e reduz a necessidade de decisões distribuídas entre diversos componentes.

A primeira decisão: identificar a intenção

O fluxo sempre começa pela identificação da intenção da requisição.

Antes de escolher qualquer modelo ou consultar qualquer base de conhecimento, a arquitetura precisa compreender qual problema está sendo apresentado.

No Trial Forge, essa classificação distingue, por exemplo, consultas rotineiras relacionadas ao protocolo clínico de solicitações destinadas à geração do Clinical Study Report.

Durante a implementação do protótipo, essa etapa utiliza um classificador determinístico simples.

Não existe necessidade de aplicar um modelo sofisticado para resolver uma tarefa cuja natureza pode ser identificada por regras objetivas.

Esse comportamento reforça um princípio recorrente ao longo de toda a disciplina.

Nem toda decisão precisa ser delegada à Inteligência Artificial.

Sempre que uma solução determinística for suficiente, ela continua sendo a melhor alternativa.

Antes de qualquer inferência

Depois que a intenção foi identificada, a arquitetura ainda não envia a solicitação para um modelo.

Existe uma decisão anterior que pode eliminar completamente a necessidade de processamento.

Nesse momento entra em funcionamento o Semantic Cache.

O sistema verifica se uma pergunta semanticamente equivalente já foi respondida anteriormente.

Caso a similaridade ultrapasse o limiar previamente calibrado, a resposta armazenada é devolvida imediatamente.

Nenhuma chamada ao modelo acontece.

Nenhuma busca documental é executada.

Todo o restante do fluxo deixa simplesmente de existir para aquela requisição.

Essa decisão possui enorme impacto financeiro.

Quando o cache consegue atender à solicitação, toda a etapa de roteamento e geração deixa de consumir recursos computacionais.

Por que o cache vem antes do roteamento

Existe uma razão arquitetural importante para essa ordem.

Pode parecer natural escolher primeiro qual modelo será utilizado e somente depois consultar o cache.

Entretanto, essa sequência introduziria trabalho desnecessário.

Se a resposta já existe armazenada, não faz sentido desperdiçar tempo classificando qual modelo deveria responder à pergunta.

A decisão de roteamento somente possui utilidade quando realmente será necessário executar uma inferência.

Essa observação demonstra que a ordem entre os padrões não representa um detalhe de implementação.

Ela influencia diretamente o custo operacional da arquitetura.

Escolhendo o modelo

Quando o cache não consegue responder à solicitação, entra em funcionamento o Model Router.

Agora a arquitetura já conhece a intenção da requisição.

Com base nessa informação, torna-se possível selecionar o modelo mais adequado.

Consultas simples podem ser encaminhadas para modelos menores e de menor custo.

Solicitações envolvendo raciocínio complexo ou documentos regulatórios seguem para modelos mais sofisticados.

Essa decisão acontece antes da geração da resposta.

O roteador não produz conteúdo.

Ele apenas escolhe qual recurso será utilizado durante a próxima etapa do fluxo.

Recuperando contexto

Depois da escolha do modelo, inicia-se a recuperação das informações necessárias.

No Trial Forge, essa recuperação combina diferentes padrões apresentados anteriormente.

O Multi-Index identifica qual índice documental deve ser consultado.

Em seguida, o Hybrid Search combina recuperação vetorial com busca lexical utilizando BM25.

Essa combinação amplia simultaneamente cobertura e precisão.

Entretanto, a arquitetura ainda possui um mecanismo adicional.

Quando a qualidade da recuperação permanece abaixo do esperado, entra em funcionamento o Agentic RAG.

Nesse momento, o sistema amplia automaticamente sua estratégia de busca.

Novas consultas são realizadas utilizando abordagens diferentes, sempre com o objetivo de localizar contexto suficiente antes da geração da resposta.

No protótipo apresentado durante a aula, esse processo admite até três tentativas antes de considerar encerrada a etapa de recuperação.

A confiança deixa de ser apenas um indicador

Um aspecto importante dessa arquitetura é que a confiança obtida durante a recuperação não permanece apenas como informação estatística.

Ela passa a influenciar diretamente as próximas decisões do fluxo.

Quanto melhor a qualidade da recuperação documental, maior tende a ser a confiança atribuída ao resultado produzido.

Esse valor torna-se posteriormente um dos principais sinais utilizados pelo mecanismo de Approval Gate.

Observe que a confiança não surge de maneira isolada.

Ela nasce durante a própria execução da arquitetura.

Streaming da resposta

Depois que o modelo conclui sua geração, entra em funcionamento o Response Streaming.

A resposta começa a ser apresentada gradualmente ao usuário.

Essa estratégia não altera o tempo total necessário para concluir a inferência.

Entretanto, reduz significativamente a percepção de espera durante interações longas.

No Trial Forge, esse comportamento torna-se especialmente importante durante a geração de documentos extensos.

Enquanto o sistema continua produzindo o restante do conteúdo, o usuário já acompanha o resultado sendo construído.

Essa característica melhora a experiência sem modificar o funcionamento interno da arquitetura.

Confidence Threshold

Ao término da geração, a arquitetura compara o nível de confiança obtido durante o processo com o limiar previamente calibrado.

Esse mecanismo transforma um valor probabilístico em uma decisão objetiva.

Quando a confiança permanece acima do limite estabelecido para aquela categoria de tarefa, o fluxo pode prosseguir automaticamente.

Caso contrário, a arquitetura interrompe sua execução e encaminha a solicitação para validação humana.

Essa decisão não depende apenas do valor numérico da confiança.

Algumas categorias de tarefa continuam exigindo aprovação independentemente desse indicador.

Approval Gate

No Trial Forge existe um exemplo bastante claro dessa situação.

Sempre que a arquitetura produz a síntese final do Clinical Study Report, o fluxo obrigatoriamente passa pelo Approval Gate.

Mesmo que a confiança calculada seja elevada, a criticidade regulatória desse documento exige confirmação humana antes de sua oficialização.

Esse comportamento demonstra que o limiar de confiança representa apenas um dos critérios considerados pela arquitetura.

O tipo da tarefa continua sendo igualmente importante.

Algumas decisões simplesmente não podem ser automatizadas.

Audit Trail

Independentemente do caminho percorrido pela requisição, a etapa final permanece a mesma.

Toda decisão relevante passa a integrar automaticamente a trilha de auditoria.

Não importa se a solicitação foi atendida diretamente pelo cache, se utilizou um modelo simples, se exigiu múltiplas buscas documentais ou se precisou de aprovação humana.

Cada uma dessas decisões produz registros permanentes.

Esses registros documentam o comportamento da arquitetura e permitem reconstruir posteriormente toda a sequência de eventos associada à execução.

Essa rastreabilidade representa um requisito especialmente importante em sistemas submetidos a auditorias regulatórias.

O Gateway Blueprint Canvas

Durante a aula foi apresentado um artefato responsável por organizar todas essas decisões antes mesmo da implementação.

Esse documento divide o projeto em quatro áreas principais.

A primeira corresponde à definição das categorias de intenção.

Para cada categoria são registrados exemplos de perguntas e o modelo mais adequado para processá-las, sempre acompanhados da justificativa correspondente.

A segunda área trata da calibração do Semantic Cache.

Em vez de assumir arbitrariamente um limiar de similaridade, a arquitetura utiliza pares reais de perguntas para medir o comportamento do mecanismo de embeddings e determinar um valor adequado ao domínio da aplicação.

A terceira seção documenta os critérios do Approval Gate.

Nela são identificadas todas as categorias de tarefa que obrigatoriamente exigirão validação humana, independentemente da confiança calculada durante a execução.

Por fim, o blueprint registra o checklist mínimo da trilha de auditoria, especificando quais informações devem ser armazenadas em cada decisão produzida pelo sistema.

Esse documento transforma a implementação do gateway em consequência direta de decisões arquiteturais previamente documentadas.

Um protótipo executável

Outro aspecto importante apresentado durante a aula é que essa arquitetura deixa de existir apenas como diagrama.

O protótipo disponibilizado juntamente com o módulo implementa efetivamente todas essas decisões.

O fluxo completo executa localmente, utilizando modelos distintos para tarefas simples e complexas, além de um modelo específico para geração de embeddings.

Essa escolha permite experimentar toda a arquitetura sem depender imediatamente de serviços externos pagos.

O objetivo do protótipo não é reproduzir exatamente um ambiente corporativo de produção.

Sua finalidade consiste em permitir que toda a sequência arquitetural seja observada funcionando de maneira integrada.

A importância da ordem

Ao observar o fluxo completo, torna-se evidente que cada padrão depende diretamente das decisões produzidas pelo anterior.

O Intent-Based Routing identifica o destino da requisição.

O Semantic Cache verifica se o processamento realmente será necessário.

O Model Router escolhe o recurso mais adequado.

O RAG recupera o contexto documental.

A confiança produzida durante essa recuperação alimenta o Confidence Threshold.

Esse limiar determina a necessidade do Approval Gate.

Por fim, toda a execução é registrada automaticamente pela Audit Trail.

Essa sequência não representa apenas uma implementação possível.

Ela reflete uma lógica arquitetural cuidadosamente construída para reduzir custo, preservar qualidade, minimizar processamento desnecessário e manter governança sobre todas as decisões produzidas pela Inteligência Artificial.

Ao final deste módulo, os padrões deixam de ser componentes independentes e passam a formar uma única arquitetura operacional, capaz de transformar uma requisição em um fluxo completo de decisão, recuperação de contexto, geração de resposta, validação e auditoria. É justamente essa integração que diferencia um conjunto de técnicas isoladas de uma plataforma AI-First preparada para ambientes corporativos.

Módulo 05: Arquitetura Enterprise

Capítulo 01: Arquitetura Enterprise (PT-1)

Ao longo dos módulos anteriores, construímos gradualmente a arquitetura do Trial Forge analisando uma requisição por vez. Estudamos como um gateway

recebe uma solicitação, como um orquestrador coordena agentes, como mecanismos de RAG recuperam contexto, como o Approval Gate controla decisões críticas e como diferentes padrões de design tornam esse fluxo mais eficiente.

Neste módulo, a perspectiva muda completamente.

Deixo de observar uma única requisição para analisar o comportamento de toda a plataforma em produção. O problema deixa de ser responder corretamente a uma pergunta individual e passa a ser administrar centenas ou milhares de requisições simultâneas, provenientes de equipes diferentes, estudos clínicos distintos e múltiplos países, todos compartilhando a mesma infraestrutura.

É justamente essa mudança de escala que caracteriza uma arquitetura enterprise.

Da requisição para a plataforma

Até este momento, todas as decisões arquiteturais estavam concentradas na execução de um fluxo específico.

O diagrama de referência apresentado nos primeiros módulos mostrava como uma submissão era processada.

O gateway estudado anteriormente decidia corretamente para onde encaminhar uma solicitação.

O orquestrador distribuía responsabilidades entre agentes.

Os modelos produziam respostas.

Os mecanismos de recuperação enriqueciam o contexto.

Entretanto, nenhuma dessas estruturas respondia a uma pergunta diferente.

O que acontece quando deixam de existir poucas requisições cuidadosamente controladas e passam a existir dezenas de equipes utilizando simultaneamente a mesma plataforma?

Essa é exatamente a mudança introduzida neste módulo.

No contexto da Vitalis Pharma, o Trial Forge deixa de atender apenas um estudo clínico e passa a operar como uma plataforma corporativa compartilhada por diversos projetos executados ao mesmo tempo.

O custo muda completamente

Durante os módulos anteriores, o custo principal discutido esteve associado ao consumo de modelos generativos e à utilização de tokens.

Em um cenário com poucos estudos clínicos, duplicar alguns componentes de infraestrutura não representa necessariamente um problema relevante.

Quando a plataforma cresce, entretanto, essa realidade muda completamente.

O maior custo deixa de ser a execução individual das inferências.

O verdadeiro desafio passa a ser o compartilhamento eficiente da infraestrutura.

Se cada novo estudo criar sua própria implementação de todos os serviços utilizados pela plataforma, o crescimento operacional torna-se rapidamente inviável.

A arquitetura enterprise surge justamente para impedir essa multiplicação desnecessária de componentes.

Os quatro componentes do stack

Para organizar essa infraestrutura, utilizo uma arquitetura composta por quatro grandes componentes.

Cada um deles resolve um problema que somente aparece quando a plataforma começa a operar em escala.

O primeiro componente é o **API Gateway**.

Em vez de cada estudo clínico possuir seu próprio ponto de entrada, toda a plataforma compartilha uma única camada responsável por receber as requisições.

Essa centralização simplifica autenticação, controle de acesso, limitação de taxa, observabilidade e gerenciamento dos diferentes modelos utilizados pela organização.

Hoje esse tipo de arquitetura já aparece em diversos produtos consolidados.

Durante a aula foram apresentados exemplos como o LightLLM, utilizado para unificar diferentes provedores de modelos por meio de uma única interface, além das soluções de AI Gateway disponibilizadas pela Cloudflare e pelo Kong, que incorporam recursos como cache, limitação de requisições e failover entre modelos.

Independentemente da tecnologia adotada, o princípio permanece o mesmo.

Todos os estudos entram pela mesma porta.

Orquestração em escala

O segundo componente corresponde à camada de orquestração.

Seu papel consiste em administrar como os serviços são executados dentro da infraestrutura.

No contexto apresentado durante a aula, essa responsabilidade é exercida pelo Kubernetes.

Entretanto, neste módulo o Kubernetes aparece sob uma perspectiva diferente daquela utilizada anteriormente.

Em disciplinas relacionadas a DevOps, normalmente ele é estudado como ferramenta operacional.

Aqui ele passa a representar a fundação sobre a qual toda a plataforma de Inteligência Artificial será executada.

Sua responsabilidade principal consiste em decidir quantas instâncias de cada serviço permanecerão ativas, ampliando ou reduzindo automaticamente essa quantidade conforme o volume de utilização.

A plataforma deixa de depender de dimensionamentos estáticos e passa a adaptar sua capacidade de processamento ao comportamento real da carga de trabalho.

Serviços compartilhados

O terceiro componente é formado pelos serviços compartilhados.

Essa camada representa um dos principais diferenciais da arquitetura enterprise.

Serviços como geração de embeddings, modelos de linguagem, mecanismos de cache ou recuperação de contexto deixam de existir individualmente dentro de cada estudo.

Em vez disso, passam a existir uma única vez em toda a plataforma.

Cada novo estudo reutiliza exatamente a mesma infraestrutura.

Esse compartilhamento reduz custos, simplifica manutenção e evita que diferentes equipes implementem repetidamente soluções equivalentes.

O antipadrão da reimplementação

Durante a aula foi destacado um antipadrão bastante comum em organizações que crescem rapidamente.

Cada equipe desenvolve sua própria versão dos mesmos serviços.

Um estudo cria um serviço de embeddings.

Outro implementa uma versão diferente.

Um terceiro utiliza outro provedor de modelos.

Depois de algum tempo, a organização passa a manter dezenas de implementações praticamente idênticas.

Cada uma possui seus próprios defeitos, sua própria configuração e seu próprio ciclo de manutenção.

Esse comportamento transforma a evolução da plataforma em uma tarefa extremamente cara.

A camada de serviços compartilhados existe justamente para impedir esse cenário.

Em vez de trinta versões diferentes do mesmo componente, a organização mantém uma implementação única, reutilizada por todos os estudos.

Observabilidade unificada

O quarto componente corresponde à observabilidade.

Assim como os demais elementos da infraestrutura, ela também deixa de ser específica de cada estudo.

Logs, métricas, trilhas de auditoria e eventos passam a compor uma visão única da plataforma.

Essa centralização facilita investigações, reduz fragmentação das informações e permite compreender o comportamento global da arquitetura.

O objetivo não é eliminar a capacidade de distinguir estudos diferentes.

Cada execução continua sendo identificada individualmente.

O que muda é a existência de uma única infraestrutura responsável por registrar todos esses eventos.

Compartilhar não significa substituir

Um aspecto importante enfatizado durante a aula é que essa nova camada enterprise não substitui a arquitetura construída anteriormente.

O fluxo individual continua existindo exatamente da mesma forma.

Cada requisição ainda percorre gateway, orquestrador, modelos, mecanismos de recuperação de contexto e Approval Gate.

A diferença está na existência de uma camada superior responsável por administrar como múltiplas instâncias desse mesmo fluxo compartilham infraestrutura sem interferirem umas nas outras.

A arquitetura enterprise multiplica a arquitetura anterior.

Ela não a substitui.

Escalando sem recriar

Dentro da camada de orquestração existe outro conceito importante.

Escalar uma plataforma não significa criar continuamente novas implementações dos mesmos serviços.

Escalar significa administrar dinamicamente as instâncias existentes.

Durante a aula foi apresentado o KServe como exemplo desse comportamento.

Entre seus recursos está a possibilidade de realizar atualizações graduais de modelos por meio da distribuição controlada do tráfego.

Uma pequena parcela das requisições passa inicialmente a utilizar a nova versão do modelo.

O restante continua sendo processado pela versão estável.

Caso algum problema seja identificado, basta interromper imediatamente o encaminhamento para a nova versão.

Não existe necessidade de interromper toda a plataforma nem realizar novas implantações.

Essa estratégia reduz significativamente o risco associado à evolução contínua dos modelos utilizados em produção.

KServe e KubeFlow

Outro ponto importante apresentado durante a aula consiste na distinção entre KServe e KubeFlow.

Embora ambos façam parte do ecossistema Kubernetes, eles resolvem problemas diferentes.

O KubeFlow atua durante o desenvolvimento dos modelos.

Sua responsabilidade está relacionada à organização dos pipelines de treinamento, experimentação e validação.

O KServe aparece posteriormente.

Ele administra a disponibilização dos modelos já treinados para utilização em produção.

Essas duas ferramentas pertencem ao mesmo ciclo de vida, mas ocupam momentos completamente diferentes dentro da arquitetura.

Promover um modelo exige mais do que implantá-lo

Atualizar um modelo em produção representa apenas parte do problema.

Ainda existe uma pergunta anterior.

A nova versão realmente deveria substituir aquela que já está em funcionamento?

Para responder essa questão, a arquitetura incorpora um mecanismo adicional denominado **Evaluation Gate**.

Antes que um novo modelo seja promovido, ele é executado contra um conjunto conhecido de perguntas e respostas utilizado como referência.

Esse conjunto funciona como um Golden Set.

A arquitetura mede indicadores de qualidade comparando o comportamento da versão candidata com o modelo atualmente utilizado.

Somente quando a nova versão mantém desempenho compatível com a tolerância previamente definida sua promoção é autorizada.

Caso contrário, a atualização é bloqueada antes mesmo de alcançar os usuários finais.

Essa abordagem impede que regressões de qualidade sejam descobertas apenas depois da implantação.

Testando configurações, não apenas modelos

Durante a demonstração apresentada na aula, um aspecto importante ficou evidente.

Nem toda regressão decorre da troca do modelo.

Uma configuração incorreta também pode produzir degradação significativa de qualidade.

Um exemplo utilizado consistiu na remoção acidental de uma cláusula do contexto enviado ao modelo.

Mesmo utilizando exatamente o mesmo modelo, o desempenho caiu além do limite aceitável, fazendo com que o Evaluation Gate bloqueasse automaticamente a promoção.

Esse exemplo demonstra que o mecanismo de avaliação protege não apenas contra mudanças de modelo, mas também contra alterações inadequadas na própria configuração da arquitetura.

Três princípios da arquitetura enterprise

Depois de compreender os quatro componentes da plataforma, surge outra questão.

Como garantir que dezenas de estudos compartilhem a mesma infraestrutura sem interferirem entre si?

A resposta está em três princípios arquiteturais fundamentais.

O primeiro deles é o **Loose Coupling**.

Seu objetivo consiste em reduzir dependências entre diferentes partes da arquitetura.

Cada estudo deve ser capaz de evoluir seus componentes sem provocar impactos sobre os demais.

Uma atualização realizada em determinado fluxo não pode quebrar automaticamente todos os outros consumidores da plataforma.

Clear Interfaces

O segundo princípio corresponde às **Clear Interfaces**.

Cada serviço compartilhado deve expor contratos explícitos.

Esses contratos definem claramente quais informações são recebidas, quais respostas serão produzidas e quais garantias permanecem válidas durante sua utilização.

Os consumidores dependem apenas dessas interfaces.

Os detalhes internos da implementação permanecem isolados.

Essa separação facilita evolução, manutenção e substituição de componentes sem exigir alterações em toda a plataforma.

Policy-Driven Control

O terceiro princípio é denominado **Policy-Driven Control**.

Nesse modelo, regras relacionadas a autorização, limites de custo e utilização dos recursos deixam de ficar distribuídas pelo código dos diferentes agentes.

Todas essas decisões passam a ser controladas por políticas centralizadas, avaliadas dinamicamente durante a execução.

Esse comportamento simplifica governança e garante que alterações nas regras organizacionais possam ser realizadas sem necessidade de modificar individualmente cada componente da arquitetura.

Durante a aula foi citado o Open Policy Agent como uma implementação de referência para esse tipo de abordagem.

Os três princípios trabalhando juntos

Embora tratem aspectos diferentes da arquitetura, esses três princípios respondem à mesma preocupação.

O que acontece quando um estudo apresenta algum problema?

O Loose Coupling impede que essa falha se propague para os demais estudos.

As Clear Interfaces garantem que nenhum consumidor dependa de detalhes internos que possam mudar inesperadamente.

O Policy-Driven Control assegura que, mesmo funcionando isoladamente, nenhum componente ultrapasse limites de utilização previamente definidos pela organização.

Em conjunto, esses princípios fornecem exatamente as características esperadas de uma plataforma enterprise: modularidade, segurança e escalabilidade.

Compartilhado versus específico

Um dos exercícios apresentados durante a aula consiste justamente em identificar quais componentes pertencem à infraestrutura compartilhada e quais continuam sendo específicos de cada estudo.

Essa distinção representa uma das decisões mais importantes da arquitetura enterprise.

Nem tudo deve ser compartilhado.

Da mesma forma, nem tudo deve permanecer isolado.

Gateway, serviços de embeddings, modelos, observabilidade e mecanismos de orquestração normalmente pertencem à infraestrutura comum.

Já documentos, protocolos clínicos, configurações específicas de determinado estudo e determinadas políticas regulatórias continuam associados ao seu respectivo domínio.

Esse equilíbrio impede tanto duplicações desnecessárias quanto compartilhamentos inadequados entre contextos independentes.

Ao final deste primeiro capítulo, fica evidente que a principal mudança da arquitetura enterprise não está na criação de novos agentes nem na introdução de novos modelos de linguagem. A verdadeira transformação acontece na forma como toda a infraestrutura passa a ser organizada para permitir que dezenas de equipes compartilhem os mesmos recursos de maneira segura, escalável e governável, preparando a plataforma para operar continuamente em ambiente de produção.

Capítulo 02: Arquitetura Enterprise (PT-2)

No capítulo anterior, ampliei a perspectiva da arquitetura AI-First para o contexto enterprise. Até então, analisávamos uma única requisição percorrendo gateway, orquestrador, modelos e mecanismos de recuperação de contexto. A partir

daquele momento, passamos a observar dezenas de estudos clínicos compartilhando a mesma infraestrutura, reutilizando serviços comuns e sendo administrados por uma única plataforma.

Entretanto, essa mudança de escala introduz um problema completamente novo.

Quando existe apenas um estudo clínico em execução, identificar falhas costuma ser relativamente simples. Já em uma plataforma multi-tenant, utilizada simultaneamente por dezenas de equipes, os problemas deixam de aparecer de forma isolada. Um comportamento anormal pode permanecer escondido no meio de milhares de requisições perfeitamente saudáveis.

Neste capítulo, apresento como a observabilidade evolui para atender esse novo cenário. Mais do que monitorar servidores ou APIs, a arquitetura passa a rastrear o comportamento da própria Inteligência Artificial, registrando evidências suficientes para explicar não apenas se o sistema funcionou, mas também por que determinada resposta foi produzida.

Observabilidade em uma plataforma AI-First

Observabilidade não representa um conceito novo dentro da arquitetura construída até aqui.

Desde os primeiros módulos, a camada de observabilidade já aparecia atravessando todos os componentes da solução. Posteriormente, formalizamos a trilha de auditoria utilizada durante os processos de aprovação humana.

O que muda agora é a escala.

Uma coisa é acompanhar o comportamento de uma única requisição.

Outra completamente diferente consiste em agregar milhares de eventos produzidos simultaneamente por dezenas de estudos clínicos distintos, identificando quais sinais realmente indicam um problema relevante.

Essa diferença transforma observabilidade em um componente estratégico da arquitetura enterprise.

A limitação da observabilidade tradicional

Existe uma armadilha bastante comum quando aplicamos práticas tradicionais de observabilidade a sistemas baseados em Inteligência Artificial.

É perfeitamente possível que toda a infraestrutura esteja funcionando corretamente enquanto a qualidade das respostas produzidas pelo modelo já começou a se deteriorar.

CPU permanece estável.

Memória continua disponível.

Latência está dentro do esperado.

As APIs respondem normalmente.

Mesmo assim, o conteúdo gerado pelo modelo deixa de atender aos padrões de qualidade esperados.

Esse tipo de situação dificilmente seria percebido apenas observando indicadores clássicos de infraestrutura.

Por esse motivo, arquiteturas AI-First exigem uma camada adicional de observabilidade, voltada especificamente para o comportamento dos modelos.

Prompt como código

O primeiro desafio discutido durante a aula envolve o próprio prompt utilizado em produção.

Durante a evolução de uma aplicação, ajustes de prompt tornam-se inevitáveis.

Entretanto, quando essas alterações acontecem sem controle, surge um problema importante.

Imagine que determinado comportamento do agente mude repentinamente.

Sem histórico de versões, torna-se praticamente impossível responder uma pergunta simples:

A mudança foi provocada pelo modelo ou por uma alteração no prompt?

Os dois cenários produzem sintomas praticamente idênticos.

A arquitetura precisa, portanto, registrar exatamente qual versão do prompt foi utilizada em cada execução.

Versionamento de prompts

A solução consiste em tratar prompts da mesma forma que tratamos código-fonte.

Cada alteração passa a representar uma nova versão.

O histórico permanece preservado.

Diferenças entre versões podem ser comparadas.

Quando necessário, torna-se possível retornar para uma versão anterior ou reproduzir exatamente uma execução antiga utilizando um prompt diferente.

Esse comportamento aproxima a engenharia de prompts das práticas tradicionais de engenharia de software.

Em vez de alterações invisíveis realizadas diretamente em produção, cada modificação passa a ser rastreável.

Diff e Replay

Durante a aula foram destacados dois recursos particularmente importantes nesse processo.

O primeiro é o **Diff**.

Antes que uma nova versão seja publicada, a arquitetura apresenta exatamente quais trechos foram modificados.

Essa comparação facilita revisões e reduz alterações acidentais.

O segundo recurso corresponde ao **Replay**.

Sempre que uma investigação precisa reproduzir determinado comportamento, torna-se possível executar novamente uma requisição antiga utilizando diferentes versões do prompt.

Essa capacidade permite comparar lado a lado como pequenas alterações influenciam a resposta produzida pelo modelo.

Sem esse mecanismo, boa parte das investigações dependeria apenas de hipóteses.

Ferramentas para rastreamento

A aula também apresentou ferramentas voltadas especificamente para esse tipo de observabilidade.

Entre elas está o Langfuse, plataforma especializada em rastreamento de aplicações baseadas em Inteligência Artificial.

Também foi apresentado o Arize Phoenix, projeto de código aberto construído sobre o OpenInference, que utiliza OpenTelemetry como infraestrutura de instrumentação, acrescentando conceitos específicos para aplicações de IA.

Independentemente da ferramenta adotada, o princípio permanece o mesmo.

Prompts deixam de ser configurações ocultas e passam a fazer parte dos artefatos oficialmente monitorados pela arquitetura.

Para organizações que trabalham com informações sensíveis, como a Vitalis Pharma, soluções executadas integralmente dentro da própria infraestrutura representam uma vantagem adicional, permitindo manter todos os registros sob controle da organização.

O que registrar em uma chamada de IA

Depois de decidir que toda execução será registrada, surge outra questão importante.

Quais informações realmente precisam compor esse registro?

Uma chamada para um modelo de linguagem carrega metadados que simplesmente não existem em uma API tradicional.

Não basta registrar apenas entrada e saída.

A arquitetura precisa documentar também o contexto da inferência.

Durante a aula foram apresentados exemplos de campos já contemplados pelas convenções semânticas de IA em desenvolvimento dentro do OpenTelemetry.

Entre eles destacam-se:

- quantidade de tokens de entrada e saída;
- modelo efetivamente utilizado;
- tempo até a geração do primeiro token;
- tempo total de processamento;
- documentos recuperados durante a etapa de RAG.

Esses elementos permitem reconstruir posteriormente toda a execução realizada pelo sistema.

Metadados que dialogam com a arquitetura

Cada um desses registros conecta-se diretamente aos módulos estudados anteriormente.

O tempo até o primeiro token relaciona-se ao padrão Response Streaming, utilizado para reduzir a percepção de espera durante respostas longas.

Os documentos recuperados permitem identificar exatamente qual mecanismo de RAG fundamentou determinada resposta.

O modelo utilizado explica quais regras do Model Router foram aplicadas durante aquela execução.

Percebe-se, portanto, que a observabilidade não introduz novos conceitos.

Ela apenas registra evidências suficientes para demonstrar como os componentes arquiteturais efetivamente trabalharam durante cada requisição.

Os quatro sinais clássicos

Quando falamos em observabilidade de sistemas distribuídos, existe um conjunto de métricas tradicionalmente utilizado para acompanhar o comportamento da infraestrutura.

Durante a aula foram retomados os quatro sinais clássicos descritos na literatura de Site Reliability Engineering:

- latência;
- tráfego;
- erros;
- saturação.

Esses indicadores continuam extremamente importantes em plataformas AI-First.

A diferença está na interpretação de alguns deles.

Em aplicações baseadas em Inteligência Artificial, saturação deixa de representar apenas utilização de CPU e memória.

Também passa a considerar filas de inferência, utilização de GPUs e disponibilidade dos modelos responsáveis pelas respostas.

Mesmo assim, esses quatro indicadores continuam descrevendo apenas o estado operacional da infraestrutura.

Quando toda a infraestrutura parece saudável

Existe um cenário particularmente perigoso.

Todos os quatro indicadores permanecem dentro da normalidade.

A infraestrutura responde rapidamente.

As filas estão controladas.

Não existem erros relevantes.

Mesmo assim, a qualidade das respostas produzidas pelo modelo piora silenciosamente.

Essa situação demonstra que uma plataforma de IA pode permanecer operacionalmente saudável enquanto seu comportamento funcional já apresenta degradação.

Os indicadores clássicos simplesmente não conseguem perceber esse tipo de alteração.

Deriva de qualidade

Durante a aula foram apresentados estudos demonstrando exatamente esse comportamento.

Pesquisas conduzidas por Stanford e Berkeley observaram mudanças significativas na qualidade de determinadas tarefas executadas por modelos de linguagem ao longo do tempo, mesmo sem alterações aparentes na interface disponibilizada aos usuários.

Também foram mencionados relatos públicos de degradação percebida por usuários em versões de modelos amplamente utilizados.

Independentemente das causas específicas, esses exemplos ilustram um princípio arquitetural importante.

A qualidade da Inteligência Artificial precisa ser monitorada separadamente da infraestrutura.

Latência baixa não garante boas respostas.

Ausência de erros também não.

Falhas comportamentais

Outro tipo de problema discutido durante a aula envolve alterações bruscas de comportamento dentro de uma única conversa.

Foi apresentado o caso de um chatbot corporativo que passou a produzir respostas completamente incompatíveis com sua finalidade após receber instruções inesperadas de um usuário.

Do ponto de vista da infraestrutura, nada estava errado.

A requisição foi processada normalmente.

A resposta foi gerada dentro do tempo esperado.

Todos os indicadores tradicionais permaneceram verdes.

Entretanto, o conteúdo produzido representava um comportamento completamente inadequado.

Esse tipo de incidente demonstra que observabilidade para IA precisa acompanhar não apenas desempenho técnico, mas também qualidade, aderência às políticas da organização e comportamento do modelo diante de diferentes situações.

Observabilidade orientada à qualidade

A partir desse ponto, torna-se evidente que uma arquitetura enterprise baseada em Inteligência Artificial necessita de duas camadas complementares de observabilidade.

A primeira continua responsável pela infraestrutura tradicional.

Ela monitora disponibilidade, desempenho, utilização de recursos e estabilidade operacional.

A segunda acompanha especificamente o comportamento da Inteligência Artificial.

Nela são registrados prompts, versões, documentos recuperados, modelos utilizados, qualidade das respostas e demais metadados necessários para compreender como cada decisão foi produzida.

Somente a combinação dessas duas perspectivas permite investigar corretamente incidentes em plataformas AI-First.

Sem essa visão integrada, um sistema pode aparentar estar perfeitamente saudável do ponto de vista operacional enquanto sua capacidade de produzir respostas confiáveis já começou a se deteriorar silenciosamente.

Capítulo 03: Arquitetura Enterprise (PT-3)

Nos capítulos anteriores, ampliei a arquitetura AI-First para um cenário enterprise, no qual diversos estudos clínicos compartilham a mesma infraestrutura. Em seguida, mostrei que observabilidade deixa de ser apenas monitoramento de servidores e passa a acompanhar também o comportamento dos modelos, dos prompts e da qualidade das respostas produzidas.

Existe, entretanto, outra decisão arquitetural igualmente importante.

Depois de definir como observar a plataforma, preciso decidir onde cada componente será executado.

Essa pergunta parece simples, mas muda completamente o custo operacional, a latência percebida pelo usuário, a escalabilidade do sistema e até mesmo sua capacidade de atender requisitos regulatórios.

Neste capítulo, analiso três modelos de implantação amplamente utilizados em arquiteturas AI-First: **Kubernetes**, **Serverless** e **Edge Computing**. O objetivo não é identificar qual deles é superior, mas compreender qual problema cada um resolve e como eles podem coexistir dentro de uma mesma plataforma corporativa. O conteúdo deste capítulo foi desenvolvido com base na aula correspondente do módulo.

A escolha da infraestrutura também é uma decisão arquitetural

Durante muito tempo, decisões sobre infraestrutura eram tratadas como responsabilidade exclusiva das equipes de operações.

Na arquitetura AI-First, essa separação deixa de existir.

Escolher onde um componente será executado influencia diretamente custo, desempenho e experiência do usuário.

Por esse motivo, essa decisão passa a fazer parte da própria arquitetura da aplicação.

No Trial Forge, diferentes componentes apresentam comportamentos completamente distintos.

Alguns permanecem ativos durante todo o dia.

Outros executam apenas em momentos específicos.

Existem ainda componentes cuja principal preocupação não é desempenho, mas privacidade dos dados processados.

Esperar que todos utilizem exatamente o mesmo modelo de implantação produz desperdícios inevitáveis.

Kubernetes continua importante

O primeiro modelo analisado é o Kubernetes.

Ao longo dos módulos anteriores ele já apareceu como base para execução da plataforma enterprise.

Sua principal característica consiste em manter os serviços permanentemente disponíveis.

Essa abordagem funciona muito bem quando determinado componente recebe tráfego constante ao longo do dia.

Nessas situações, manter instâncias sempre prontas elimina atrasos relacionados à inicialização da aplicação e permite responder imediatamente às novas requisições.

Entretanto, essa mesma característica também representa sua principal limitação.

Infraestrutura permanentemente ativa continua consumindo recursos mesmo quando não existe trabalho sendo executado.

Quando Kubernetes deixa de ser a melhor opção

Um componente que recebe milhares de requisições por hora naturalmente justifica permanecer ativo continuamente.

O mesmo raciocínio deixa de fazer sentido quando determinado serviço trabalha apenas em horários específicos.

Imagine um componente responsável por processar uma grande quantidade de documentos às nove horas da manhã e permanecer praticamente ocioso durante o restante do dia.

Mesmo sem executar nenhuma tarefa relevante, a infraestrutura continua reservada e gerando custos.

Esse comportamento demonstra que manter todos os componentes permanentemente ativos nem sempre representa a alternativa mais eficiente.

É justamente nesse ponto que surge o segundo modelo de implantação.

Serverless

O modelo **Serverless** parte de um princípio diferente.

Em vez de manter infraestrutura constantemente disponível, o ambiente é criado apenas quando uma requisição realmente precisa ser processada.

Quando não existe utilização, a infraestrutura pode reduzir sua capacidade até chegar praticamente a zero.

O principal benefício dessa abordagem está na cobrança baseada no trabalho efetivamente realizado.

O sistema deixa de consumir recursos continuamente apenas por permanecer disponível.

Esse comportamento torna-se particularmente interessante para componentes utilizados de forma esporádica ou sujeitos a grandes variações de demanda.

O problema do cold start

Essa flexibilidade, entretanto, possui um custo.

Sempre que uma nova requisição chega após um período de inatividade, a plataforma precisa reconstruir todo o ambiente necessário para executar aquela tarefa.

Esse processo recebe o nome de **cold start**.

Em aplicações simples, esse atraso normalmente passa despercebido.

No contexto de Inteligência Artificial, porém, o impacto pode ser significativamente maior.

Além da criação da infraestrutura, ainda existe a necessidade de inicializar o ambiente de inferência e carregar o próprio modelo para memória, etapa que costuma representar o maior gargalo de todo o processo. Esse comportamento

foi discutido utilizando como referência ambientes de inferência em nuvem voltados para IA.

As etapas do cold start

Durante a aula, o cold start foi apresentado como uma sequência de operações.

Primeiro ocorre o provisionamento da infraestrutura.

Depois a imagem da aplicação é carregada.

Na sequência acontece a inicialização do mecanismo responsável pela inferência.

Somente então o modelo propriamente dito é carregado para memória, processo que normalmente movimenta grandes volumes de dados para a GPU.

Essa última etapa costuma concentrar a maior parte do tempo necessário para que o serviço esteja efetivamente pronto para responder.

Por esse motivo, reduzir o tempo de carregamento do modelo tornou-se um dos principais objetivos das plataformas modernas de inferência.

Estratégias de mitigação

Diversas técnicas procuram minimizar esse atraso inicial.

Uma delas consiste na utilização de modelos quantizados.

Ao reduzir a quantidade de bits utilizada para representar os pesos do modelo, diminui-se também o volume de dados transferidos para memória.

Como consequência, o tempo necessário para disponibilizar o serviço tende a diminuir.

Outra estratégia discutida durante a aula utiliza snapshots de memória.

Em vez de reconstruir completamente o ambiente a cada inicialização, parte do estado previamente carregado é reaproveitada, reduzindo significativamente o tempo necessário para disponibilizar novamente a aplicação.

Essas técnicas não eliminam o cold start, mas tornam seu impacto muito menor em ambientes de produção.

Plataformas Serverless para IA

O modelo Serverless deixou de representar apenas uma proposta conceitual.

Atualmente diversas plataformas já oferecem esse tipo de funcionamento para cargas de Inteligência Artificial.

Durante a aula foram utilizados exemplos como o AWS Bedrock, que abstrai completamente o gerenciamento da infraestrutura e cobra conforme a utilização dos modelos, além do Google Cloud Run, que incorporou suporte para GPUs em cargas de IA, permitindo escalar recursos conforme a demanda.

Independentemente da plataforma utilizada, o princípio permanece o mesmo.

A infraestrutura deixa de permanecer permanentemente ativa e passa a existir apenas enquanto existe trabalho a ser executado.

Edge Computing

O terceiro modelo apresentado resolve um problema completamente diferente.

Enquanto Kubernetes e Serverless discutem como administrar infraestrutura em servidores, o **Edge Computing** desloca parte do processamento para muito mais próximo do usuário.

Em alguns cenários, esse processamento acontece em um datacenter localizado na borda da rede.

Em outros, ocorre diretamente no dispositivo utilizado pelo próprio usuário.

O objetivo principal deixa de ser economia de infraestrutura.

Passa a ser reduzir a distância física entre quem solicita o processamento e quem realmente executa a inferência.

O custo da distância

Mesmo que um servidor central seja extremamente rápido, ainda existe um tempo inevitável para que a informação viaje pela rede.

Quando aplicações dependem de respostas praticamente instantâneas, essa distância passa a representar parte importante da latência percebida.

Edge Computing atua exatamente nesse ponto.

Ao aproximar o processamento do usuário, reduz-se o tempo gasto com deslocamento dos dados pela infraestrutura de comunicação.

Esse benefício não pode ser obtido apenas aumentando a capacidade de processamento de um servidor central.

Trata-se de uma limitação física relacionada à própria localização da infraestrutura.

Privacidade como motivação

Existe ainda uma segunda razão para utilizar Edge.

Nem todo dado deveria sair do dispositivo onde foi produzido.

Em aplicações que manipulam informações sensíveis, processar localmente reduz significativamente a exposição desses dados.

Durante a aula foram apresentados exemplos de plataformas capazes de executar modelos diretamente em dispositivos móveis, utilizando versões compactas dos modelos originais para preservar privacidade e reduzir dependência de conectividade permanente.

Esse comportamento demonstra que Edge não compete diretamente com Kubernetes ou Serverless.

Ele atende uma restrição diferente.

Edge não elimina governança

Outro aspecto importante destacado durante a aula diz respeito à auditoria.

Processar informações localmente não elimina a necessidade de manter registros centralizados das decisões produzidas pela plataforma.

No contexto do Trial Forge, por exemplo, a trilha de auditoria continua sendo armazenada em um repositório central.

O que permanece no dispositivo é apenas o processamento da informação sensível.

Os registros necessários para comprovar posteriormente o funcionamento da arquitetura continuam sincronizados com a infraestrutura principal.

Esse comportamento preserva simultaneamente privacidade e rastreabilidade.

Um modelo híbrido

Um dos pontos mais importantes discutidos durante a aula é que arquiteturas enterprise raramente utilizam apenas um único modelo de implantação.

Na prática, diferentes componentes apresentam necessidades distintas.

Serviços submetidos a carga constante podem permanecer em Kubernetes.

Componentes utilizados ocasionalmente tornam-se excelentes candidatos para execução Serverless.

Já funcionalidades que exigem baixa latência ou forte proteção dos dados podem ser deslocadas para Edge.

O objetivo não consiste em escolher uma tecnologia vencedora.

Consiste em posicionar corretamente cada componente conforme suas características.

Casos apresentados

Durante a aula foram discutidos exemplos de organizações que adotam exatamente essa estratégia híbrida.

Um caso apresentado mostrou uma empresa utilizando processamento Serverless para cargas elásticas de treinamento e previsão, enquanto mantinha a inferência responsável pelo atendimento em tempo real executando continuamente sobre Kubernetes.

Outro exemplo destacou ambientes hospitalares executando modelos localmente dentro da própria infraestrutura para preservar dados clínicos sensíveis.

Embora pertençam a domínios diferentes, ambos ilustram o mesmo princípio arquitetural.

A escolha da infraestrutura depende do comportamento da carga de trabalho, do custo esperado, da latência aceitável e das restrições de privacidade envolvidas.

Um framework de decisão

Em vez de perguntar qual tecnologia deve ser utilizada por toda a plataforma, a arquitetura passa a responder outra questão.

Qual tecnologia atende melhor este componente específico?

Quando a carga é contínua e previsível, Kubernetes tende a oferecer maior estabilidade.

Quando o uso é esporádico ou altamente variável, Serverless normalmente apresenta melhor relação entre custo e utilização.

Quando a distância física ou a confidencialidade dos dados tornam-se fatores determinantes, Edge passa a representar a alternativa mais adequada.

Esse raciocínio permite construir arquiteturas híbridas nas quais diferentes modelos convivem de forma complementar.

Aplicando ao Trial Forge

No Trial Forge, essa estratégia torna-se particularmente clara.

Os serviços centrais da plataforma, responsáveis por orquestração, autenticação e componentes compartilhados, permanecem executando continuamente em Kubernetes devido ao volume constante de utilização.

Processamentos ocasionais, como determinadas tarefas de apoio ou rotinas executadas sob demanda, representam candidatos naturais ao modelo Serverless, reduzindo custos quando permanecem longos períodos sem utilização.

Já funcionalidades relacionadas ao processamento de informações altamente sensíveis podem beneficiar-se da execução em Edge, preservando privacidade sem abrir mão da sincronização da trilha de auditoria.

Ao final deste capítulo, torna-se evidente que a arquitetura enterprise não escolhe entre Kubernetes, Serverless ou Edge como alternativas concorrentes. Cada um desses modelos resolve um problema específico. O verdadeiro trabalho do arquiteto consiste em compreender o comportamento de cada componente da plataforma e posicioná-lo no ambiente de execução que oferece o melhor equilíbrio entre custo, desempenho, escalabilidade, latência e governança.

Capítulo 04: Arquitetura Enterprise (PT-4)

Ao longo deste módulo, construí uma visão completa de uma plataforma AI-First executando em ambiente corporativo. Inicialmente, organizei a infraestrutura compartilhada em torno de gateway, orquestração, serviços comuns e observabilidade. Em seguida, analisei como monitorar não apenas servidores e APIs, mas também prompts, modelos e qualidade das respostas produzidas. Depois, estudei diferentes estratégias de implantação utilizando Kubernetes,

Serverless e Edge Computing, compreendendo que cada uma atende necessidades específicas de custo, escalabilidade, latência e privacidade.

Neste capítulo, reúno todas essas decisões em um último componente da arquitetura: o controle inteligente de custos.

Até este momento, estudamos como escolher modelos, como rotear requisições e como distribuir infraestrutura. Entretanto, ainda permanece uma pergunta importante.

Como impedir que uma plataforma utilizada simultaneamente por dezenas de estudos clínicos ultrapasse o orçamento previsto?

Responder corretamente às requisições não basta.

Também é necessário administrar continuamente quanto cada decisão arquitetural custa para a organização.

É justamente esse problema que encerra a disciplina. O conteúdo apresentado neste capítulo segue diretamente a aula correspondente do módulo e utiliza o Trial Forge como referência para consolidar todos os conceitos estudados.

O custo deixa de ser consequência

Nos primeiros módulos da disciplina, o custo aparecia como um dos critérios utilizados durante a escolha entre componentes determinísticos e modelos generativos.

Posteriormente, o Model Router passou a selecionar automaticamente modelos diferentes conforme a complexidade da tarefa.

Essas decisões já produziam economia significativa.

Entretanto, existe uma limitação importante nessa abordagem.

Mesmo escolhendo corretamente entre um modelo barato e outro mais sofisticado, ainda existe desperdício quando tarefas simples continuam sendo

encaminhadas diretamente para modelos mais caros apenas porque foram classificadas previamente como complexas.

Essa observação conduz naturalmente a uma estratégia ainda mais eficiente.

Da escolha para a tentativa

O Model Router toma sua decisão antes que qualquer resposta seja produzida.

Ele analisa a solicitação e escolhe previamente qual modelo deverá executá-la.

A estratégia apresentada neste capítulo segue um caminho diferente.

Em vez de tentar adivinhar antecipadamente qual modelo será necessário, a arquitetura permite que o modelo mais barato responda primeiro.

Somente depois de analisar a qualidade dessa resposta decide-se se vale a pena escalar para um modelo mais sofisticado.

Essa mudança parece sutil.

Entretanto, altera completamente a lógica do fluxo arquitetural.

O conceito de cascata

Essa estratégia recebe o nome de **Model Cascading**.

Seu funcionamento baseia-se em uma sequência de tentativas.

A arquitetura começa sempre utilizando o modelo de menor custo.

Se a resposta produzida atingir os critérios de qualidade estabelecidos previamente, ela é imediatamente aceita.

Caso contrário, a requisição sobe para um segundo modelo.

Se ainda assim a qualidade permanecer insuficiente, novos níveis podem ser acionados sucessivamente até alcançar o modelo mais sofisticado disponível.

Em vez de escolher um único modelo antes da execução, a arquitetura constrói uma sequência progressiva de decisões.

Por que a cascata economiza

O benefício dessa abordagem torna-se evidente quando observamos o comportamento típico das aplicações corporativas.

Grande parte das solicitações rotineiras não exige modelos extremamente sofisticados.

Permitir que o modelo mais simples tente responder primeiro evita utilizar recursos caros em situações nas quais eles não seriam realmente necessários.

Quando a resposta já atende aos critérios definidos pela arquitetura, não existe motivo para continuar escalando.

O modelo mais caro sequer é executado.

Essa característica diferencia a cascata do roteamento tradicional.

A economia acontece justamente porque a decisão de escalar depende da resposta produzida e não apenas da expectativa inicial sobre sua dificuldade.

O papel da confiança

Para decidir se a resposta produzida pelo primeiro modelo pode ser aceita, a arquitetura utiliza sinais de confiança.

Esses sinais representam medidas objetivas utilizadas para estimar a qualidade da resposta gerada.

Quando a confiança ultrapassa determinado limiar, considera-se que o resultado é suficientemente confiável para ser entregue ao usuário.

Caso contrário, o processamento continua utilizando um modelo mais sofisticado.

Observe que a decisão não depende exclusivamente da complexidade estimada da pergunta.

Ela considera o comportamento efetivamente observado durante a execução.

Uma decisão posterior

Essa diferença representa uma mudança importante de perspectiva.

No Model Router, toda a decisão acontece antes da geração da resposta.

Na cascata, parte da decisão ocorre depois que o modelo mais barato já executou sua tarefa.

Somente após avaliar o resultado produzido torna-se possível determinar se realmente existe necessidade de escalar para outro modelo.

Essa abordagem reduz significativamente o número de chamadas realizadas aos modelos mais caros.

Evidências apresentadas

Durante a aula foi discutido o trabalho **FrugalGPT**, que formaliza exatamente esse mecanismo de cascata entre modelos.

Os experimentos apresentados mostram que, em diferentes conjuntos de dados, a estratégia consegue preservar níveis elevados de qualidade enquanto reduz significativamente o custo operacional por meio da utilização progressiva de modelos de capacidades distintas.

Independentemente dos percentuais específicos obtidos em cada experimento, o aspecto arquitetural mais importante permanece o mesmo.

O modelo mais caro deixa de representar a primeira opção.

Ele passa a ser utilizado apenas quando os níveis anteriores demonstram que realmente não conseguiram produzir uma resposta adequada.

A cascata possui múltiplos níveis

Outro ponto importante discutido durante a aula é que a cascata não precisa limitar-se a apenas dois modelos.

Uma arquitetura pode possuir diversos níveis intermediários.

Cada camada atende um intervalo diferente de complexidade e custo.

Entretanto, isso não significa que utilizar mais níveis seja sempre a melhor decisão.

Cada novo estágio aumenta também a complexidade da manutenção.

Por esse motivo, uma arquitetura normalmente começa utilizando apenas dois níveis e evolui gradualmente apenas quando os dados de utilização demonstram que novos estágios realmente produzirão benefícios relevantes.

Limiar de escalonamento

Cada nível da cascata possui seu próprio limiar de confiança.

Não existe um único valor válido para toda a arquitetura.

Modelos diferentes apresentam comportamentos diferentes.

Consequentemente, cada um deles exige uma calibração específica.

Durante a aula foi enfatizado que esses valores não devem ser copiados de exemplos genéricos.

Eles precisam ser medidos utilizando dados representativos do domínio da aplicação.

Essa preocupação reforça um princípio recorrente ao longo de toda a disciplina.

Arquiteturas AI-First são calibradas empiricamente.

Elas não são construídas apenas com base em intuição.

Plataformas comerciais

O conceito de cascata já aparece em produtos utilizados em ambientes corporativos.

Durante a aula foram apresentados exemplos como o Azure AI Foundry, que oferece mecanismos de roteamento entre diferentes famílias de modelos conforme objetivos relacionados a custo ou qualidade, além do Amazon Bedrock, que também incorpora estratégias inteligentes para reduzir gastos utilizando modelos distintos.

Embora cada plataforma utilize implementações próprias, todas compartilham o mesmo objetivo.

Utilizar o modelo mais barato sempre que isso não comprometer a qualidade final percebida pelo usuário.

Controle por tenant

Além da escolha inteligente dos modelos, a arquitetura enterprise introduz outro mecanismo de governança.

Cada estudo clínico passa a possuir seu próprio orçamento.

Essa característica impede que um único projeto consuma recursos ilimitados da infraestrutura compartilhada.

Antes mesmo de qualquer modelo ser executado, a plataforma verifica se aquele tenant ainda possui orçamento disponível.

Caso o limite tenha sido atingido, a solicitação é interrompida imediatamente.

Essa decisão acontece antes de qualquer processamento mais caro.

Bloquear antes de gastar

Essa ordem possui uma justificativa arquitetural importante.

Se a arquitetura verificasse o orçamento somente após iniciar parte do processamento, recursos já teriam sido consumidos desnecessariamente.

Por esse motivo, o controle financeiro torna-se uma das primeiras verificações executadas pelo gateway.

A lógica é exatamente a mesma observada anteriormente no Semantic Cache.

Sempre que existir uma condição capaz de interromper completamente a requisição, ela deve ser avaliada o mais cedo possível.

Essa estratégia evita desperdiçar processamento em fluxos que inevitavelmente serão encerrados.

Integrando a cascata ao Trial Forge

No protótipo apresentado durante a aula, essa integração torna-se bastante clara.

Consultas rotineiras relacionadas ao protocolo clínico iniciam sempre pelo modelo mais econômico.

Caso os indicadores de confiança permaneçam acima do limiar estabelecido, a resposta é entregue imediatamente.

Quando esses indicadores revelam incerteza suficiente, a arquitetura escala automaticamente para um modelo mais sofisticado.

Existe, entretanto, uma exceção importante.

A síntese do Clinical Study Report continua sendo encaminhada diretamente ao modelo mais avançado.

Essa decisão não depende da confiança.

Ela decorre do elevado risco associado à geração desse documento.

Essa mesma lógica já havia sido estabelecida anteriormente durante o estudo do Approval Gate.

Dois sinais de confiança

Durante a implementação do protótipo, um detalhe interessante tornou-se evidente.

Utilizar apenas um único indicador de confiança não era suficiente.

A arquitetura passou a considerar simultaneamente dois sinais diferentes.

O primeiro está relacionado à qualidade da recuperação das informações utilizadas como contexto.

O segundo mede a confiança associada à própria resposta produzida pelo modelo.

Essa combinação evita situações nas quais uma resposta aparentemente consistente tenha sido construída sobre informações recuperadas de forma inadequada.

Os dois indicadores trabalham juntos para decidir corretamente quando a cascata deve continuar.

Reutilizando componentes

Outro aspecto importante observado durante a demonstração é que praticamente todos os componentes utilizados já haviam sido construídos nos módulos anteriores.

A busca vetorial, o mecanismo de streaming, o Approval Gate e a trilha de auditoria permaneceram exatamente os mesmos.

A novidade consistiu apenas em reorganizar esses elementos dentro de uma arquitetura mais abrangente.

Essa observação ilustra um princípio importante de engenharia.

Uma boa arquitetura cresce por composição.

Ela não exige reconstruir continuamente componentes já consolidados.

A importância da ordem

Durante os testes apresentados na aula surgiu um ajuste arquitetural interessante.

Inicialmente, a verificação de orçamento acontecia apenas depois da classificação da intenção da requisição.

Na prática, isso significava executar parte do processamento mesmo quando o estudo já havia ultrapassado seu limite financeiro.

A solução foi simples.

Mover a verificação de orçamento para o início absoluto do fluxo.

Essa alteração eliminou processamento desnecessário e reforçou um princípio recorrente durante toda a disciplina.

Sempre que possível, as verificações mais baratas e capazes de interromper completamente o fluxo devem acontecer antes das etapas mais custosas.

O framework final

Como encerramento do módulo, foi apresentado um canvas destinado à calibração da cascata.

Esse artefato organiza três grupos principais de decisões.

O primeiro identifica os diferentes níveis de modelos utilizados pela arquitetura.

O segundo registra os limiares de confiança responsáveis por determinar quando ocorre o escalonamento.

O terceiro documenta os critérios relacionados ao controle de orçamento por tenant.

Esse documento transforma uma estratégia de otimização financeira em um conjunto explícito de decisões arquiteturais, permitindo que diferentes equipes compreendam exatamente como a plataforma distribui custo, qualidade e desempenho entre seus diversos componentes.

Encerrando a arquitetura

Ao final da disciplina, torna-se possível observar a evolução completa da arquitetura construída.

O gateway apresentado no primeiro módulo continua recebendo todas as requisições.

Os agentes especializados continuam organizando o processamento.

Os mecanismos de recuperação de contexto permanecem fundamentando as respostas.

O Approval Gate continua protegendo decisões críticas.

A observabilidade registra permanentemente todo o comportamento da plataforma.

Agora, porém, todos esses componentes passam a operar também sob uma estratégia explícita de otimização de custos.

A arquitetura deixa de apenas responder corretamente.

Ela aprende também a utilizar seus recursos de forma inteligente.

Esse é justamente o objetivo de uma arquitetura enterprise preparada para produção: combinar qualidade, governança, escalabilidade e sustentabilidade operacional em uma única plataforma AI-First.

